

## 4. AI视觉玩法

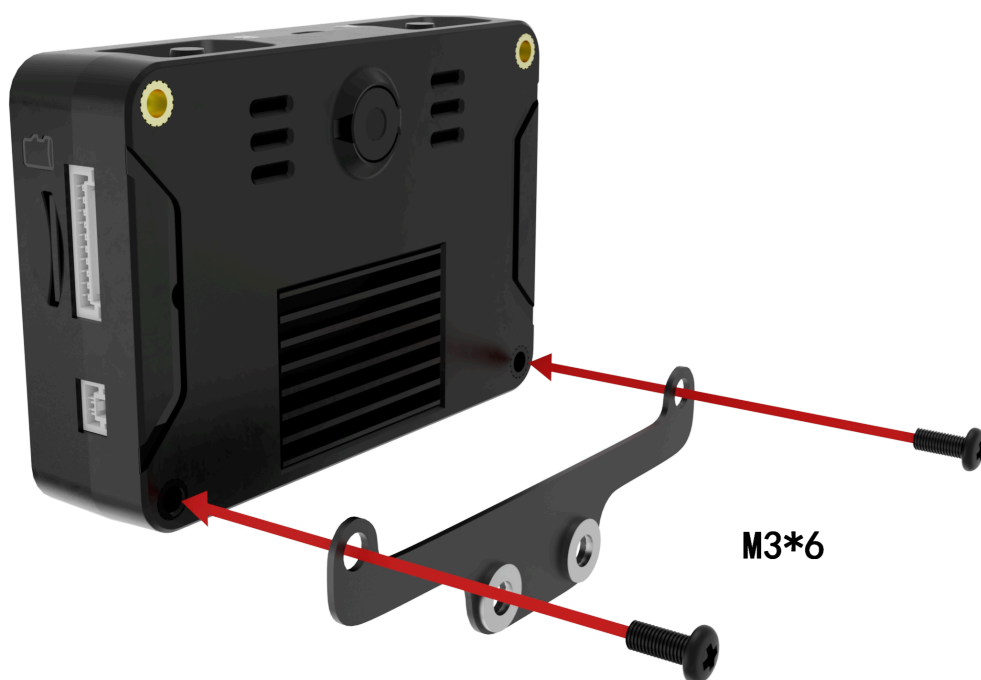
### Note

- 在开启AI视觉玩法前，需要将机械臂进行视觉偏移校准。具体步骤可参考产品资料中“\1.教程资料\1. 快速入门\01 快速入门.pdf”中的“1.4.2.6 AI视觉玩法”小节。
- 本章节课程只适用于搭配K230视觉模块的NexArm豪华版、开发板和旗舰版。

## 4.1 WonderMK视觉模块安装与配置

### 4.1.1 WonderMK的安装与接线说明

- 使用M3\*6螺丝将K230视觉模块和支架进行组装。



- 将组装好的K230视觉模块和支架固定在机械臂上。



3. 使用配套的1000mm模块线将K230视觉模块和机械臂IIC接口进行连接。



#### 4.1.2 WonderMK镜像烧录(选看)

##### ③ Note

WonderMK在出厂时已经烧录好固件，如果用户需要重新烧录固件可参考此步骤。

### 4.1.2.1 准备工作

烧写镜像将会格式化SD卡，如果SD卡有数据请提前备份。我们需要准备一个读卡器、一张32G SD卡（建议8G以上）及可联网的电脑，然后将插入SD卡的读卡器连入电脑的任意一个USB接口。

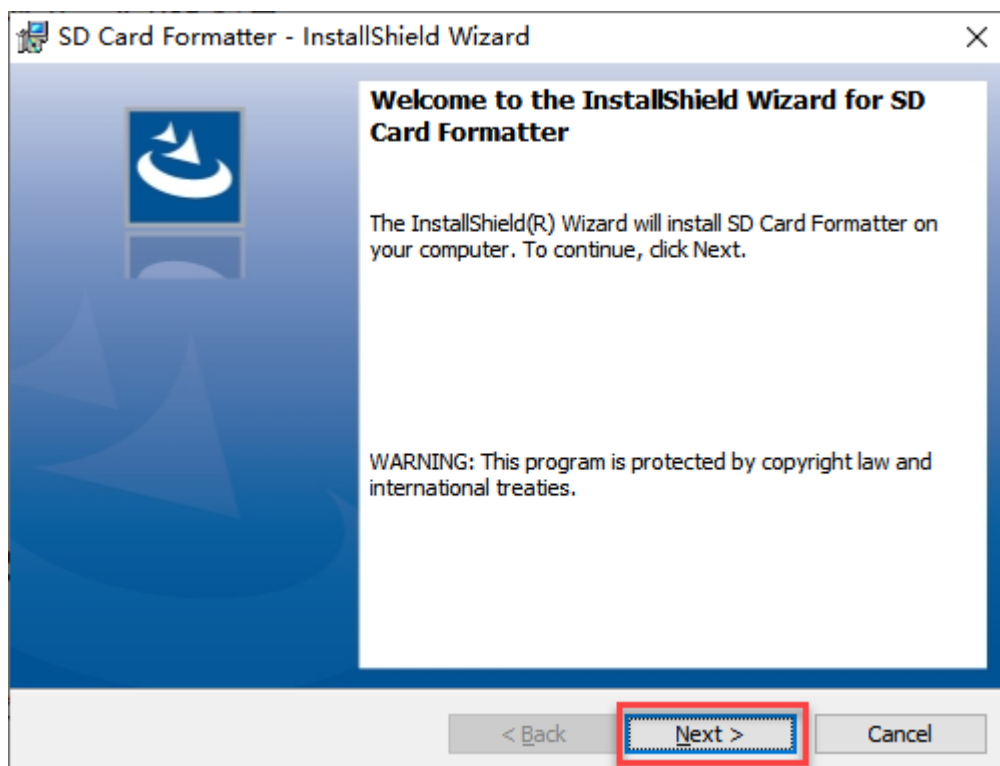


### 4.1.2.2 安装格式化SD工具

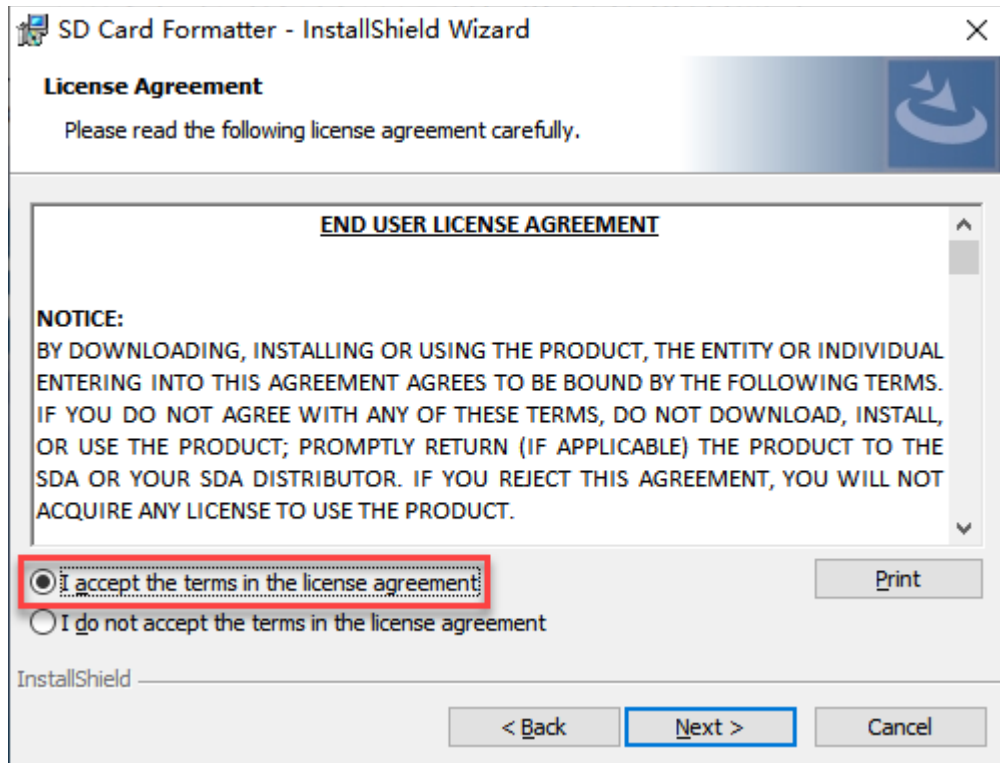
1. 我们首先安装SD卡格式化工具SD Card Formatter工具。可在产品资料中“\2.软件&工具合集\6. K230视觉模块（WonderMK）使用教程和工具\02 SD卡格式化工具”找到“SD卡格式化工具”。

 SD Card Formatter 5.0.1 Setup.exe

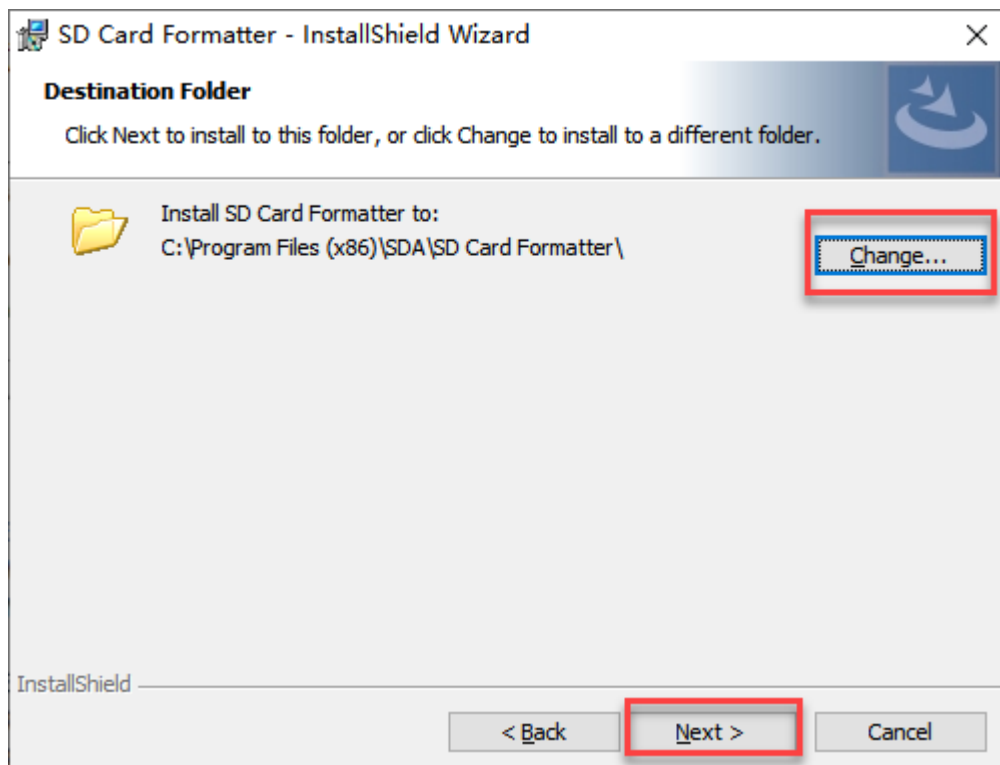
2. 安装方法如下所示，双击打开安装包后，稍等片刻接着点击“Next”。



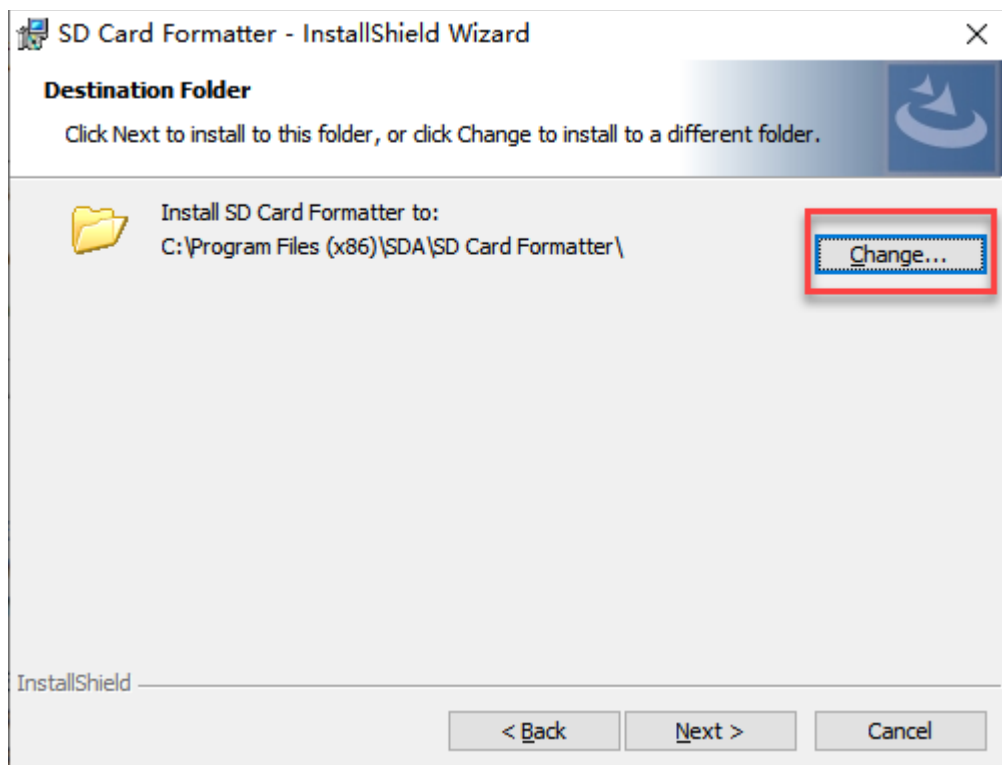
3. 勾选上同意协议，接着点击“Next”。



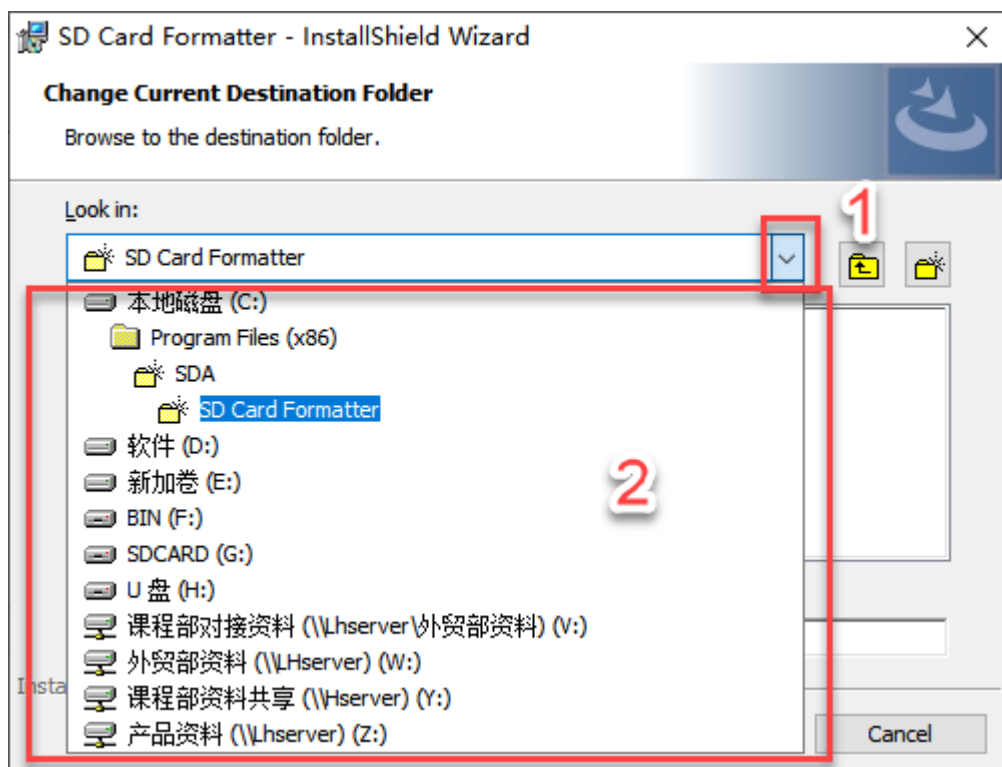
4. 我们可以点击Change...选择安装的目录，也可以直接点击“Next”进入安装。



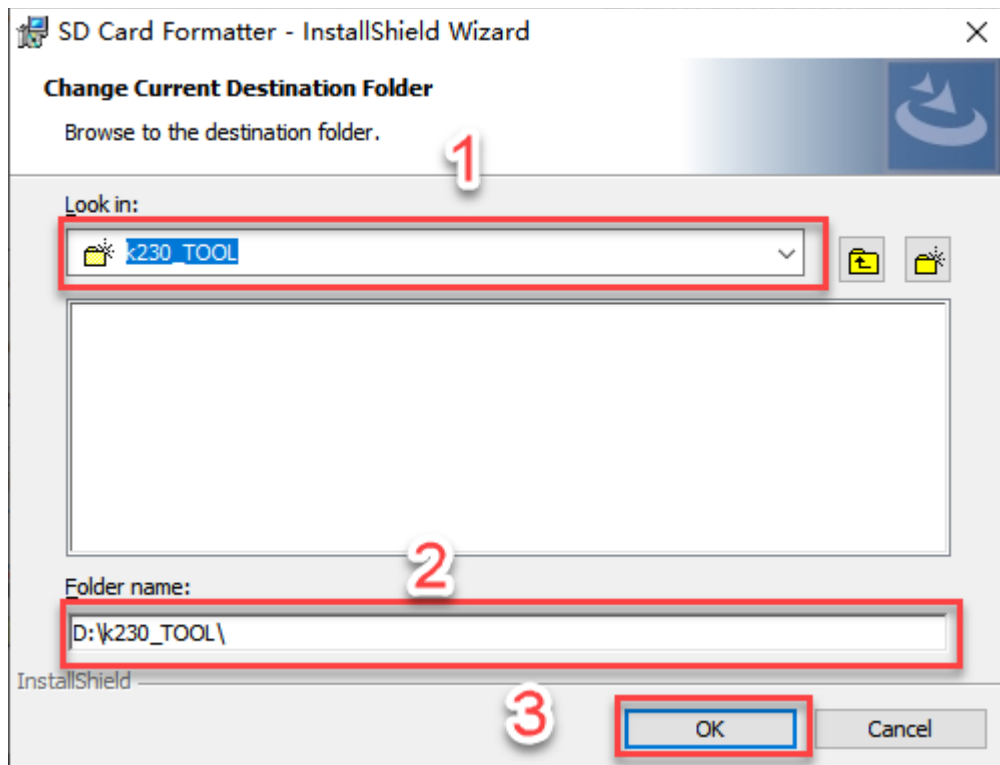
5. 我们这里以安装修改路径为例子，点击Change...



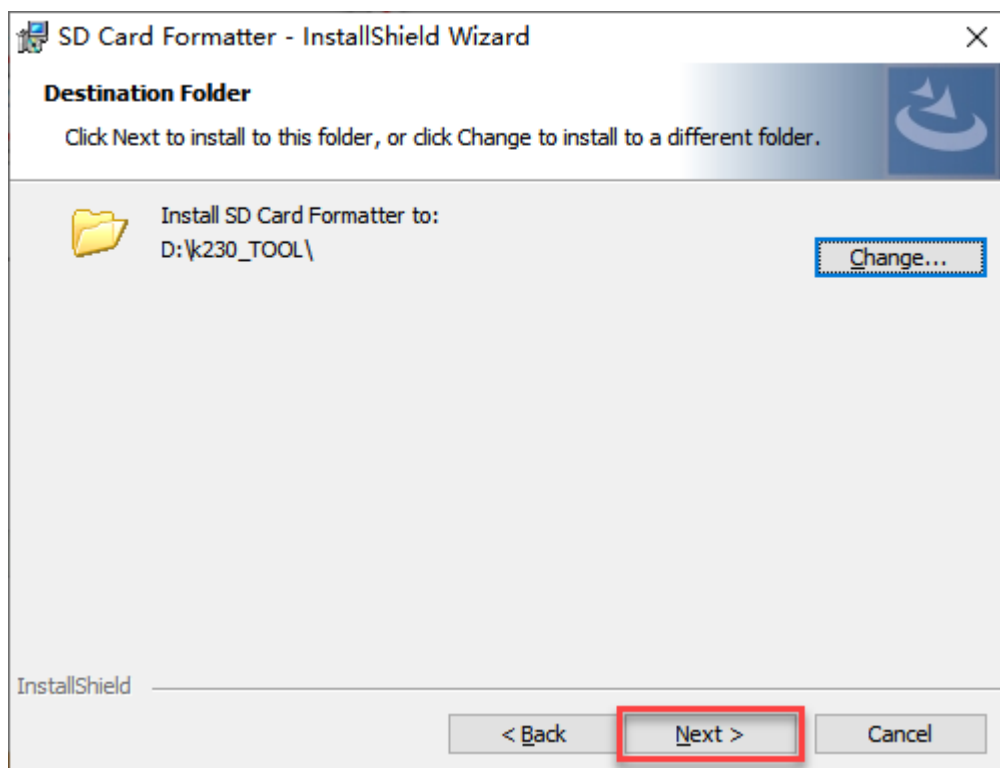
6. 点击序号1的图标下拉出目录栏，在2选择想要安装的路径。



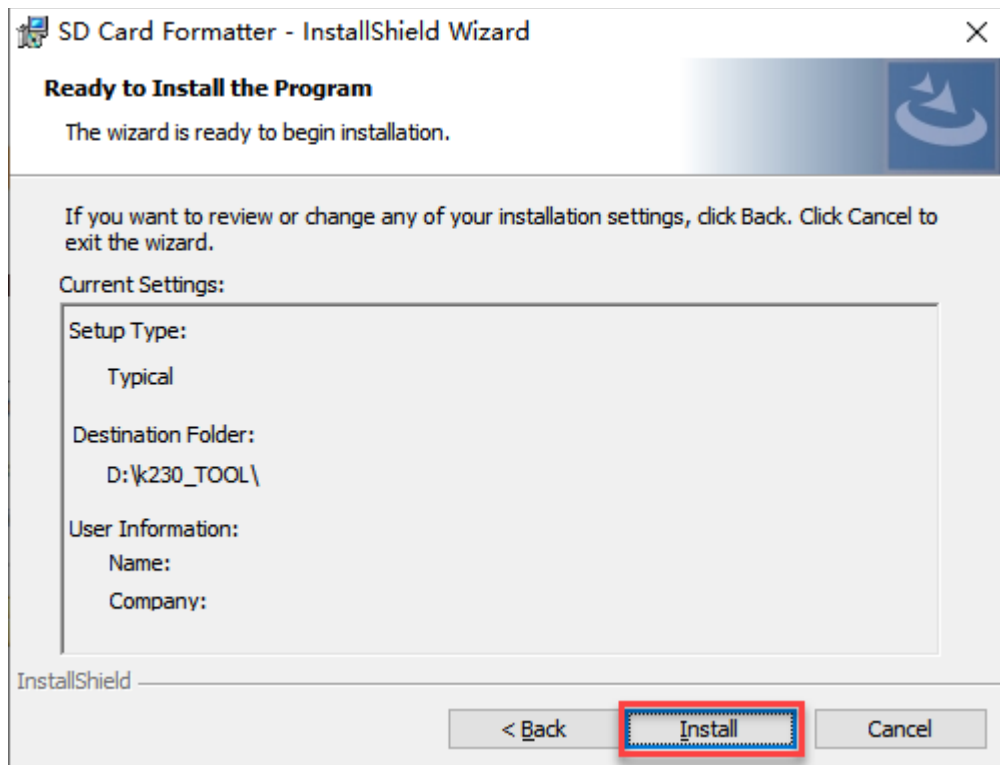
7. 可以在序号1中选择下载的路径，也可以在序号2中填写路径。在选择完成后点击序号3确定。



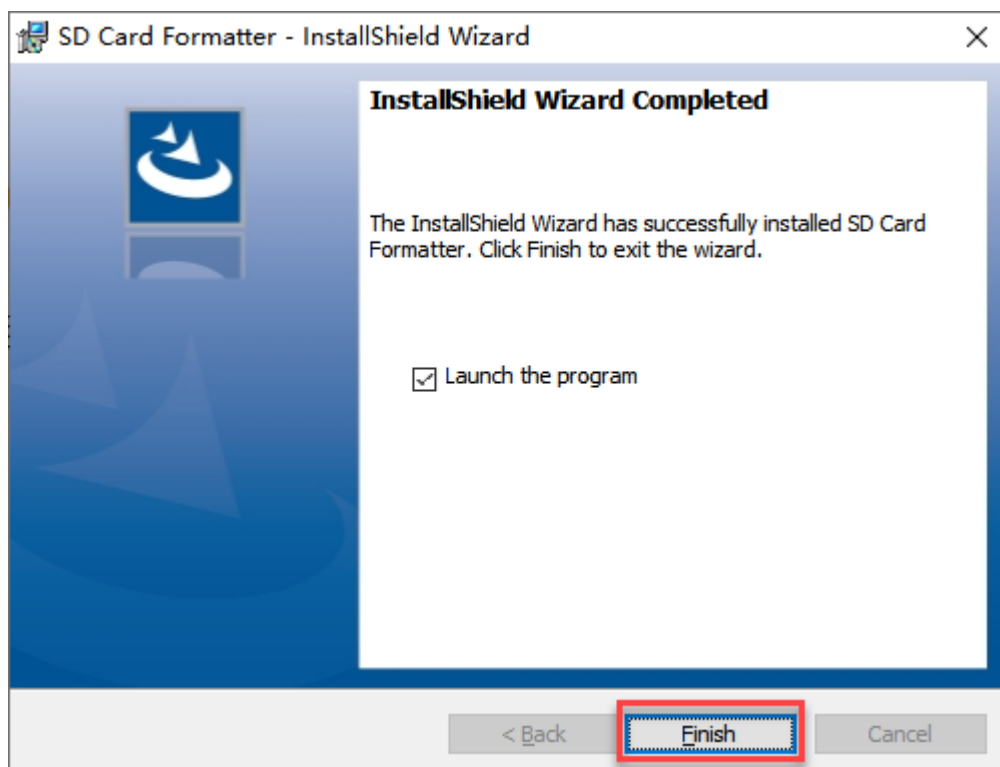
8. 点击“Next”继续安装。



9. 确认安装的目录地址，确认完毕后点击Install即可开始安装。

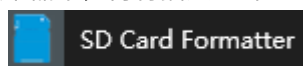


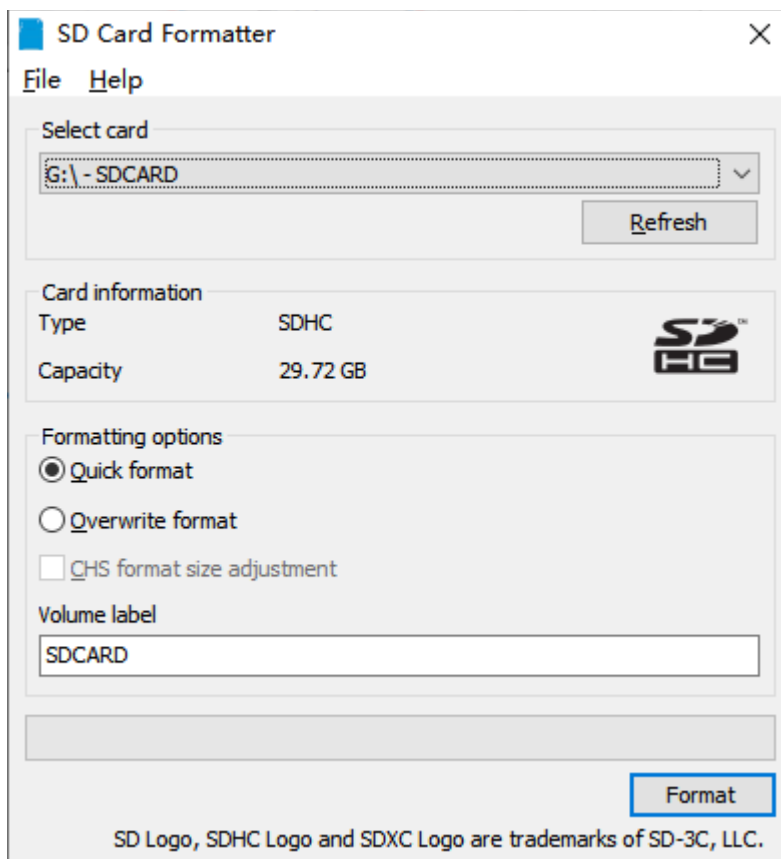
10. 点击“Finish”完成安装。



#### 4.1.2.3 格式化SD卡

1. 将SD卡插入读卡器中，再将插入SD卡的读卡器连入电脑的任意一个USB接口。打开格式化工具“SD Card Formatter”。

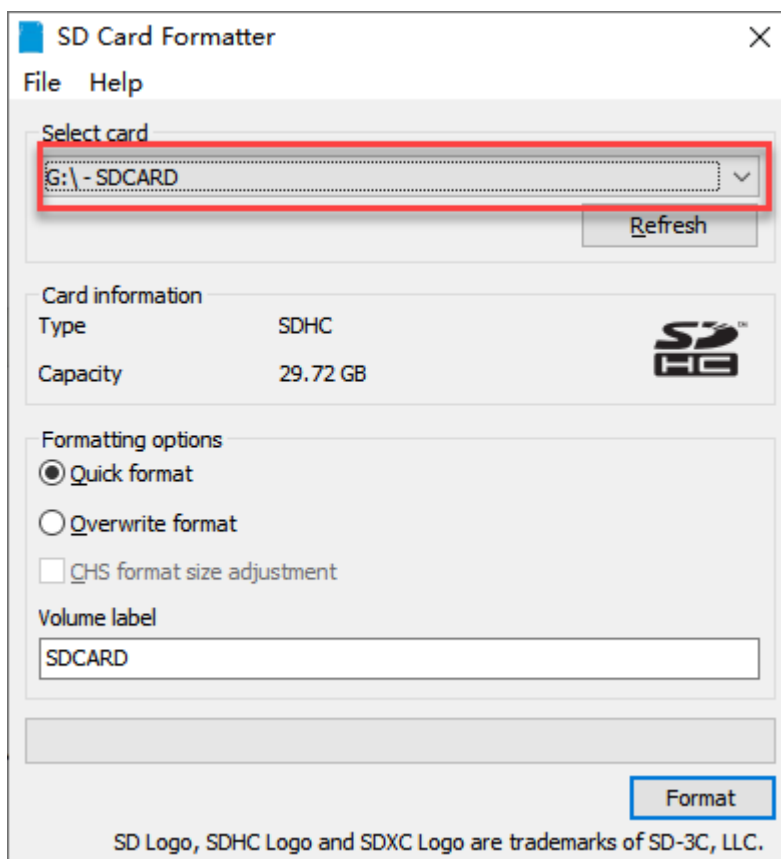




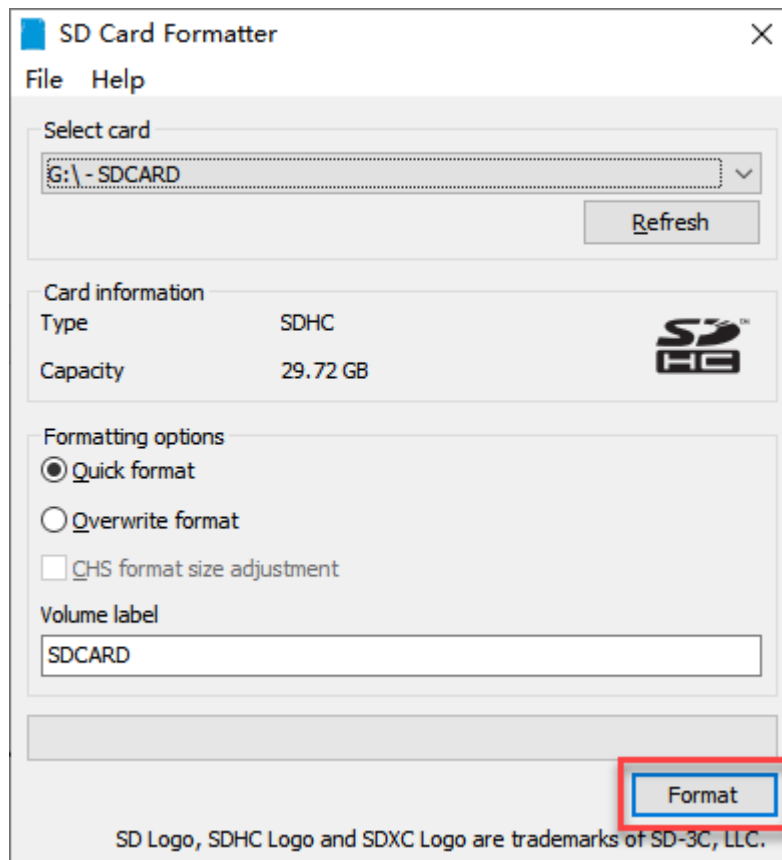
2. 选择对应的盘符（如果不知道哪一个SD卡盘符，可以先拔掉再打开"我的电脑",当插入读卡器后电脑检测到多出来的盘符就是SD卡的盘符）。

**Note**

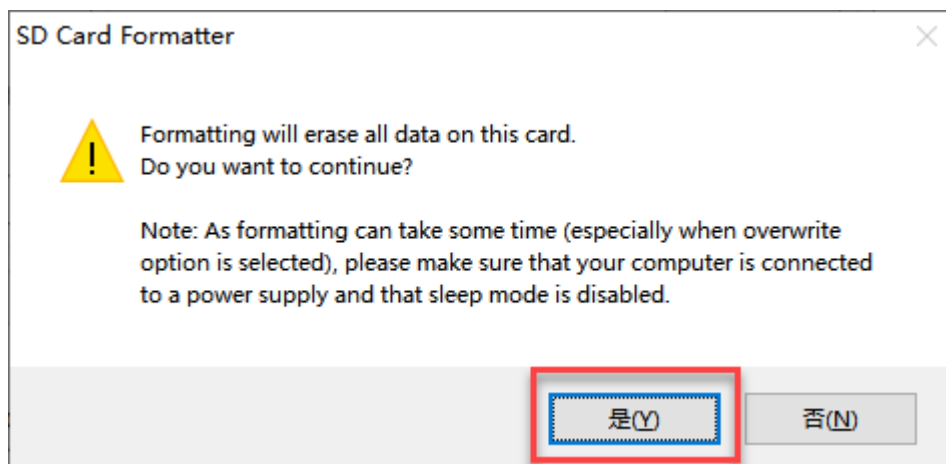
如果SD卡被系统检测出自带多个盘符，任选一个即可。



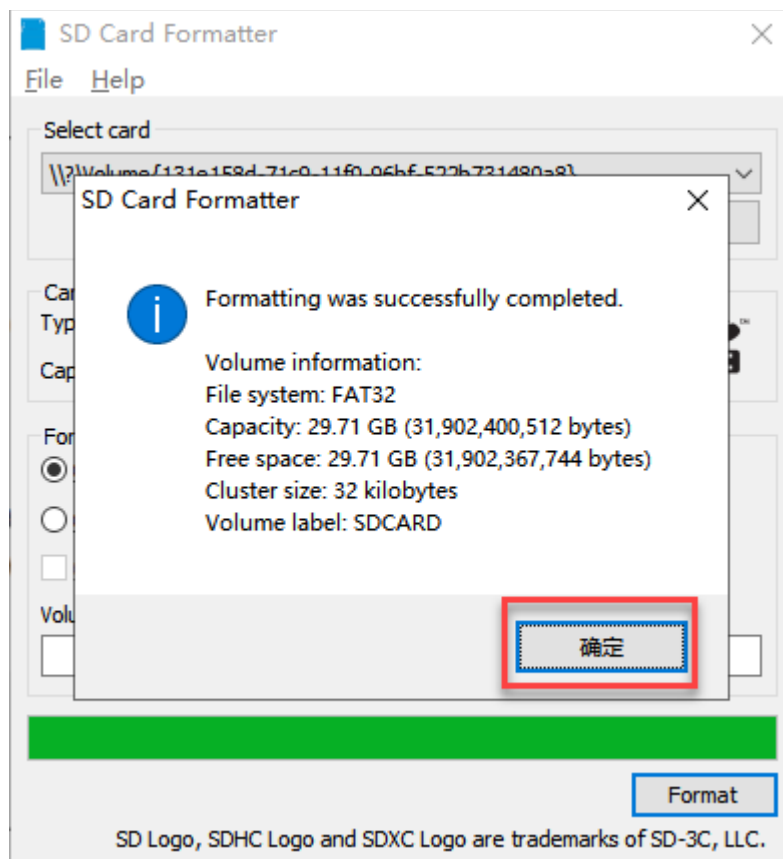
3. 点击“**Format**”,开始格式化。



4. 点击“是”，确认并开始格式化。



5. 点击“**确定**”，完成SD卡格式化。



#### 4.2.2.4 固件烧录

1. 将格式化好的SD卡插入读卡器中，再将插入SD卡的读卡器连入电脑的任意一个USB接口。打开 2.软件工具\2.4 镜像烧录工具 路径下的“rufus” 镜像烧录工具。

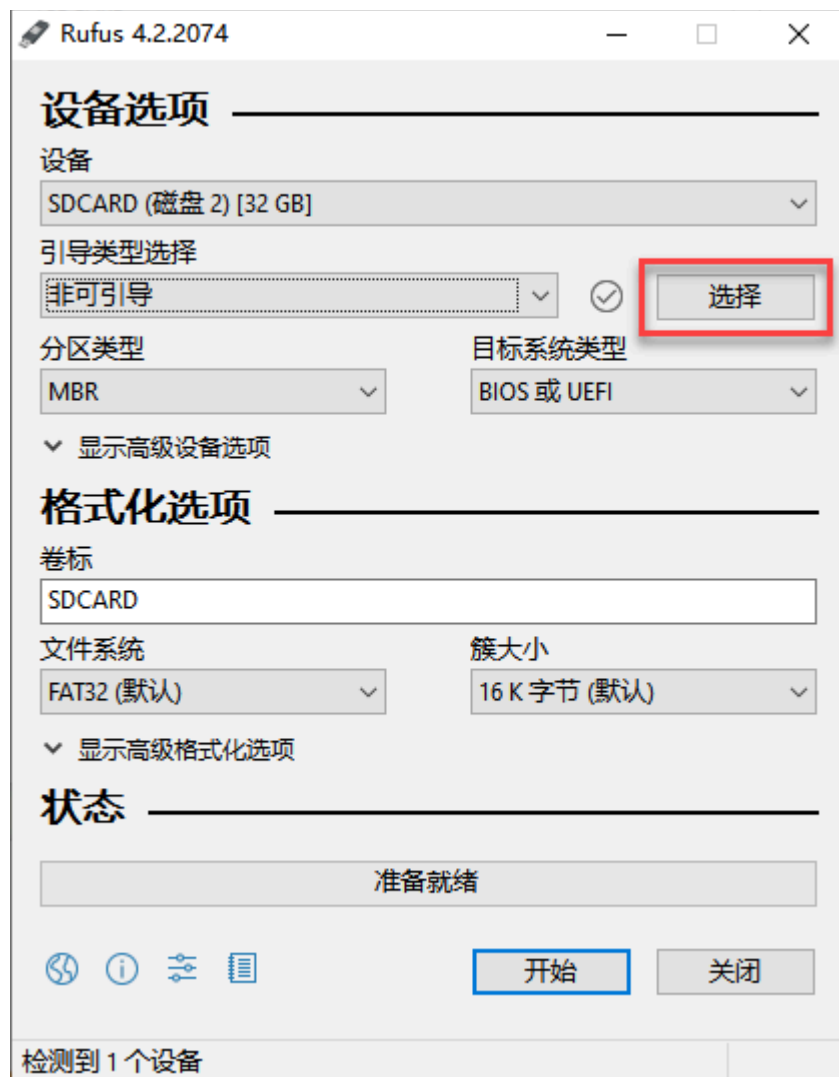




2. 选择对应的SD卡盘符。



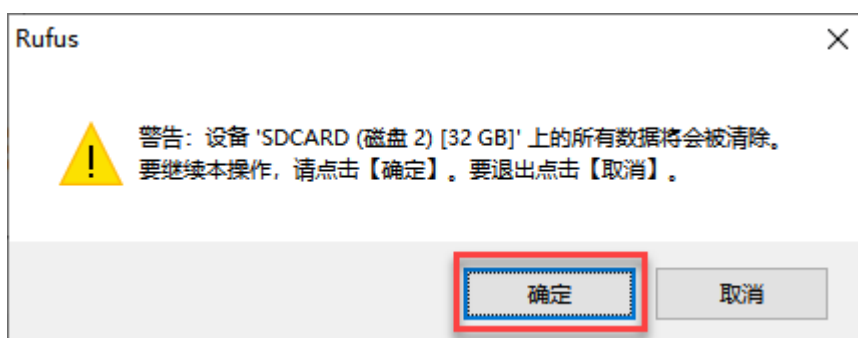
3. 点击“选择”，选择烧录的镜像。



4. 选择"\2.软件&工具合集\6. K230视觉模块（WonderMK）使用教程和工具\04 K230视觉模块镜像"路径下的 **WonderMK\_v2.0.0\_20260514.img.gz** 镜像，再点击打开。
5. 点击 "开始"，进行固件的烧录。



6. 在弹出的对话框中选择“确定”，稍等片刻即可完成烧录。



7. 点击“关闭”，完成SD卡烧录。



## 4.2 程序下载说明（必看）

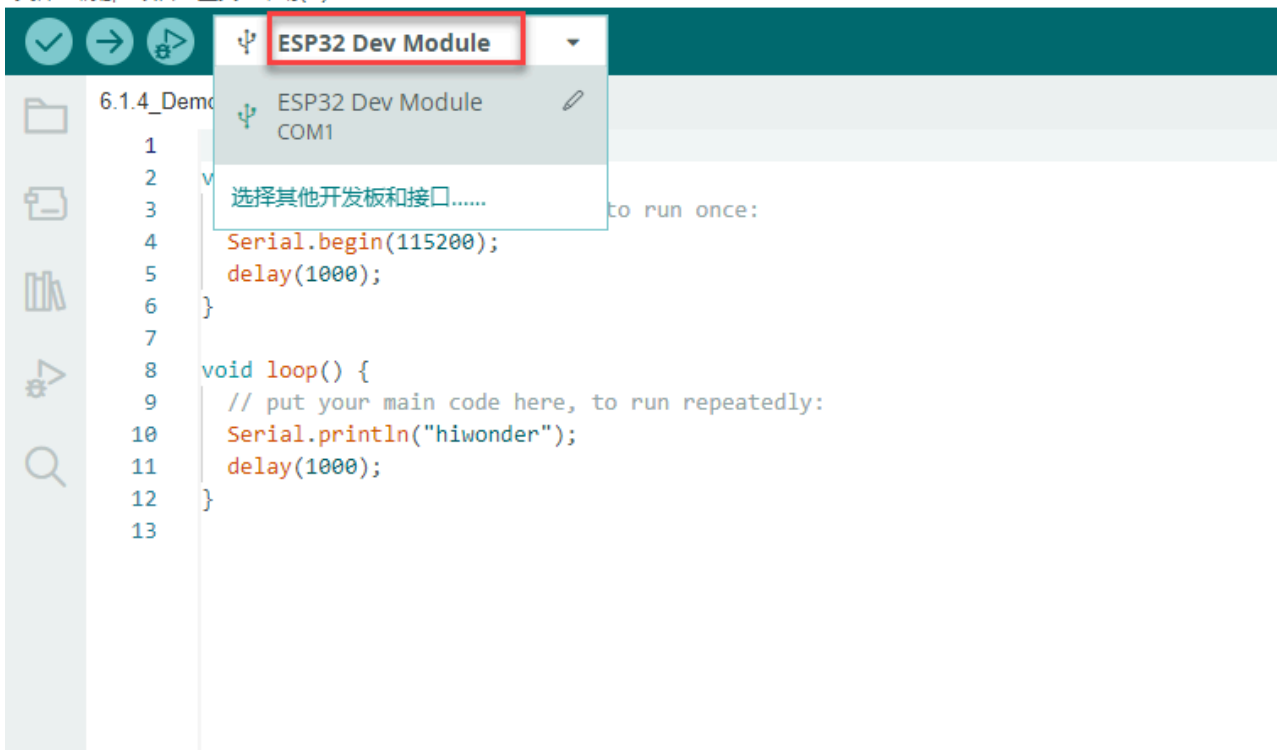
1. 这里我们以一个打印"hiwonder"字样的例程为例进行说明。在本文档02 程序源码\01 Arduino安装包及开发包\Demo文件夹中双击打开"Demo.ino"的示例程序。



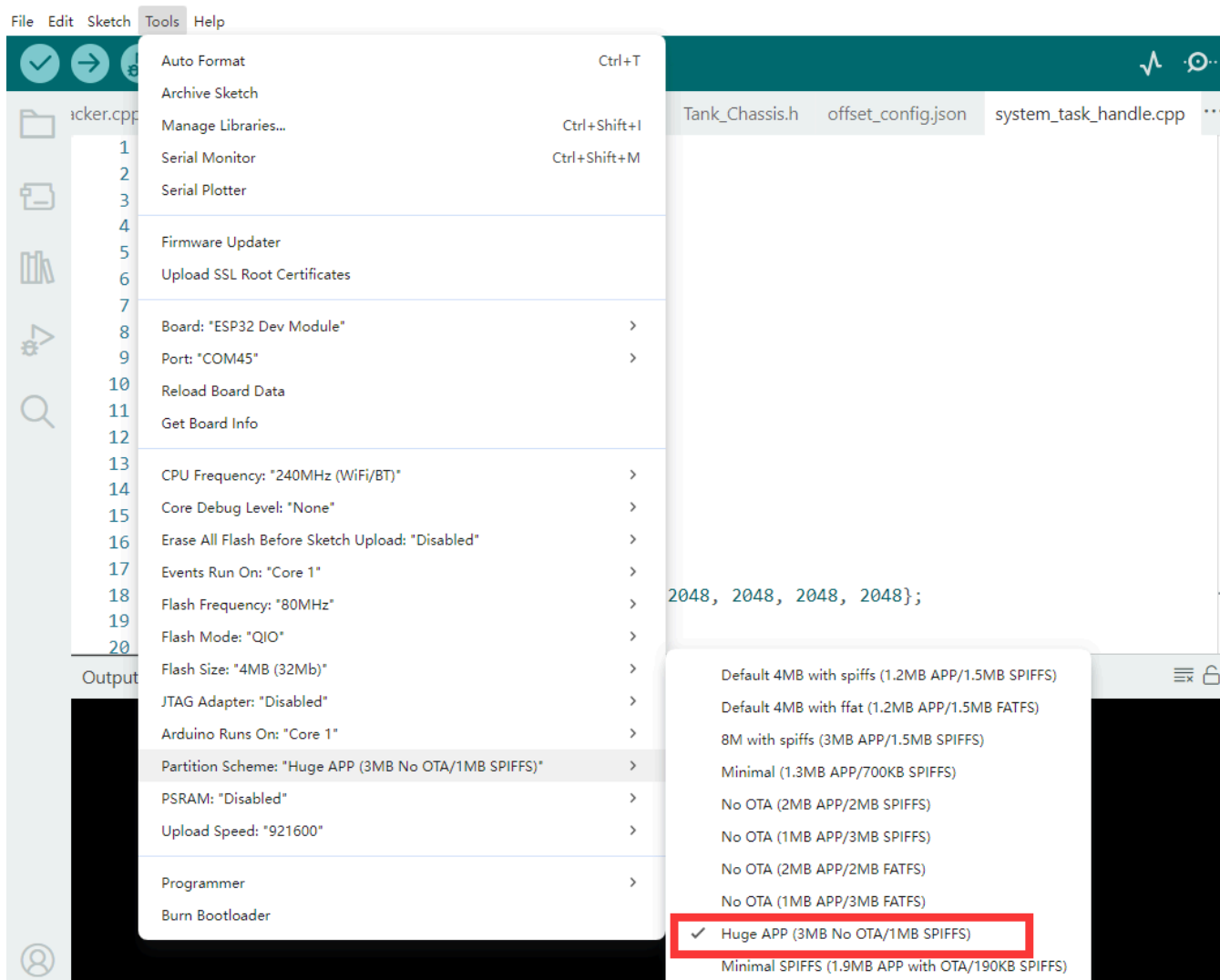
2. 将NexArm通过数据线连接至电脑。



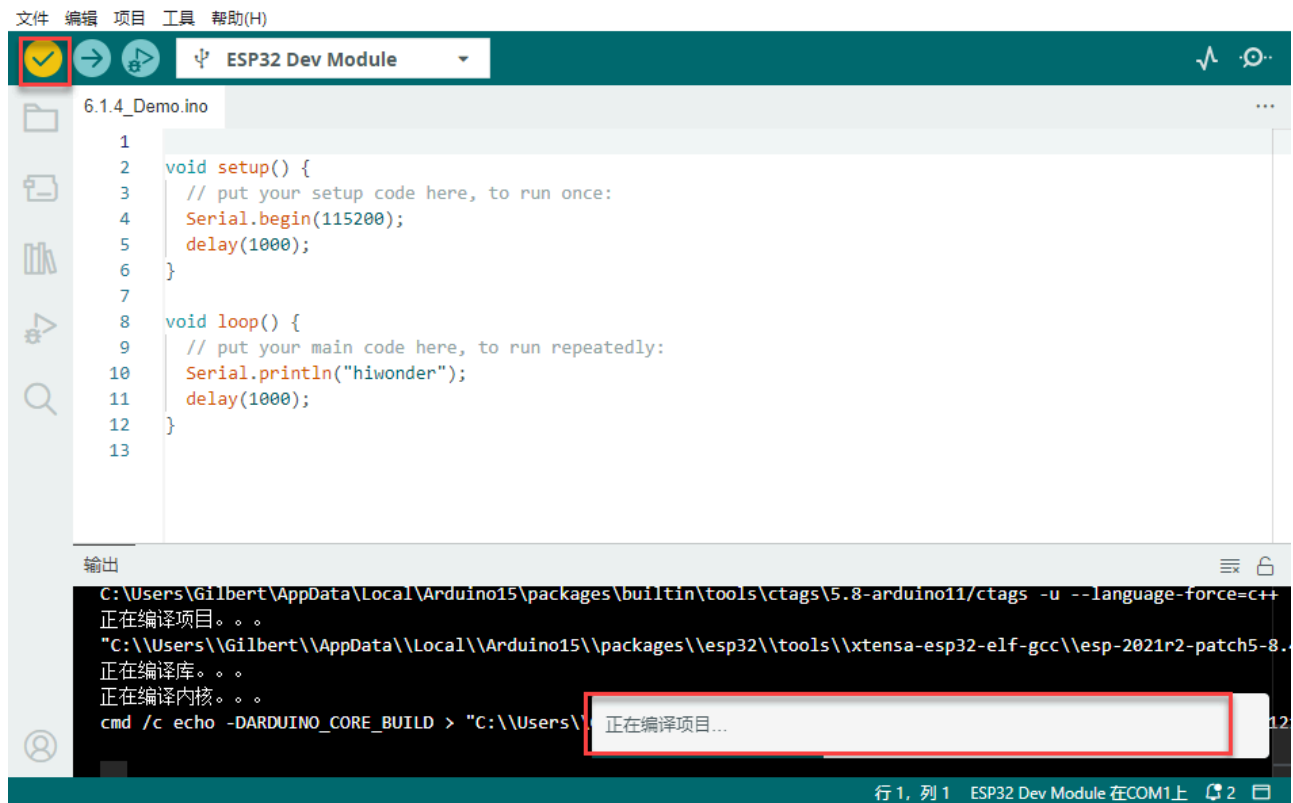
3. 在"选择开发板"选项中找到Arduino对应的开发板。(COM口不唯一，我们可以通过电脑的设备管理中查看COM号，此处以COM6为例)。



4. 点击菜单栏的"工具", 按照下图选择相应的ESP32的开发板配置:



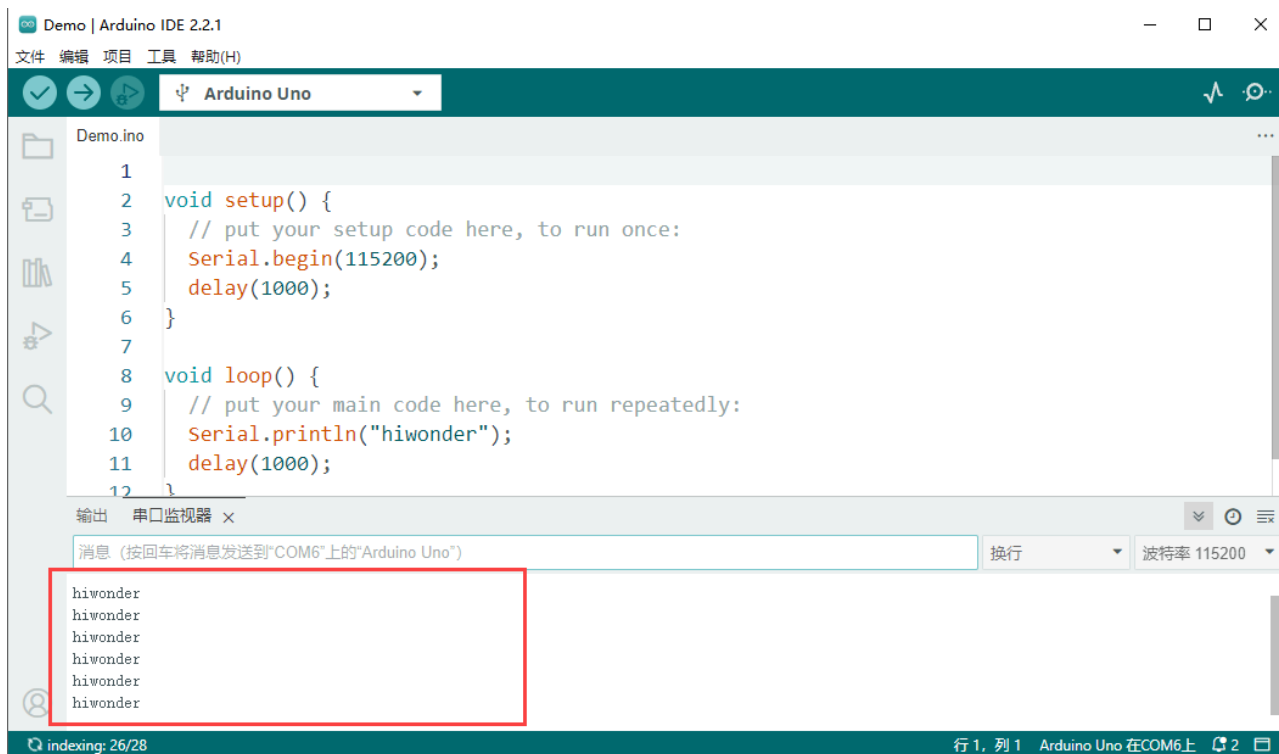
5. 点击  按钮可对程序进行编译，可以验证程序是否存在语法错误等问题。



6. 编译成功后，再点击  按钮会将程序上传到ESP32核心板上。



7. 上传完成后，点击  打开串口监视器，我们可以看到在串口监视器中打印出了"hiwonder"字样。

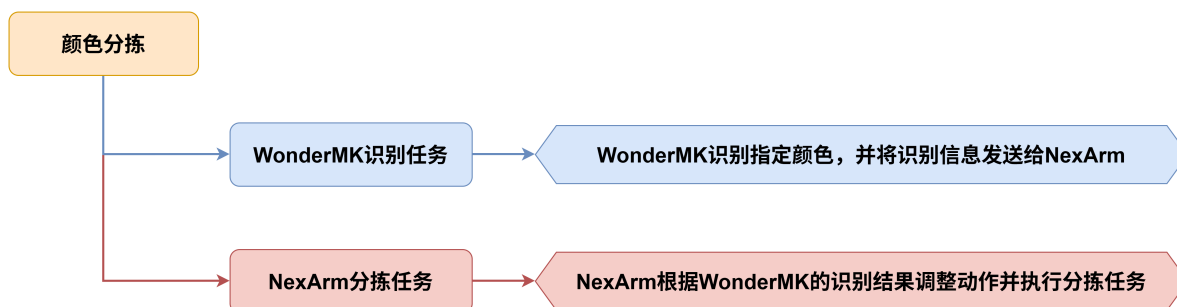


## 4.3 颜色分拣

### 4.3.1 项目介绍

颜色分拣功能通过调用WonderMK识别不同颜色的目标物体，并锁定指定颜色进行机械臂分拣任务。

### 4.3.2 实现流程



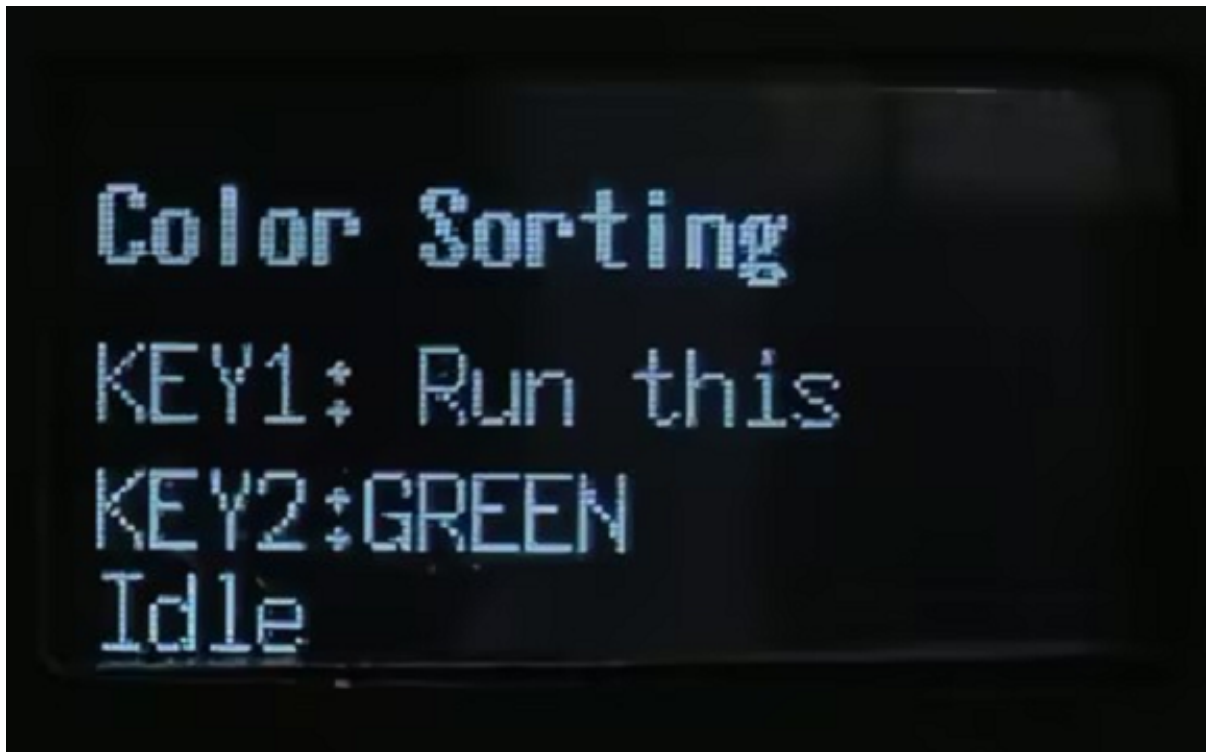
### 4.3.3 程序下载

找到本文档同目录下的“02 程序源码”文件夹，找到“01 颜色分拣”程序进行下载，具体下载步骤可参考[4.2 程序下载说明\(必看\)](#)。

### 4.3.4 运行效果

系统上电后会先完成初始化，机械臂自动回到初始位置，屏幕显示当前状态和目标颜色，随后进入等待状态。此时设备不会立刻执行抓取，而是等待用户选择颜色并发出启动指令。

实际操作时，先给设备正常上电，等待初始化完成并确认机械臂回到初始位；再将待分拣的目标物体放到摄像头可识别的区域内；如果需要切换目标颜色，可以通过 KEY2 按键循环选择；确认颜色后按下 KEY1 启动按键，机械臂便会开始搜索并抓取对应颜色的物体；一次分拣完成后，系统会自动回到等待状态，若需要继续分拣，再次启动即可。显示屏会显示对应的提示信息。



### 4.3.5 程序分析

#### system\_task\_handle.cpp

1. 文件开头先引入系统任务、全局命令、机械臂、USB 串口、参数存储、颜色夹取、AT32 OTA 和文件系统等模块。这里可以看出，这个文件负责把多个底层模块组织成完整的颜色分拣运行流程。`Global.h` 提供命令号，`Robot_Arm.h` 提供机械臂控制对象，`usb_ctrl.h` 负责 PC 串口通信，`ColorGrabber.h` 负责颜色识别后的夹取状态机，`AT32_OTA.h` 负责底层 AT32 固件检测和升级。

```
#include "system_task_handle.h"
#include "Global.h"
#include "Robot_Arm.h"
#include "usb_ctrl.h"
#include <Preferences.h>
#include "ColorGrabber.h"
#include "AT32_OTA.h"
#include <LittleFS.h>
#include <math.h>
#include <string.h>

#define CMD_SERVO_STREAM 0x68
```

2. 这些全局对象和变量保存了颜色分拣运行期间需要共享的系统状态。`loop_with_sys_task` 用于后续注册周期事件，`at32_protocol` 用来解析机械臂底层控制器反馈，`prefs` 用来保存颜色分拣标定参数和机械臂参数，`is_downloading` 用来在动作组下载时暂停常规运行逻辑，避免颜色分拣任务和下载流程互相干扰。

```
static const char* TAG = "ai_vision_sys";
static TimerHandle_t TIMER;
static esp_event_loop_handle_t loop_with_sys_task;
static CommProtocol_t at32_protocol;
static Preferences prefs;
```

```
static bool is_downloading = false;

static int16_t g_last_servo_positions[6] = {2048, 2048, 2048, 2048, 2048, 2048};
static uint32_t g_last_servo_feedback_ms = 0;

uint8_t global_channel = 2;
uint8_t global_acc = 245;

static float arm_kin[9] = {
    110.45f, 225.00f, 36.97f, 145.00f, 0.0f, 130.23f, 0.0f, 50.0f, 70.5f
};
```

3. 颜色分拣的可选目标被固定为红、绿、蓝三种颜色。kProtoColors 是传给 colorGrabber 的协议颜色名，kOledColor 是 OLED 显示用文本，s\_color\_sel\_idx 记录当前选择的颜色，s\_KEY1\_used\_since\_idle 用来判断用户是否手动选过颜色，s\_was\_grabber\_busy 用来检测一次分拣任务是否刚结束。

```
ESP_EVENT_DEFINE_BASE(SYS_TIMING_EVENTS);

static const char* kProtoColors[] = {"red", "green", "blue"};
static const char* kOledColor[] = {"RED", "GREEN", "BLUE"};

static uint8_t s_color_sel_idx = 0;
static bool s_KEY1_used_since_idle = false;
static bool s_was_grabber_busy = false;
```

4. apply\_oled\_standby\_screen() 负责根据颜色分拣任务状态刷新 OLED。colorGrabber.isBusy() 为真时显示正在运行，空闲时显示按键操作提示和当前颜色；如果用户还没有用 KEY1 选色，则提示 KEY2 会默认运行红色分拣。

```
static void apply_oled_standby_screen(void)
{
    arm.board.oled.set_custom_text(0, "Color Sorting");
    if (colorGrabber.isBusy()) {
        arm.board.oled.set_custom_text(1, "Running...");
        arm.board.oled.set_custom_text(2, "");
        arm.board.oled.set_custom_text(3, "Please wait");
    } else {
        if (s_KEY1_used_since_idle) {
            arm.board.oled.set_custom_text(1, "KEY1: Run this");
        } else {
            arm.board.oled.set_custom_text(1, "KEY1: Run(def RED)");
        }
        String line2 = "KEY2:";
        line2 += kOledColor[s_color_sel_idx % 3];
        arm.board.oled.set_custom_text(2, line2);
        arm.board.oled.set_custom_text(3, "Idle");
    }
    arm.board.oled.show_custom();
}
```

5. `perform_arm_poweron_reset()` 是颜色分拣开始前的机械臂上电复位流程。它先在 OLED 显示回零提示，再调用 `arm.reset_all(2500)` 让机械臂回到初始姿态，并在等待期间持续读取 AT32 反馈，确保复位过程中的底层数据不会堆积。

```
static void perform_arm_poweron_reset(void)
{
    arm.board.oled.set_custom_text(0, "Color Sorting");
    arm.board.oled.set_custom_text(1, "Arm homing...");
    arm.board.oled.set_custom_text(2, "Please wait");
    arm.board.oled.set_custom_text(3, "");
    arm.board.oled.show_custom();

    arm.reset_all(2500);
    uint32_t t0 = millis();
    while (millis() - t0 < 2600) {
        pump_at32_feedback();
        delay(20);
    }
    for (int i = 0; i < 8; i++) {
        arm.update_status();
        delay(50);
        while (servo.uart->available()) {
            uint8_t c = servo.uart->read();
            at32_protocol.parsing(&c, 1);
        }
    }
    Serial.println("[ColorSort] Power-on arm reset done");
}
```

6. `on_KEY1_select_color()` 是本工程的本地选色入口。只有 `colorGrabber` 空闲时 KEY1 才有效，每次按下都会把颜色索引切换到下一项，并用蜂鸣器和 OLED 给出反馈。

```
static void on_KEY1_select_color(void)
{
    if (colorGrabber.isBusy()) return;
    s_color_sel_idx = (uint8_t)((s_color_sel_idx + 1) % 3);
    s_KEY1_used_since_idle = true;
    arm.board.buzzer.set(30, 30, 1, 2500);
    apply_oled_standby_screen();
    Serial.printf("[ColorSort] KEY1 -> next color %s\n", kProtoColors[s_color_sel_idx % 3]);
}
```

7. `on_KEY2_start_sorting()` 是本地启动颜色分拣的入口。颜色夹取模块忙碌时会忽略 KEY2；空闲时，如果用户没有按过 KEY1，就默认分拣红色，如果已经按过 KEY1，就使用当前选中的颜色启动 `colorGrabber.start()`。

```
static void on_KEY2_start_sorting(void)
{
    if (colorGrabber.isBusy()) {
        Serial.println("[ColorSort] Busy, ignore KEY2");
        return;
    }
}
```

```

}
const char* cname = "red";
if (s_KEY1_used_since_idle) {
    cname = kProtoColors[s_color_sel_idx % 3];
}
const char* colors[] = {cname};
colorGrabber.start(colors, 1);
arm.board.buzzer.set(100, 100, 2, 2000);
apply_oled_standby_screen();
Serial.printf("[ColorSort] KEY2 -> start '%s' (KEY1_used=%d)\n", cname,
(int)s_KEY1_used_since_idle);
}

```

8. `save_config()` 和 `load_config()` 保存并恢复颜色分拣相关参数。这里除了通道、加速度和机械臂运动学参数外，还会保存 `grab_z_adj`，并在加载时恢复 `ag_x`、`ag_y`、`ag_z` 三个颜色抓取偏移量，这些参数直接影响识别目标后的实际抓取位置。

```

void save_config(void)
{
    prefs.begin("robot_cfg", false);
    prefs.putUChar("channel", global_channel);
    prefs.putUChar("acc", global_acc);
    prefs.putBytes("arm_kin", arm_kin, sizeof(arm_kin));
    prefs.putFloat("grab_z_adj", colorGrabber.getGrabZAdjust());
    prefs.end();
}

void load_config(void)
{
    prefs.begin("robot_cfg", true);
    global_channel = prefs.getUChar("channel", 2);
    global_acc = prefs.getUChar("acc", 245);

    float ag_x = prefs.getFloat("ag_x", 50.0f);
    float ag_y = prefs.getFloat("ag_y", 15.0f);
    float ag_z = prefs.getFloat("ag_z", 60.0f);
    colorGrabber.setOffsets(ag_x, ag_y, ag_z);
    colorGrabber.setGrabZAdjust(prefs.getFloat("grab_z_adj", -12.0f));

    if (prefs.isKey("arm_kin")) {
        float saved_kin[9];
        prefs.getBytes("arm_kin", saved_kin, sizeof(saved_kin));
        if (fabsf(saved_kin[0] - arm_kin[0]) < 0.01f) {
            memcpy(arm_kin, saved_kin, sizeof(arm_kin));
        }
    }
    prefs.end();
}

```

9. 周期定时器只保留和颜色分拣运行有关的基础维护：蜂鸣器刷新、电池刷新、机械臂状态刷新和 OLED 刷新。定时器回调不直接执行硬件操作，而是向 ESP 事件循环投递事件，后续由事件处理函数统一执行。

```

static void sys_timer_post_callback(TimerHandle_t xTimer)
{
    if (is_downloading) return;

    static uint32_t count = 0;
    if (count % BUZZER_UPDATE_PERIOD == 0) {
        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS, TIMING_EVENT_BUZZER_UPDATE,
        NULL, 0, 0);
    }
    if (count % BAT_UPDATE_PERIOD == 0) {
        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS, TIMING_EVENT_BAT_UPDATE,
        NULL, 0, 0);
    }
    if (count % SERVO_STATUS_UPDATE_PERIOD == 0) {
        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS,
        TIMING_EVENT_SERVO_STATUS_UPDATE, NULL, 0, 0);
    }
    if (count % OLED_REFRESH_PERIOD == 0) {
        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS, TIMING_EVENT_OLED_REFRESH,
        NULL, 0, 0);
    }

    count += TIMER_PERIOD;
}

```

10. `sys_timer_sub_handler()` 接收周期事件并执行具体维护动作。蜂鸣器用于按键和启动提示，电池状态用于主板状态维护，`arm.update_status()` 让系统持续刷新机械臂状态，OLED 则持续显示颜色分拣当前状态。

```

static void sys_timer_sub_handler(void* handler_args, esp_event_base_t base, int32_t id,
void* event_data)
{
    if (is_downloading) return;

    switch (id) {
        case TIMING_EVENT_BUZZER_UPDATE:
            arm.board.buzzer.update();
            break;
        case TIMING_EVENT_BAT_UPDATE:
            arm.board.bat.update();
            break;
        case TIMING_EVENT_SERVO_STATUS_UPDATE:
            arm.update_status();
            break;
        case TIMING_EVENT_OLED_REFRESH:
            apply_oled_standby_screen();
            break;
        default:
            break;
    }
}

```

11. `func_ctrl_callback()` 中和本工程直接相关的串口命令是动作组下载、机械臂运动学参数设置、颜色抓取偏移设置和颜色分拣启停。动作组下载期间会设置 `is_downloading`，使主循环暂停颜色分拣状态机和按键处理。

```
static void func_ctrl_callback(PacketTypeDef* self)
{
    uint8_t len;
    uint16_t bat_level;

    if (self->elements.id == 0xFF && self->elements.cmd == CMD_ACTION_GROUP_DOWNLOAD) {
        is_downloading = true;
        arm.board.action_group_download(self->elements.args[0], self->elements.args, self->elements.length - 2);
        is_downloading = false;
        return;
    }

    if (self->elements.id != 0xFF) return;

    switch (self->elements.cmd) {
```

12. `CMD_SET_KINEMATICS_PARAM` 用于更新机械臂运动学参数。颜色分拣最终要依赖机械臂到达识别、抓取和放置位置，因此这组参数会被写入本地变量、转发给舵机控制侧，并通过 `save_config()` 持久化保存。

```
case CMD_SET_KINEMATICS_PARAM:
    if (self->elements.length - 2 >= (int)sizeof(arm_kin)) {
        memcpy(arm_kin, self->elements.args, sizeof(arm_kin));
        servo.tx_frame_write(0xFF, CMD_SET_KINEMATICS_PARAM, (uint8_t*)arm_kin,
sizeof(arm_kin));
        save_config();
        arm.board.buzzer.set(100, 50, 1, 3000);
    }
    break;
```

13. `CMD_APRIITAG_SET_OFFSET` 在这个工程中被复用来设置颜色抓取偏移。上位机传入 X、Y 和抓取 Z 位置后，代码调用 `colorGrabber.setOffsets()` 更新颜色抓取补偿，并把参数写入 `Preferences`，保证下次开机仍然使用标定结果。

```
case CMD_APRIITAG_SET_OFFSET:
    if (self->elements.length >= 14) {
        float x_off, y_off, z_grab;
        memcpy(&x_off, &self->elements.args[0], 4);
        memcpy(&y_off, &self->elements.args[4], 4);
        memcpy(&z_grab, &self->elements.args[8], 4);
        colorGrabber.setOffsets(x_off, y_off, z_grab);
        prefs.begin("robot_cfg", false);
        prefs.putFloat("ag_x", x_off);
        prefs.putFloat("ag_y", y_off);
        prefs.putFloat("ag_z", z_grab);
        prefs.end();
        arm.board.buzzer.set(100, 50, 2, 3000);
    }
```

```

    }
    break;

```

14. `CMD_COLOR_GRAB` 是上位机控制颜色分拣的主要命令。`sw == 0` 时停止当前任务并关闭蜂鸣器；`sw != 0` 时根据 `color_id` 选择红、绿、蓝，再调用 `colorGrabber.start()` 启动单颜色分拣。

```

case CMD_COLOR_GRAB:
    if (self->elements.length >= 4) {
        uint8_t sw = self->elements.args[0];
        uint8_t color_id = self->elements.args[1];
        if (sw == 0) {
            colorGrabber.stop();
            arm.board.buzzer.off();
        } else {
            const char* cName = "";
            if (color_id == 1) cName = "red";
            else if (color_id == 2) cName = "green";
            else if (color_id == 3) cName = "blue";
            if (strlen(cName) > 0) {
                colorGrabber.stop();
                delay(100);
                const char* colors[] = {cName};
                colorGrabber.start(colors, 1);
                arm.board.buzzer.set(100, 100, 2, 2000);
            }
        }
    }
    break;

```

15. `at32_packet_callback()` 中和颜色分拣相关的核心作用是缓存机械臂舵机反馈。颜色分拣过程中需要知道机械臂是否有有效反馈，因此代码在收到舵机流数据后解析 6 个舵机位置，并更新时间戳。

```

void at32_packet_callback(PacketTypeDef* rx_packet)
{
    if (is_downloading) return;

    if (rx_packet->elements.id == 0xFF &&
        rx_packet->elements.cmd == CMD_SERVO_STREAM &&
        rx_packet->elements.length >= 14) {
        for (int i = 0; i < 6; ++i) {
            int idx = i * 2;
            g_last_servo_positions[i] = (int16_t)((rx_packet->elements.args[idx + 1] << 8) |
rx_packet->elements.args[idx]);
        }
        g_last_servo_feedback_ms = millis();
        return;
    }

    if (rx_packet->elements.id == 0xFF &&
        rx_packet->elements.cmd == 96 &&
        rx_packet->elements.length >= 14) {
        for (int i = 0; i < 6; ++i) {

```

```

        int idx = i * 2;
        g_last_servo_positions[i] = (int16_t)((rx_packet->elements.args[idx + 1] << 8) |
rx_packet->elements.args[idx]);
    }
    g_last_servo_feedback_ms = millis();
    return;
}

```

16. 当 AT32 返回当前坐标时，代码会把机械臂位置、姿态、夹爪值和舵机位置解析出来。颜色夹取动作依赖机械臂当前状态，这里更新 `arm.current_pose` 和舵机缓存，为后续抓取动作和状态同步提供数据基础。

```

    if (rx_packet->elements.cmd == CMD_GET_CUR_COORDS && rx_packet->elements.length
>= 26) {
        int16_t raw_x = (int16_t)((rx_packet->elements.args[1] << 8) | rx_packet-
>elements.args[0]);
        int16_t raw_y = (int16_t)((rx_packet->elements.args[3] << 8) | rx_packet-
>elements.args[2]);
        int16_t raw_z = (int16_t)((rx_packet->elements.args[5] << 8) | rx_packet-
>elements.args[4]);
        int16_t raw_p = (int16_t)((rx_packet->elements.args[7] << 8) | rx_packet-
>elements.args[6]);
        int16_t raw_r = (int16_t)((rx_packet->elements.args[9] << 8) | rx_packet-
>elements.args[8]);
        int16_t raw_c = (int16_t)((rx_packet->elements.args[11] << 8) | rx_packet-
>elements.args[10]);

        arm.current_pose.x = (float)raw_x;
        arm.current_pose.y = (float)raw_y;
        arm.current_pose.z = (float)raw_z;
        arm.current_pose.pitch = (float)raw_p / 10.0f;
        arm.current_pose.roll = (float)raw_r;
        arm.current_pose.claw = (float)raw_c;

        for (int i = 0; i < 6; ++i) {
            int idx = 12 + i * 2;
            g_last_servo_positions[i] = (int16_t)((rx_packet->elements.args[idx + 1]
<< 8) | rx_packet->elements.args[idx]);
        }
        g_last_servo_feedback_ms = millis();
    }
}

```

17. `pump_at32_feedback()` 和反馈同步函数负责在颜色分拣运行中持续抽取、解析和提供机械臂反馈。

`sync_arm_feedback()` 会主动请求一次机械臂状态，并在超时时间内同时处理 PC 串口和 AT32 串口，直到拿到新的舵机反馈。

```

void pump_at32_feedback(void)
{
    while (servo.uart->available()) {
        uint8_t c = servo.uart->read();
        at32_protocol.parsing(&c, 1);
    }
}

```

```

bool sync_arm_feedback(uint32_t timeout_ms)
{
    arm.update_status();
    uint32_t start = millis();
    uint32_t prev_feedback_ms = g_last_servo_feedback_ms;

    while (millis() - start < timeout_ms) {
        serial_port.rec_handler();
        pump_at32_feedback();
        if (g_last_servo_feedback_ms != 0 && g_last_servo_feedback_ms != prev_feedback_ms) {
            return true;
        }
        delay(2);
    }

    return g_last_servo_feedback_ms != 0;
}

bool get_last_servo_positions(int16_t out_pos[6])
{
    if (g_last_servo_feedback_ms == 0) {
        return false;
    }
    memcpy(out_pos, g_last_servo_positions, sizeof(g_last_servo_positions));
    return true;
}

```

18. `system_loop_handler()` 是颜色分拣运行阶段的主循环。非下载状态下，它依次处理上位机串口、AT32 反馈、颜色夹取状态机、任务结束检测、按键扫描和蜂鸣器刷新，其中 `colorGrabber.update()` 是推动分拣状态机继续运行的关键调用。

```

void system_loop_handler(void)
{
    if (!is_downloading) {
        serial_port.rec_handler();
        pump_at32_feedback();

        colorGrabber.update();

        bool busy_now = colorGrabber.isBusy();
        if (s_was_grabber_busy && !busy_now) {
            s_KEY1_used_since_idle = false;
            Serial.println("[ColorSort] Run finished, idle (KEY1 pick / KEY2 start)");
        }
        s_was_grabber_busy = busy_now;

        arm.board.button.update();
        arm.board.buzzer.update();
    }
}

```

19. 主循环只在颜色夹取模块空闲时响应 KEY1 和 KEY2。这样可以避免机械臂正在执行颜色分拣时被新的按键操作打断；下载状态下则只处理上位机串口，不推进颜色分拣状态机。

```

        if (!busy_now) {
            if (arm.board.button.is_clicked(1)) {
                on_KEY1_select_color();
            }
            if (arm.board.button.is_clicked(0)) {
                on_KEY2_start_sorting();
            }
        }
    } else {
        serial_port.rec_handler();
    }
}

```

20. `register_system_task()` 是颜色分拣工程的初始化入口。它先保存事件循环句柄、提高 CPU 频率、加载颜色分拣和机械臂配置，然后挂载 LittleFS，为后续动作组、配置和系统文件访问做好准备。

```

void register_system_task(esp_event_loop_handle_t *event_loop)
{
    loop_with_sys_task = *event_loop;
    setCpuFrequencyMhz(240);
    load_config();

    if (!LittleFS.begin(true)) {
        Serial.println("[LittleFS] Mount failed, formatting...");
        LittleFS.format();
        LittleFS.begin();
    }
    Serial.println("[LittleFS] Mounted successfully");
}

```

21. 初始化过程中会启动 PC 串口和 AT32 舵机串口，并初始化机械臂主板。

`serial_port.register_ops_callback(func_ctrl_callback)` 把上位机命令入口绑定到本文件，后续上位机发送颜色分拣启停或标定参数时，都会进入 `func_ctrl_callback()`。

```

serial_port.begin(Serial1, 1000000);
serial_port.register_ops_callback(func_ctrl_callback);

servo.begin(Serial1, 1000000, 16, 17);
arm.begin();

arm.board.oled.set_icon(4);
for (int i = 0; i < 25; i++) {
    arm.board.oled.show_icon();
    delay(100);
}

AT32_OTA::check_and_update(serial_port.protocol, *servo.uart, arm.board.oled);

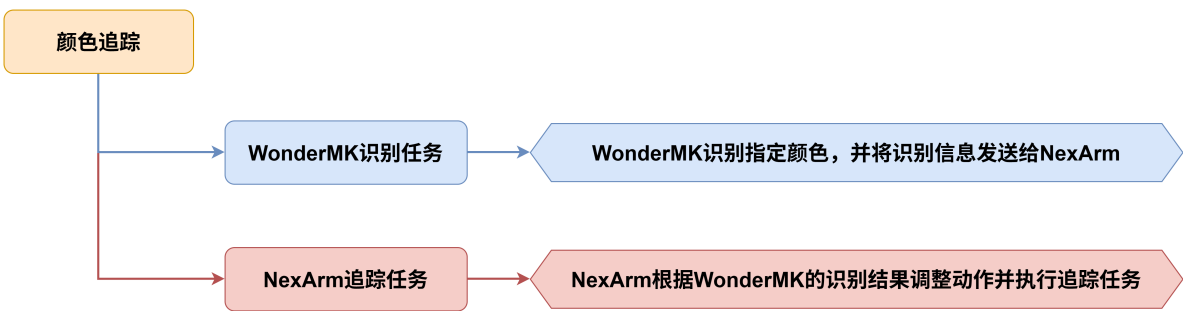
```

## 4.4 颜色追踪

### 4.4.1 项目介绍

颜色分拣功能通过调用WonderMK识别不同颜色的目标物体，并锁定指定颜色进行机械臂追踪任务。

### 4.4.2 实现流程



### 4.4.3 程序下载

找到本文档同目录下的“02 程序源码”文件夹，找到“02 颜色追踪”程序进行下载，具体下载步骤可参考[4.2 程序下载说明\(必看\)](#)。

### 4.4.4 运行效果

设备上电后先完成初始化，机械臂自动回到初始观察位，屏幕上会显示当前运行状态和目标颜色，随后进入等待状态。启动后，机械臂会围绕当前选定的颜色目标进行持续跟随，当目标在画面中发生偏移时，机械臂末端会按识别结果不断做小幅调整，使目标逐步回到画面中心附近；当停止运行后，跟随会结束，机械臂回到预设的初始姿态，界面状态也同步切回关闭状态。整个过程表现为设备能够对选定颜色进行实时识别，并驱动机械臂连续修正位置，形成明显的动态追踪效果。

实际使用时，先正常上电并等待初始化结束，确认机械臂已经回到初始位、屏幕显示正常；如果需要切换目标颜色，可短按 KEY1 切换按键，在红色、绿色、蓝色之间循环选择；确认颜色后按下 KEY2 启动按键，系统便开始进入跟随状态，机械臂会根据画面中的颜色目标持续调整位置；如果需要结束跟随，再按一次 KEY2 启动按键即可停止，系统会退出运行状态并恢复到等待界面。



#### 4.4.5 程序分析

##### system\_task\_handle.cpp

1. 文件开头先引入颜色追踪工程需要的核心模块，包括系统任务接口、全局命令、机械臂控制、USB 串口控制、参数存储、颜色追踪状态机、AT32 OTA 和文件系统等。和颜色分拣工程不同，这里使用的是 `ColorTrackerRot.h`，说明本工程的核心业务不是夹取物块，而是让机械臂根据颜色目标持续调整姿态并完成追踪。

```
#include "system_task_handle.h"
#include "Global.h"
#include "Robot_Arm.h"
#include "usb_ctrl.h"
#include <Preferences.h>
#include "ColorTrackerRot.h"
#include "AT32_OTA.h"
#include <LittleFS.h>
#include <math.h>
#include <string.h>

#define CMD_SERVO_STREAM 0x68
```

2. 这些静态对象和全局变量保存系统运行状态。`loop_with_sys_task` 用于注册 ESP 事件循环，`at32_protocol` 用于解析 AT32 侧反馈，`prefs` 用于保存机械臂参数，`is_downloading` 用于在动作组下载期间暂停常规任务，`g_last_servo_positions` 和 `g_last_servo_feedback_ms` 则缓存最近一次舵机反馈，为颜色追踪过程中的机械臂状态同步提供基础数据。

```
static const char* TAG = "color_tracking";
static TimerHandle_t TIMER;
static esp_event_loop_handle_t loop_with_sys_task;
static CommProtocol_t at32_protocol;
```

```

static Preferences prefs;
static bool is_downloading = false;

static int16_t g_last_servo_positions[6] = {2048, 2048, 2048, 2048, 2048, 2048};
static uint32_t g_last_servo_feedback_ms = 0;

uint8_t global_channel = 2;
uint8_t global_acc = 245;
volatile bool servo_uart_busy = false;

static float arm_kin[9] = {
    110.45f, 225.00f, 36.97f, 145.00f, 0.0f, 130.23f, 0.0f, 50.0f, 70.5f
};

```

3. 颜色追踪目标被限定为红、绿、蓝三种。kTrackColors 是传给颜色追踪模块的颜色名，kTrackLabels 是 OLED 显示用文本，s\_track\_color\_idx、s\_track\_color 和 s\_track\_label 共同记录当前追踪目标，s\_was\_tracker\_busy 用来检测追踪任务是否从运行切回空闲，s\_buttons\_armed 和 s\_button\_enable\_time 用于开机后的按键延迟使能。

```

static const char* kTrackColors[] = {"red", "green", "blue"};
static const char* kTrackLabels[] = {"RED", "GREEN", "BLUE"};

static uint8_t s_track_color_idx = 0;
static const char* s_track_color = kTrackColors[0];
static const char* s_track_label = kTrackLabels[0];
static bool s_was_tracker_busy = false;
static uint32_t s_button_enable_time = 0;
static bool s_buttons_armed = false;

ESP_EVENT_DEFINE_BASE(SYS_TIMING_EVENTS);

```

4. 这组三个函数负责在上位机颜色编号、颜色字符串和当前追踪目标之间转换。color\_name\_from\_id() 把协议里的 1、2、3 转成 red、green、blue，color\_index\_from\_name() 把颜色名转成数组索引，set\_tracking\_target() 则统一更新当前颜色索引、协议颜色名和 OLED 标签。

```

static const char* color_name_from_id(uint8_t color_id)
{
    if (color_id >= 1 && color_id <= 3) {
        return kTrackColors[color_id - 1];
    }
    return nullptr;
}

static uint8_t color_index_from_name(const char* color_name)
{
    if (!color_name) return 0;
    for (uint8_t i = 0; i < 3; ++i) {
        if (strcmp(color_name, kTrackColors[i]) == 0) {
            return i;
        }
    }
}

```

```

    return 0;
}

static void set_tracking_target(const char* color_name)
{
    s_track_color_idx = color_index_from_name(color_name);
    s_track_color = kTrackColors[s_track_color_idx];
    s_track_label = kTrackLabels[s_track_color_idx];
}

```

5. `apply_oled_tracking_screen()` 负责刷新颜色追踪工程的 OLED 界面。追踪运行时显示当前追踪颜色，并根据 `colorTrackerRot.isCentered()` 显示目标是否已经居中；空闲时显示追踪入口、颜色切换入口和当前目标颜色。

```

static void apply_oled_tracking_screen(void)
{
    arm.board.oled.set_custom_text(0, "Color Tracking");

    if (colorTrackerRot.isBusy()) {
        String running_line = "Tracking: ";
        running_line += s_track_label;
        arm.board.oled.set_custom_text(1, running_line);
        arm.board.oled.set_custom_text(2, colorTrackerRot.isCentered() ? "Status: Center" :
"Status: Search");
        arm.board.oled.set_custom_text(3, "KEY2: Stop");
    } else {
        String target_line = "Target: ";
        target_line += s_track_label;
        arm.board.oled.set_custom_text(1, "KEY2: Track");
        arm.board.oled.set_custom_text(2, "KEY1: Color");
        arm.board.oled.set_custom_text(3, target_line);
    }

    arm.board.oled.show_custom();
}

```

6. `perform_arm_poweron_reset()` 是追踪任务开始前的机械臂上电复位流程。它先显示回零提示，再调用 `arm.reset_all(2500)` 让机械臂回到初始姿态，并在等待期间持续解析 AT32 反馈，保证进入追踪逻辑前机械臂状态已经被刷新。

```

static void perform_arm_poweron_reset(void)
{
    arm.board.oled.set_custom_text(0, "Color Tracking");
    arm.board.oled.set_custom_text(1, "Arm homing...");
    arm.board.oled.set_custom_text(2, "Please wait");
    arm.board.oled.set_custom_text(3, "");
    arm.board.oled.show_custom();

    arm.reset_all(2500);
    uint32_t t0 = millis();
    while (millis() - t0 < 2600) {

```

```

        pump_at32_feedback();
        delay(20);
    }
    for (int i = 0; i < 8; i++) {
        arm.update_status();
        delay(50);
        while (servo.uart->available()) {
            uint8_t c = servo.uart->read();
            at32_protocol.parsing(&c, 1);
        }
    }
    Serial.println("[ColorTrack] Power-on arm reset done");
}

```

7. `start_color_tracking()` 是本地启动颜色追踪的函数。它只在 `ColorTrackerRot` 空闲时生效，启动前先调用 `stop()` 清理旧状态，再把当前目标颜色 `s_track_color` 传给 `colorTrackerRot.start()`，启动成功和失败会用不同蜂鸣器提示区分。

```

static void start_color_tracking(void)
{
    if (colorTrackerRot.isBusy()) {
        Serial.println("[ColorTrack] Busy, ignore KEY1");
        return;
    }

    colorTrackerRot.stop();
    delay(100);
    colorTrackerRot.start(s_track_color);

    if (colorTrackerRot.isBusy()) {
        arm.board.buzzer.set(100, 100, 2, 2000);
        Serial.printf("[ColorTrack] KEY1 -> start '%s'\n", s_track_color);
    } else {
        arm.board.buzzer.set(300, 100, 1, 1000);
        Serial.printf("[ColorTrack] start '%s' failed\n", s_track_color);
    }

    apply_oled_tracking_screen();
}

```

8. `stop_color_tracking()` 负责停止正在运行的颜色追踪。只有追踪模块处于忙碌状态时才会执行停止，停止后蜂鸣器短响、OLED 立即回到追踪待机界面。

```
static void stop_color_tracking(void)
{
    if (!colorTrackerRot.isBusy()) {
        return;
    }

    colorTrackerRot.stop();
    arm.board.buzzer.set(80, 80, 1, 1800);
    apply_oled_tracking_screen();
    Serial.println("[ColorTrack] KEY1 -> stop");
}

```

9. `select_next_tracking_color()` 负责在空闲状态下切换追踪颜色。追踪运行中切换颜色会被拒绝，空闲时每次调用都会在红、绿、蓝之间循环，并同步更新协议颜色名、OLED 标签、蜂鸣器提示和串口日志。

```
static void select_next_tracking_color(void)
{
    if (colorTrackerRot.isBusy()) {
        Serial.println("[ColorTrack] Busy, ignore KEY2");
        return;
    }

    s_track_color_idx = (uint8_t)((s_track_color_idx + 1) % 3);
    s_track_color = kTrackColors[s_track_color_idx];
    s_track_label = kTrackLabels[s_track_color_idx];

    arm.board.buzzer.set(30, 30, 1, 2500);
    apply_oled_tracking_screen();
    Serial.printf("[ColorTrack] KEY2 -> select '%s'\n", s_track_color);
}

```

10. `notify_KEY2_locked()` 和 `buttons_ready()` 处理按键保护逻辑。前者用于追踪运行中按下颜色切换键时给出锁定提示，后者在开机后先清掉已有按键状态，并等待 `s_button_enable_time` 到达后才真正允许按键触发，避免上电阶段误触发追踪或切色。

```
static void notify_KEY2_locked(void)
{
    arm.board.buzzer.set(60, 40, 1, 1200);
    apply_oled_tracking_screen();
    Serial.println("[ColorTrack] KEY2 locked while tracking");
}

static bool buttons_ready(void)
{
    if (s_buttons_armed) {
        return true;
    }

    (void)arm.board.button.is_clicked(0);
    (void)arm.board.button.is_clicked(1);
}

```

```

    if (millis() < s_button_enable_time) {
        return false;
    }

    s_buttons_armed = true;
    Serial.println("[ColorTrack] Buttons armed");
    return false;
}

```

11. `save_config()` 和 `load_config()` 负责保存与颜色追踪相关的机械臂基础参数。这里保存通道、加速度和运动学参数，加载时如果存在 `arm_kin`，会先做一个简单校验，再覆盖默认运动学参数，避免异常数据影响追踪动作。

```

void save_config(void)
{
    prefs.begin("robot_cfg", false);
    prefs.putUChar("channel", global_channel);
    prefs.putUChar("acc", global_acc);
    prefs.putBytes("arm_kin", arm_kin, sizeof(arm_kin));
    prefs.end();
}

void load_config(void)
{
    prefs.begin("robot_cfg", true);
    global_channel = prefs.getUChar("channel", 2);
    global_acc = prefs.getUChar("acc", 245);

    if (prefs.isKey("arm_kin")) {
        float saved_kin[9];
        prefs.getBytes("arm_kin", saved_kin, sizeof(saved_kin));
        if (fabsf(saved_kin[0] - arm_kin[0]) < 0.01f) {
            memcpy(arm_kin, saved_kin, sizeof(arm_kin));
        }
    }
    prefs.end();
}

```

12. 周期定时器回调负责按固定节拍投递系统维护事件。对颜色追踪来说，这些周期事件主要用于蜂鸣器刷新、电池状态刷新、机械臂状态刷新和 OLED 追踪界面刷新；动作组下载期间会直接跳过，避免干扰下载流程。

```

static void sys_timer_post_callback(TimerHandle_t xTimer)
{
    if (is_downloading) return;

    static uint32_t count = 0;
    if (count % BUZZER_UPDATE_PERIOD == 0) {
        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS, TIMING_EVENT_BUZZER_UPDATE,
        NULL, 0, 0);
    }
    if (count % BAT_UPDATE_PERIOD == 0) {

```

```

        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS, TIMING_EVENT_BAT_UPDATE,
NULL, 0, 0);
    }
    if (count % SERVO_STATUS_UPDATE_PERIOD == 0) {
        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS,
TIMING_EVENT_SERVO_STATUS_UPDATE, NULL, 0, 0);
    }
    if (count % OLED_REFRESH_PERIOD == 0) {
        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS, TIMING_EVENT_OLED_REFRESH,
NULL, 0, 0);
    }

    count += TIMER_PERIOD;
}

```

13. 周期事件处理函数根据事件 ID 执行具体维护动作。`TIMING_EVENT_SERVO_STATUS_UPDATE` 会调用 `arm.update_status()` 刷新机械臂状态，`TIMING_EVENT_OLED_REFRESH` 会调用 `apply_oled_tracking_screen()` 让 OLED 始终显示当前追踪颜色、搜索状态或居中状态。

```

static void sys_timer_sub_handler(void* handler_args, esp_event_base_t base, int32_t id,
void* event_data)
{
    if (is_downloading) return;

    switch (id) {
        case TIMING_EVENT_BUZZER_UPDATE:
            arm.board.buzzer.update();
            break;
        case TIMING_EVENT_BAT_UPDATE:
            arm.board.bat.update();
            break;
        case TIMING_EVENT_SERVO_STATUS_UPDATE:
            arm.update_status();
            break;
        case TIMING_EVENT_OLED_REFRESH:
            apply_oled_tracking_screen();
            break;
        default:
            break;
    }
}

```

14. `func_ctrl_callback()` 是上位机串口命令入口，其中和颜色追踪运行直接相关的是动作组下载保护、运动学参数设置和 `CMD_COLOR_TRACK`。动作组下载时会设置 `is_downloading`，让主循环暂停追踪状态机；非广播 ID 的数据包会直接返回，不进入本工程的整机控制逻辑。

```

static void func_ctrl_callback(PacketTypeDef* self)
{
    uint8_t len;
    uint16_t bat_level;

```

```

    if (self->elements.id == 0xFF && self->elements.cmd == CMD_ACTION_GROUP_DOWNLOAD) {
        is_downloading = true;
        arm.board.action_group_download(self->elements.args[0], self->elements.args, self->elements.length - 2);
        is_downloading = false;
        return;
    }

    if (self->elements.id != 0xFF) return;

    switch (self->elements.cmd) {

```

15. `CMD_SET_KINEMATICS_PARAM` 用于从上位机更新机械臂运动学参数。颜色追踪依赖机械臂根据视觉偏差进行运动修正，因此新的运动学参数会同时写入本地变量、下发给舵机控制侧，并调用 `save_config()` 持久化保存。

```

    case CMD_SET_KINEMATICS_PARAM:
        if (self->elements.length - 2 >= (int)sizeof(arm_kin)) {
            memcpy(arm_kin, self->elements.args, sizeof(arm_kin));
            servo.tx_frame_write(0xFF, CMD_SET_KINEMATICS_PARAM, (uint8_t*)arm_kin,
sizeof(arm_kin));
            save_config();
            arm.board.buzzer.set(100, 50, 1, 3000);
        }
        break;

```

16. `CMD_COLOR_TRACK` 是上位机控制颜色追踪目标的主要命令。`sw == 0` 时停止当前追踪并刷新 OLED；`sw != 0` 时根据 `color_id` 选择目标颜色，但代码只设置目标并提示按键启动，没有直接调用 `colorTrackerRot.start()`，说明上位机命令在这里偏向“选择目标”，真正启动仍交给本机按键流程。

```

    case CMD_COLOR_TRACK:
        if (self->elements.length >= 4) {
            uint8_t sw = self->elements.args[0];
            uint8_t color_id = self->elements.args[1];
            if (sw == 0) {
                colorTrackerRot.stop();
                arm.board.buzzer.off();
                apply_oled_tracking_screen();
            } else {
                const char* color_name = color_name_from_id(color_id);
                if (color_name) {
                    set_tracking_target(color_name);
                    arm.board.buzzer.set(30, 30, 1, 2500);
                    apply_oled_tracking_screen();
                    Serial.printf("[ColorTrack] CMD select '%s', press KEY1 to start\n",
s_track_color);
                } else {
                    Serial.printf("[ColorTrack] CMD invalid color id=%u\n", color_id);
                }
            }
        }
        break;

```

17. `at32_packet_callback()` 处理 AT32 或舵机侧返回的数据。对颜色追踪来说，最直接相关的是舵机位置缓存：当收到舵机位置流时，代码解析 6 个舵机位置并更新时间戳，使追踪逻辑和状态同步函数能拿到最新的机械臂反馈。

```
void at32_packet_callback(PacketTypeDef* rx_packet)
{
    if (is_downloading) return;

    if (rx_packet->elements.id == 0xFF &&
        rx_packet->elements.cmd == CMD_SERVO_STREAM &&
        rx_packet->elements.length >= 14) {
        for (int i = 0; i < 6; ++i) {
            int idx = i * 2;
            g_last_servo_positions[i] = (int16_t)((rx_packet->elements.args[idx + 1] << 8) |
rx_packet->elements.args[idx]);
        }
        g_last_servo_feedback_ms = millis();
        return;
    }

    if (rx_packet->elements.id == 0xFF &&
        rx_packet->elements.cmd == 96 &&
        rx_packet->elements.length >= 14) {
        for (int i = 0; i < 6; ++i) {
            int idx = i * 2;
            g_last_servo_positions[i] = (int16_t)((rx_packet->elements.args[idx + 1] << 8) |
rx_packet->elements.args[idx]);
        }
        g_last_servo_feedback_ms = millis();
        return;
    }
}
```

18. 当 AT32 返回当前机械臂坐标时，代码会解析 X、Y、Z、pitch、roll、claw 和 6 个舵机位置，并写入 `arm.current_pose` 与舵机缓存。颜色追踪过程中，机械臂状态越及时，视觉追踪到运动修正的闭环越稳定。

```
    if (rx_packet->elements.cmd == CMD_GET_CUR_COORDS && rx_packet->elements.length
>= 26) {
        int16_t raw_x = (int16_t)((rx_packet->elements.args[1] << 8) | rx_packet-
>elements.args[0]);
        int16_t raw_y = (int16_t)((rx_packet->elements.args[3] << 8) | rx_packet-
>elements.args[2]);
        int16_t raw_z = (int16_t)((rx_packet->elements.args[5] << 8) | rx_packet-
>elements.args[4]);
        int16_t raw_p = (int16_t)((rx_packet->elements.args[7] << 8) | rx_packet-
>elements.args[6]);
        int16_t raw_r = (int16_t)((rx_packet->elements.args[9] << 8) | rx_packet-
>elements.args[8]);
        int16_t raw_c = (int16_t)((rx_packet->elements.args[11] << 8) | rx_packet-
>elements.args[10]);

        arm.current_pose.x = (float)raw_x;
        arm.current_pose.y = (float)raw_y;
```

```

        arm.current_pose.z = (float)row_z;
        arm.current_pose.pitch = (float)row_p / 10.0f;
        arm.current_pose.roll = (float)row_r;
        arm.current_pose.claw = (float)row_c;

        for (int i = 0; i < 6; ++i) {
            int idx = 12 + i * 2;
            g_last_servo_positions[i] = (int16_t)((rx_packet->elements.args[idx + 1]
<< 8) | rx_packet->elements.args[idx]);
        }
        g_last_servo_feedback_ms = millis();
    }

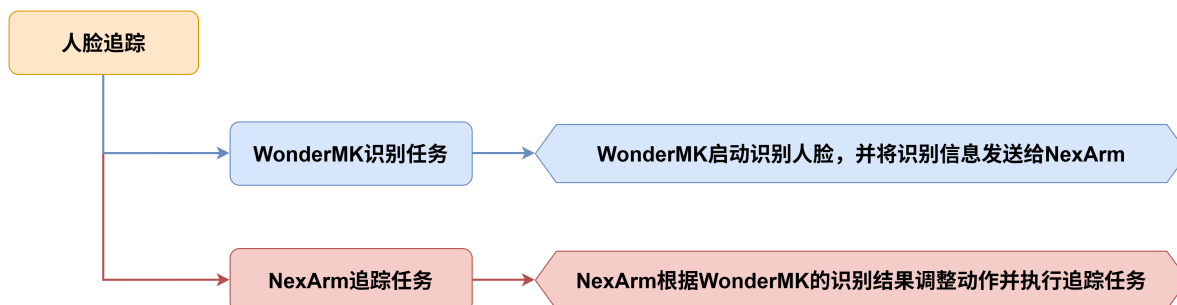
```

## 4.5 人脸追踪

### 4.5.1 项目介绍

本实验实现了基于视觉识别的人脸自动追踪功能。系统能够根据人脸在画面中的位置变化，持续调整机械臂末端姿态，使目标尽量保持在画面中心附近。

### 4.5.2 实现流程



### 4.5.3 程序下载

找到本文档同目录下的“02 程序源码”文件夹，找到“03 人脸追踪”程序进行下载，具体下载步骤可参考[4.2 程序下载说明\(必看\)](#)。

### 4.5.4 运行效果

设备上电后会先完成初始化，机械臂自动回到初始观察位，屏幕上显示当前运行状态，随后进入等待状态。启动后，系统会持续检测画面中的人脸目标，并根据目标在画面中的左右和上下偏移不断调整机械臂位置，使末端能够跟随人脸运动；当目标接近画面中心时，机械臂的调整会明显减小，从而让整体跟随效果更加平稳。停止运行后，追踪过程结束，机械臂回到初始姿态，界面状态也同步恢复到关闭状态。

实际使用时，先正常上电并等待初始化结束，确认机械臂已经回到初始位；让人脸进入摄像头可识别范围内；按下 **KEY2** 启动按键后，系统开始进入人脸追踪状态，机械臂会根据人脸位置变化持续做出跟随调整；如果需要结束追踪，再按一次启动按键即可停止；如果需要让机械臂重新回到初始观察位，可以按下复位按键，设备会立即执行回位动作，便于重新开始新一轮演示。



#### 4.5.5 程序分析

##### system\_task\_handle.cpp

1. 文件开头先引入人脸追踪工程需要的核心模块，包括系统任务接口、全局命令、机械臂控制、USB 串口控制、参数存储、人脸追踪状态机、AT32 OTA 和文件系统等。这里最关键的是 `FaceTracker.h`，它负责和视觉侧通信并推动人脸追踪状态机；`Robot_Arm.h` 负责机械臂动作；`usb_ctrl.h` 负责上位机串口命令入口。

```
#include "system_task_handle.h"
#include "Global.h"
#include "Robot_Arm.h"
#include "usb_ctrl.h"
#include <Preferences.h>
#include "FaceTracker.h"
#include "AT32_OTA.h"
#include <LittleFS.h>
#include <math.h>
#include <string.h>

#define CMD_SERVO_STREAM 0x68
```

2. 这些静态对象和全局变量保存人脸追踪运行期间需要共享的系统状态。`loop_with_sys_task` 用于注册周期事件，`at32_protocol` 用来解析 AT32 侧机械臂反馈，`prefs` 用来保存机械臂参数，`is_downloading` 用来在动作组下载时暂停常规任务，`g_last_servo_positions` 和 `g_last_servo_feedback_ms` 则缓存最近一次舵机反馈。

```
static const char* TAG = "face_tracking";
static TimerHandle_t TIMER;
static esp_event_loop_handle_t loop_with_sys_task;
static CommProtocol_t at32_protocol;
static Preferences prefs;
```

```
static bool is_downloading = false;

static int16_t g_last_servo_positions[6] = {2048, 2048, 2048, 2048, 2048, 2048};
static uint32_t g_last_servo_feedback_ms = 0;

uint8_t global_channel = 2;
uint8_t global_acc = 245;
volatile bool servo_uart_busy = false;

static float arm_kin[9] = {
    110.45f, 225.00f, 36.97f, 145.00f, 0.0f, 130.23f, 0.0f, 50.0f, 70.5f
};
```

3. 这里定义了人脸追踪工程的运行状态标记。 `s_was_tracker_busy` 用来判断人脸追踪是否刚刚从运行状态回到空闲状态， `s_button_enable_time` 和 `s_buttons_armed` 用于开机后的按键延迟使能，避免上电过程中误触发追踪启动或停止。

```
static bool s_was_tracker_busy = false;
static uint32_t s_button_enable_time = 0;
static bool s_buttons_armed = false;

ESP_EVENT_DEFINE_BASE(SYS_TIMING_EVENTS);
```

4. `apply_oled_face_screen()` 负责刷新人脸追踪工程的 OLED 界面。 `faceTracker.isBusy()` 为真时显示正在追踪人脸和搜索状态，空闲时显示追踪入口、 `Face Ready` 和 `Idle`，让用户能直接从屏幕判断当前是否可以启动追踪。

```
static void apply_oled_face_screen(void)
{
    arm.board.oled.set_custom_text(0, "Face Tracking");

    if (faceTracker.isBusy()) {
        arm.board.oled.set_custom_text(1, "Tracking Face");
        arm.board.oled.set_custom_text(2, "Status: Search");
        arm.board.oled.set_custom_text(3, "KEY2: Stop");
    } else {
        arm.board.oled.set_custom_text(1, "KEY2: Track");
        arm.board.oled.set_custom_text(2, "Face Ready");
        arm.board.oled.set_custom_text(3, "Idle");
    }

    arm.board.oled.show_custom();
}
```

5. `perform_arm_poweron_reset()` 是人脸追踪开始前的机械臂上电复位流程。它先在 OLED 显示回零提示，再调用 `arm.reset_all(2500)` 让机械臂回到初始姿态，并在等待过程中持续抽取 AT32 反馈，保证进入追踪运行前机械臂状态已经可用。

```
static void perform_arm_poweron_reset(void)
{
```

```

arm.board.oled.set_custom_text(0, "Face Tracking");
arm.board.oled.set_custom_text(1, "Arm homing...");
arm.board.oled.set_custom_text(2, "Please wait");
arm.board.oled.set_custom_text(3, "");
arm.board.oled.show_custom();

arm.reset_all(2500);
uint32_t t0 = millis();
while (millis() - t0 < 2600) {
    pump_at32_feedback();
    delay(20);
}
for (int i = 0; i < 8; i++) {
    arm.update_status();
    delay(50);
    while (servo.uart->available()) {
        uint8_t c = servo.uart->read();
        at32_protocol.parsing(&c, 1);
    }
}
Serial.println("[FaceTrack] Power-on arm reset done");
}

```

6. `start_face_tracking()` 是本地启动人脸追踪的函数。它只在 `FaceTracker` 空闲时执行，启动前先调用 `faceTracker.stop()` 清理旧状态，再调用 `faceTracker.start()` 进入追踪流程，随后根据 `faceTracker.isBusy()` 判断启动是否成功并给出蜂鸣器提示。

```

static void start_face_tracking(void)
{
    if (faceTracker.isBusy()) {
        Serial.println("[FaceTrack] Busy, ignore KEY1 start");
        return;
    }

    faceTracker.stop();
    delay(100);
    faceTracker.start();

    if (faceTracker.isBusy()) {
        arm.board.buzzer.set(100, 100, 2, 2000);
        Serial.println("[FaceTrack] KEY1 -> start");
    } else {
        arm.board.buzzer.set(300, 100, 1, 1000);
        Serial.println("[FaceTrack] start failed");
    }

    apply_oled_face_screen();
}

```

7. `stop_face_tracking()` 负责停止正在运行的人脸追踪。函数先判断 `faceTracker.isBusy()`，只有追踪确实在运行时才停止状态机、触发蜂鸣器提示并刷新 OLED，避免空闲状态下重复执行停止逻辑。

```
static void stop_face_tracking(void)
{
    if (!faceTracker.isBusy()) {
        return;
    }

    faceTracker.stop();
    arm.board.buzzer.set(80, 80, 1, 1800);
    apply_oled_face_screen();
    Serial.println("[FaceTrack] KEY1 -> stop");
}
```

8. `buttons_ready()` 是开机后的按键保护函数。系统刚启动时会先读取并清掉两个按键的点击状态，在 `s_button_enable_time` 到达之前都不允许按键触发；时间到达后把 `s_buttons_armed` 置为真，后续按键才真正生效。

```
static bool buttons_ready(void)
{
    if (s_buttons_armed) {
        return true;
    }

    (void)arm.board.button.is_clicked(0);
    (void)arm.board.button.is_clicked(1);

    if (millis() < s_button_enable_time) {
        return false;
    }

    s_buttons_armed = true;
    Serial.println("[FaceTrack] Buttons armed");
    return false;
}
```

9. `save_config()` 和 `load_config()` 负责保存与人脸追踪相关的机械臂基础参数。这里保存通道、加速度和运动学参数；加载时如果 NVS 中存在 `arm_kin`，会先做一个简单校验，再覆盖默认运动学参数，避免异常参数影响追踪运动。

```
void save_config(void)
{
    prefs.begin("robot_cfg", false);
    prefs.putUChar("channel", global_channel);
    prefs.putUChar("acc", global_acc);
    prefs.putBytes("arm_kin", arm_kin, sizeof(arm_kin));
    prefs.end();
}

void load_config(void)
{
    prefs.begin("robot_cfg", true);
    global_channel = prefs.getUChar("channel", 2);
}
```

```

global_acc = prefs.getUChar("acc", 245);

if (prefs.isKey("arm_kin")) {
    float saved_kin[9];
    prefs.getBytes("arm_kin", saved_kin, sizeof(saved_kin));
    if (fabsf(saved_kin[0] - arm_kin[0]) < 0.01f) {
        memcpy(arm_kin, saved_kin, sizeof(arm_kin));
    }
}
prefs.end();
}

```

10. 周期定时器回调负责投递人脸追踪运行时需要的基础维护事件。它不会直接操作硬件，而是按周期投递蜂鸣器、电池、机械臂状态和 OLED 刷新事件；如果正在下载动作组，则直接返回，避免下载流程被周期任务干扰。

```

static void sys_timer_post_callback(TimerHandle_t xTimer)
{
    if (is_downloading) return;

    static uint32_t count = 0;
    if (count % BUZZER_UPDATE_PERIOD == 0) {
        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS, TIMING_EVENT_BUZZER_UPDATE,
        NULL, 0, 0);
    }
    if (count % BAT_UPDATE_PERIOD == 0) {
        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS, TIMING_EVENT_BAT_UPDATE,
        NULL, 0, 0);
    }
    if (count % SERVO_STATUS_UPDATE_PERIOD == 0) {
        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS,
        TIMING_EVENT_SERVO_STATUS_UPDATE, NULL, 0, 0);
    }
    if (count % OLED_REFRESH_PERIOD == 0) {
        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS, TIMING_EVENT_OLED_REFRESH,
        NULL, 0, 0);
    }

    count += TIMER_PERIOD;
}

```

11. `sys_timer_sub_handler()` 接收周期事件并执行具体维护动作。蜂鸣器和电池是主板基础状态维护，`arm.update_status()` 用于持续刷新机械臂状态，`apply_oled_face_screen()` 则让 OLED 根据 `FaceTracker` 当前状态持续显示追踪或空闲界面。

```

static void sys_timer_sub_handler(void* handler_args, esp_event_base_t base, int32_t id,
void* event_data)
{
    if (is_downloading) return;

    switch (id) {
        case TIMING_EVENT_BUZZER_UPDATE:
            arm.board.buzzer.update();
    }
}

```

```

        break;
    case TIMING_EVENT_BAT_UPDATE:
        arm.board.bat.update();
        break;
    case TIMING_EVENT_SERVO_STATUS_UPDATE:
        arm.update_status();
        break;
    case TIMING_EVENT_OLED_REFRESH:
        apply_oled_face_screen();
        break;
    default:
        break;
}
}

```

12. `func_ctrl_callback()` 是上位机串口命令入口。和人脸追踪主流程直接相关的是动作组下载保护、机械臂运动学参数设置和 `CMD_FACE_TRACK`；动作组下载时会把 `is_downloading` 置为真，让主循环暂停人脸追踪状态机和按键处理。

```

static void func_ctrl_callback(PacketTypeDef* self)
{
    uint8_t len;
    uint16_t bat_level;

    if (self->elements.id == 0xFF && self->elements.cmd == CMD_ACTION_GROUP_DOWNLOAD) {
        is_downloading = true;
        arm.board.action_group_download(self->elements.args[0], self->elements.args, self->elements.length - 2);
        is_downloading = false;
        return;
    }

    if (self->elements.id != 0xFF) return;

    switch (self->elements.cmd) {

```

13. `CMD_SET_KINEMATICS_PARAM` 用于从上位机更新机械臂运动学参数。人脸追踪要依赖机械臂根据视觉偏差执行运动修正，所以参数更新后会写入本地 `arm_kin`、转发给舵机控制侧，并调用 `save_config()` 保存到 NVS。

```

    case CMD_SET_KINEMATICS_PARAM:
        if (self->elements.length - 2 >= (int)sizeof(arm_kin)) {
            memcpy(arm_kin, self->elements.args, sizeof(arm_kin));
            servo.tx_frame_write(0xFF, CMD_SET_KINEMATICS_PARAM, (uint8_t*)arm_kin,
sizeof(arm_kin));
            save_config();
            arm.board.buzzer.set(100, 50, 1, 3000);
        }
        break;

```

14. `CMD_FACE_TRACK` 是上位机控制人脸追踪的命令。 `sw == 0` 时停止当前追踪、关闭蜂鸣器并刷新 OLED； `sw != 0` 时只做准备提示和界面刷新，没有直接调用 `faceTracker.start()`，说明这个工程里真正启动追踪仍由本机按键触发。

```
case CMD_FACE_TRACK:
    if (self->elements.length >= 3) {
        uint8_t sw = self->elements.args[0];
        if (sw == 0) {
            faceTracker.stop();
            arm.board.buzzer.off();
            apply_oled_face_screen();
            Serial.println("[FaceTrack] CMD stop");
        } else {
            arm.board.buzzer.set(30, 30, 1, 2500);
            apply_oled_face_screen();
            Serial.println("[FaceTrack] CMD ready, press KEY1 to start");
        }
    }
    break;
```

15. `at32_packet_callback()` 处理 AT32 或舵机侧返回的数据。对人脸追踪来说，最直接相关的是舵机位置反馈：收到 `CMD_SERVO_STREAM` 或命令 96 后，代码解析 6 个舵机位置并更新时间戳，为追踪过程中的机械臂状态同步提供数据。

```
void at32_packet_callback(PacketTypeDef* rx_packet)
{
    if (is_downloading) return;

    if (rx_packet->elements.id == 0xFF &&
        rx_packet->elements.cmd == CMD_SERVO_STREAM &&
        rx_packet->elements.length >= 14) {
        for (int i = 0; i < 6; ++i) {
            int idx = i * 2;
            g_last_servo_positions[i] = (int16_t)((rx_packet->elements.args[idx + 1] << 8) |
rx_packet->elements.args[idx]);
        }
        g_last_servo_feedback_ms = millis();
        return;
    }

    if (rx_packet->elements.id == 0xFF &&
        rx_packet->elements.cmd == 96 &&
        rx_packet->elements.length >= 14) {
        for (int i = 0; i < 6; ++i) {
            int idx = i * 2;
            g_last_servo_positions[i] = (int16_t)((rx_packet->elements.args[idx + 1] << 8) |
rx_packet->elements.args[idx]);
        }
        g_last_servo_feedback_ms = millis();
        return;
    }
}
```

16. 当 AT32 返回当前机械臂坐标时，代码会解析 X、Y、Z、pitch、roll、claw 和 6 个舵机位置，并写入 `arm.current_pose` 与舵机缓存。人脸追踪要根据视觉结果持续修正机械臂位置，这些反馈数据可以帮助系统维持机械臂状态同步。

```
if (rx_packet->elements.cmd == CMD_GET_CUR_COORDS && rx_packet->elements.length
    >= 26) {
    int16_t raw_x = (int16_t)((rx_packet->elements.args[1] << 8) | rx_packet-
    >elements.args[0]);
    int16_t raw_y = (int16_t)((rx_packet->elements.args[3] << 8) | rx_packet-
    >elements.args[2]);
    int16_t raw_z = (int16_t)((rx_packet->elements.args[5] << 8) | rx_packet-
    >elements.args[4]);
    int16_t raw_p = (int16_t)((rx_packet->elements.args[7] << 8) | rx_packet-
    >elements.args[6]);
    int16_t raw_r = (int16_t)((rx_packet->elements.args[9] << 8) | rx_packet-
    >elements.args[8]);
    int16_t raw_c = (int16_t)((rx_packet->elements.args[11] << 8) | rx_packet-
    >elements.args[10]);

    arm.current_pose.x = (float)raw_x;
    arm.current_pose.y = (float)raw_y;
    arm.current_pose.z = (float)raw_z;
    arm.current_pose.pitch = (float)raw_p / 10.0f;
    arm.current_pose.roll = (float)raw_r;
    arm.current_pose.claw = (float)raw_c;

    for (int i = 0; i < 6; ++i) {
        int idx = 12 + i * 2;
        g_last_servo_positions[i] = (int16_t)((rx_packet->elements.args[idx + 1]
        << 8) | rx_packet->elements.args[idx]);
    }
    g_last_servo_feedback_ms = millis();
}
```

17. `pump_at32_feedback()`、`sync_arm_feedback()` 和 `get_last_servo_positions()` 是机械臂反馈辅助函数。它们分别负责从 AT32 串口抽取原始数据、等待新的机械臂反馈、向其他模块提供最近一次舵机位置缓存，保证追踪运行时不会只依赖过期状态。

```
void pump_at32_feedback(void)
{
    while (servo.uart->available()) {
        uint8_t c = servo.uart->read();
        at32_protocol.parsing(&c, 1);
    }
}

bool sync_arm_feedback(uint32_t timeout_ms)
{
    arm.update_status();
    uint32_t start = millis();
    uint32_t prev_feedback_ms = g_last_servo_feedback_ms;
```

```

while (millis() - start < timeout_ms) {
    serial_port.rec_handler();
    pump_at32_feedback();
    if (g_last_servo_feedback_ms != 0 && g_last_servo_feedback_ms != prev_feedback_ms) {
        return true;
    }
    delay(2);
}

return g_last_servo_feedback_ms != 0;
}

bool get_last_servo_positions(int16_t out_pos[6])
{
    if (g_last_servo_feedback_ms == 0) {
        return false;
    }
    memcpy(out_pos, g_last_servo_positions, sizeof(g_last_servo_positions));
    return true;
}

```

18. `system_loop_handler()` 是人脸追踪运行阶段的主循环。非下载状态下，它依次处理上位机串口、AT32 反馈、人脸追踪状态机、追踪结束检测、按键扫描和蜂鸣器刷新，其中 `faceTracker.update()` 是推动人脸检测和机械臂追踪动作继续运行的关键调用。

```

void system_loop_handler(void)
{
    if (!is_downloading) {
        serial_port.rec_handler();
        pump_at32_feedback();

        faceTracker.update();

        bool busy_now = faceTracker.isBusy();
        if (s_was_tracker_busy && !busy_now) {
            Serial.println("[FaceTrack] Tracking stopped, idle");
            apply_oled_face_screen();
        }
        s_was_tracker_busy = busy_now;

        arm.board.button.update();
        arm.board.buzzer.update();
    }
}

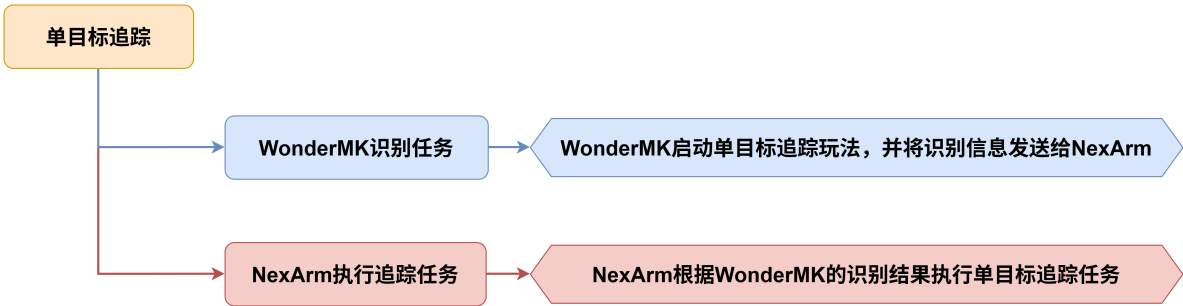
```

## 4.6 单目标追踪

### 4.6.1 项目介绍

本实验实现了基于视觉识别的单目标跟踪功能。系统能够选定画面中的一个目标，并驱动机械臂根据目标位置变化持续调整姿态，形成稳定的动态跟随效果。

### 4.6.2 实现流程



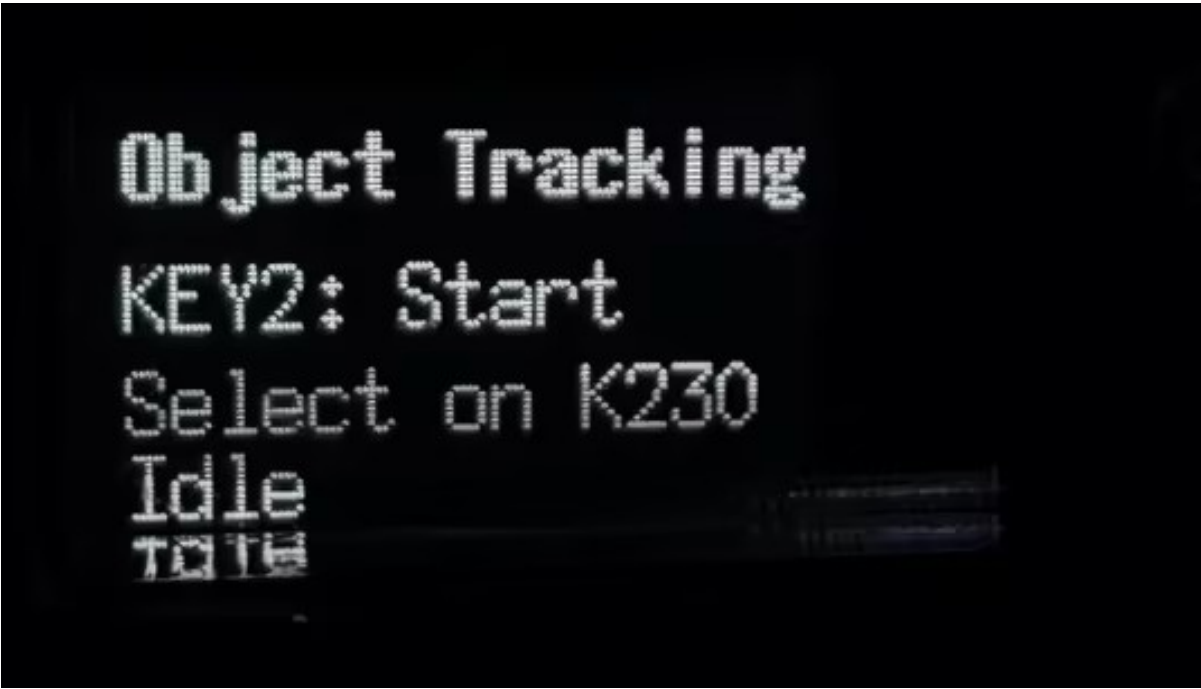
### 4.6.3 程序下载

找到本文档同目录下的“02 程序源码”文件夹，找到“04 单目标追踪”程序进行下载，具体下载步骤可参考[4.2 程序下载说明\(必看\)](#)。

### 4.6.4 运行效果

设备上电后会先完成初始化，机械臂自动回到初始观察位，屏幕上显示当前运行状态，随后进入等待状态。启动后，系统会开始对画面中的单个目标进行持续跟踪，当目标在画面中发生偏移时，机械臂末端会根据识别结果不断做小幅调整，使目标逐步回到画面中心附近；如果需要重新指定跟踪对象，可以把目标移动到画面中心区域后按下确认按键，系统就会把当前位置重新设为新的跟踪目标。整个过程表现为设备能够围绕一个被选中的目标持续做实时跟随，并在目标移动时同步修正机械臂姿态。

实际使用时，先正常上电并等待初始化结束，确认机械臂已经回到初始位、屏幕显示正常；按下 KEY2 启动按键后，系统进入目标跟踪状态；将需要跟踪的目标放到摄像头视野中，如果需要重新确认目标，就把目标移动到画面中心位置后按下确认按键；如果需要结束跟踪，再按一次启动按键即可退出运行状态并恢复等待界面。



## 4.6.5 程序分析

### system\_task\_handle.cpp

1. 文件开头先引入单目标追踪工程需要的核心模块，包括系统任务接口、全局命令、机械臂控制、USB 串口控制、参数存储、单目标追踪状态机、AT32 OTA 和文件系统等。这里最关键的是 `ObjectTrack.h`，它负责和 K230 视觉侧通信，并在人为选定一个目标后持续追踪该目标；`Robot_Arm.h` 负责机械臂动作；`usb_ctrl.h` 负责接收上位机串口命令。

```
#include "system_task_handle.h"
#include "Global.h"
#include "Robot_Arm.h"
#include "usb_ctrl.h"
#include <Preferences.h>
#include "ObjectTrack.h"
#include "AT32_OTA.h"
#include <LittleFS.h>
#include <math.h>
#include <string.h>

#define CMD_SERVO_STREAM 0x68
```

2. 这些静态对象和全局变量保存单目标追踪运行期间需要共享的系统状态。`loop_with_sys_task` 用于注册周期事件，`at32_protocol` 用来解析 AT32 侧机械臂反馈，`prefs` 用来保存机械臂参数，`is_downloading` 用来在动作组下载时暂停常规任务，`g_last_servo_positions` 和 `g_last_servo_feedback_ms` 则缓存最近一次舵机反馈。

```
static const char* TAG = "single_object_tracking";
static TimerHandle_t TIMER;
static esp_event_loop_handle_t loop_with_sys_task;
static CommProtocol_t at32_protocol;
static Preferences prefs;
static bool is_downloading = false;

static int16_t g_last_servo_positions[6] = {2048, 2048, 2048, 2048, 2048, 2048};
static uint32_t g_last_servo_feedback_ms = 0;

uint8_t global_channel = 2;
uint8_t global_acc = 245;
volatile bool servo_uart_busy = false;

static float arm_kin[9] = {
    110.45f, 225.00f, 36.97f, 145.00f, 0.0f, 130.23f, 0.0f, 50.0f, 70.5f
};
```

3. 这里定义了单目标追踪工程的运行状态标记。`s_was_tracker_busy` 用来判断追踪任务是否刚从运行状态回到空闲状态，`s_was_target_tracking` 用来判断是否刚从“等待目标”进入“正在追踪目标”，`s_button_enable_time` 和 `s_buttons_armed` 用于开机后的按键延迟使能，避免上电误触发。

```

static bool s_was_tracker_busy = false;
static bool s_was_target_tracking = false;
static uint32_t s_button_enable_time = 0;
static bool s_buttons_armed = false;

ESP_EVENT_DEFINE_BASE(SYS_TIMING_EVENTS);

```

4. `apply_oled_single_object_screen()` 负责刷新单目标追踪工程的 OLED 界面。`objectTrack.isBusy()` 为真时表示追踪功能已经启动；如果 `objectTrack.isTracking()` 为真，OLED 显示正在追踪目标，否则显示正在等待用户在 K230 端选择目标；空闲时则提示可以启动并显示 `Idle`。

```

static void apply_oled_single_object_screen(void)
{
    arm.board.oled.set_custom_text(0, "Object Tracking");

    if (objectTrack.isBusy()) {
        if (objectTrack.isTracking()) {
            arm.board.oled.set_custom_text(1, "Tracking Target");
            arm.board.oled.set_custom_text(2, "Arm Tracking");
        } else {
            arm.board.oled.set_custom_text(1, "Waiting Target");
            arm.board.oled.set_custom_text(2, "Select on K230");
        }
        arm.board.oled.set_custom_text(3, "KEY2: Stop");
    } else {
        arm.board.oled.set_custom_text(1, "KEY2: Start");
        arm.board.oled.set_custom_text(2, "Select on K230");
        arm.board.oled.set_custom_text(3, "Idle");
    }

    arm.board.oled.show_custom();
}

```

5. `perform_arm_poweron_reset()` 是单目标追踪开始前的机械臂上电复位流程。它先在 OLED 显示回零提示，再调用 `arm.reset_all(2500)` 让机械臂回到初始姿态，并在等待过程中持续抽取 AT32 反馈，保证进入追踪运行前机械臂状态已经可用。

```

static void perform_arm_poweron_reset(void)
{
    arm.board.oled.set_custom_text(0, "Object Tracking");
    arm.board.oled.set_custom_text(1, "Arm homing...");
    arm.board.oled.set_custom_text(2, "Please wait");
    arm.board.oled.set_custom_text(3, "");
    arm.board.oled.show_custom();

    arm.reset_all(2500);
    uint32_t t0 = millis();
    while (millis() - t0 < 2600) {
        pump_at32_feedback();
        delay(20);
    }
}

```

```

    for (int i = 0; i < 8; i++) {
        arm.update_status();
        delay(50);
        while (servo.uart->available()) {
            uint8_t c = servo.uart->read();
            at32_protocol.parsing(&c, 1);
        }
    }
    Serial.println("[SingleObj] Power-on arm reset done");
}

```

6. `start_single_object_tracking()` 是本地启动单目标追踪的函数。它只在 `objectTrack` 空闲时执行，启动前先调用 `objectTrack.stop()` 清理旧状态，再调用 `objectTrack.start()` 进入等待目标选择的流程；启动成功后蜂鸣器提示，并在串口日志中说明正在等待 K230 选择目标。

```

static void start_single_object_tracking(void)
{
    if (objectTrack.isBusy()) {
        Serial.println("[SingleObj] Busy, ignore KEY1 start");
        return;
    }

    objectTrack.stop();
    delay(100);
    objectTrack.start();

    if (objectTrack.isBusy()) {
        arm.board.buzzer.set(100, 100, 2, 2000);
        Serial.println("[SingleObj] KEY1 -> start, waiting K230 selection");
    } else {
        arm.board.buzzer.set(300, 100, 1, 1000);
        Serial.println("[SingleObj] start failed");
    }

    apply_oled_single_object_screen();
}

```

7. `stop_single_object_tracking()` 负责停止正在运行的单目标追踪。函数先判断 `objectTrack.isBusy()`，只有追踪任务已经启动时才停止状态机、触发蜂鸣器提示并刷新 OLED，避免空闲状态下重复执行停止逻辑。

```

static void stop_single_object_tracking(void)
{
    if (!objectTrack.isBusy()) {
        return;
    }

    objectTrack.stop();
    arm.board.buzzer.set(80, 80, 1, 1800);
    apply_oled_single_object_screen();
    Serial.println("[SingleObj] KEY1 -> stop");
}

```

8. `buttons_ready()` 是开机后的按键保护函数。系统刚启动时会先读取并清掉两个按键的点击状态，在 `s_button_enable_time` 到达之前都不允许按键触发；时间到达后把 `s_buttons_armed` 置为真，后续按键才真正生效。

```
static bool buttons_ready(void)
{
    if (s_buttons_armed) {
        return true;
    }

    (void)arm.board.button.is_clicked(0);
    (void)arm.board.button.is_clicked(1);

    if (millis() < s_button_enable_time) {
        return false;
    }

    s_buttons_armed = true;
    Serial.println("[SingleObj] Buttons armed");
    return false;
}
```

9. `save_config()` 和 `load_config()` 负责保存与单目标追踪相关的机械臂基础参数。这里保存通道、加速度和运动学参数；加载时如果 NVS 中存在 `arm_kin`，会先做一个简单校验，再覆盖默认运动学参数，避免异常参数影响目标追踪动作。

```
void save_config(void)
{
    prefs.begin("robot_cfg", false);
    prefs.putUChar("channel", global_channel);
    prefs.putUChar("acc", global_acc);
    prefs.putBytes("arm_kin", arm_kin, sizeof(arm_kin));
    prefs.end();
}

void load_config(void)
{
    prefs.begin("robot_cfg", true);
    global_channel = prefs.getUChar("channel", 2);
    global_acc = prefs.getUChar("acc", 245);

    if (prefs.isKey("arm_kin")) {
        float saved_kin[9];
        prefs.getBytes("arm_kin", saved_kin, sizeof(saved_kin));
        if (fabsf(saved_kin[0] - arm_kin[0]) < 0.01f) {
            memcpy(arm_kin, saved_kin, sizeof(arm_kin));
        }
    }
    prefs.end();
}
```

10. 周期定时器回调负责投递单目标追踪运行时需要的基础维护事件。它不会直接操作硬件，而是按周期投递蜂鸣器、电池、机械臂状态和 OLED 刷新事件；如果正在下载动作组，则直接返回，避免下载流程被周期任务干扰。

```
static void sys_timer_post_callback(TimerHandle_t xTimer)
{
    if (is_downloading) return;

    static uint32_t count = 0;
    if (count % BUZZER_UPDATE_PERIOD == 0) {
        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS, TIMING_EVENT_BUZZER_UPDATE,
        NULL, 0, 0);
    }
    if (count % BAT_UPDATE_PERIOD == 0) {
        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS, TIMING_EVENT_BAT_UPDATE,
        NULL, 0, 0);
    }
    if (count % SERVO_STATUS_UPDATE_PERIOD == 0) {
        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS,
        TIMING_EVENT_SERVO_STATUS_UPDATE, NULL, 0, 0);
    }
    if (count % OLED_REFRESH_PERIOD == 0) {
        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS, TIMING_EVENT_OLED_REFRESH,
        NULL, 0, 0);
    }

    count += TIMER_PERIOD;
}
```

11. `sys_timer_sub_handler()` 接收周期事件并执行具体维护动作。蜂鸣器和电池是主板基础状态维护，`arm.update_status()` 用于持续刷新机械臂状态，`apply_oled_single_object_screen()` 则让 OLED 根据 `ObjectTrack` 当前状态显示空闲、等待目标或正在追踪。

```
static void sys_timer_sub_handler(void* handler_args, esp_event_base_t base, int32_t id,
void* event_data)
{
    if (is_downloading) return;

    switch (id) {
        case TIMING_EVENT_BUZZER_UPDATE:
            arm.board.buzzer.update();
            break;
        case TIMING_EVENT_BAT_UPDATE:
            arm.board.bat.update();
            break;
        case TIMING_EVENT_SERVO_STATUS_UPDATE:
            arm.update_status();
            break;
        case TIMING_EVENT_OLED_REFRESH:
            apply_oled_single_object_screen();
            break;
        default:
            break;
    }
}
```

```

    }
}

```

12. `func_ctrl_callback()` 是上位机串口命令入口。和单目标追踪主流程直接相关的是动作组下载保护、机械臂运动学参数设置和 `CMD_SELF_LEARN_TRACK`；动作组下载时会把 `is_downloading` 置为真，让主循环暂停单目标追踪状态机和按键处理。

```

static void func_ctrl_callback(PacketTypeDef* self)
{
    uint8_t len;
    uint16_t bat_level;

    if (self->elements.id == 0xFF && self->elements.cmd == CMD_ACTION_GROUP_DOWNLOAD) {
        is_downloading = true;
        arm.board.action_group_download(self->elements.args[0], self->elements.args, self->elements.length - 2);
        is_downloading = false;
        return;
    }

    if (self->elements.id != 0xFF) return;

    switch (self->elements.cmd) {

```

13. `CMD_SET_KINEMATICS_PARAM` 用于从上位机更新机械臂运动学参数。单目标追踪需要根据目标中心位置持续修正机械臂动作，因此参数更新后会写入本地 `arm_kin`、转发给舵机控制侧，并调用 `save_config()` 保存到 NVS。

```

case CMD_SET_KINEMATICS_PARAM:
    if (self->elements.length - 2 >= (int)sizeof(arm_kin)) {
        memcpy(arm_kin, self->elements.args, sizeof(arm_kin));
        servo.tx_frame_write(0xFF, CMD_SET_KINEMATICS_PARAM, (uint8_t*)arm_kin, sizeof(arm_kin));
        save_config();
        arm.board.buzzer.set(100, 50, 1, 3000);
    }
    break;

```

14. `CMD_SELF_LEARN_TRACK` 是上位机参与单目标追踪的主要命令。`sw == 0` 时停止追踪；`sw == 2` 时表示目标已经确认，代码会解析目标中心坐标和目标 ID，然后调用 `objectTrack.confirm(id, cx, cy)` 把 K230 或上位机选中的目标交给追踪状态机。

```

case CMD_SELF_LEARN_TRACK:
    if (self->elements.length >= 3) {
        uint8_t sw = self->elements.args[0];
        if (sw == 0) {
            objectTrack.stop();
            arm.board.buzzer.off();
            apply_oled_single_object_screen();
            Serial.println("[SingleObj] CMD stop");

```

```

        } else if (sw == 2) {
            int cx = 160;
            int cy = 120;
            string id = "1";

            if (self->elements.length >= 7) {
                cx = (int16_t)((self->elements.args[1] << 8) | self-
>elements.args[2]);
                cy = (int16_t)((self->elements.args[3] << 8) | self-
>elements.args[4]);

                uint8_t n_len = self->elements.args[5];
                if (n_len > 0 && self->elements.length >= 7 + n_len) {
                    char name_buf[32] = {0};
                    int copy_len = (n_len < 31) ? n_len : 31;
                    memcpy(name_buf, &self->elements.args[6], copy_len);
                    id = String(name_buf);
                }
            }
            objectTrack.confirm(id, cx, cy);
            arm.board.buzzer.set(100, 50, 2, 3000);
            apply_oled_single_object_screen();
            Serial.println("[SingleObj] CMD confirm target");

```

15. 同一个 `CMD_SELF_LEARN_TRACK` 分支里，除停止和确认目标以外的非零 `sw` 会进入准备状态。代码不会直接完成目标确认，而是蜂鸣器提示、刷新 OLED，并打印“按键启动”的日志，表示当前只是让系统进入可启动或等待用户选择目标的状态。

```

        } else {
            arm.board.buzzer.set(30, 30, 1, 2500);
            apply_oled_single_object_screen();
            Serial.println("[SingleObj] CMD ready, press KEY1 to start");
        }
    }
    break;

```

16. `at32_packet_callback()` 处理 AT32 或舵机侧返回的数据。对单目标追踪来说，最直接相关的是舵机位置反馈：收到 `CMD_SERVO_STREAM` 或命令 96 后，代码解析 6 个舵机位置并更新时间戳，为追踪过程中的机械臂状态同步提供数据。

```

void at32_packet_callback(PacketTypeDef* rx_packet)
{
    if (is_downloading) return;

    if (rx_packet->elements.id == 0xFF &&
        rx_packet->elements.cmd == CMD_SERVO_STREAM &&
        rx_packet->elements.length >= 14) {
        for (int i = 0; i < 6; ++i) {
            int idx = i * 2;
            g_last_servo_positions[i] = (int16_t)((rx_packet->elements.args[idx + 1] << 8) |
rx_packet->elements.args[idx]);

```

```

    }
    g_last_servo_feedback_ms = millis();
    return;
}

if (rx_packet->elements.id == 0xFF &&
    rx_packet->elements.cmd == 96 &&
    rx_packet->elements.length >= 14) {
    for (int i = 0; i < 6; ++i) {
        int idx = i * 2;
        g_last_servo_positions[i] = (int16_t)((rx_packet->elements.args[idx + 1] << 8) |
rx_packet->elements.args[idx]);
    }
    g_last_servo_feedback_ms = millis();
    return;
}

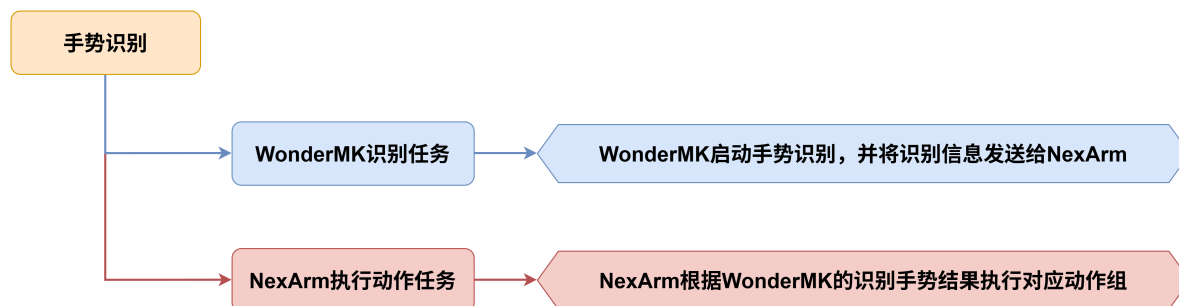
```

## 4.7 手势识别

### 4.7.1 项目介绍

本实验实现了基于视觉识别的手势控制功能。系统能够识别不同手势结果，并驱动机械臂执行对应的预设动作组，同时在屏幕上显示当前状态和最近一次识别到的手势。

### 4.7.2 实现流程



### 4.7.3 程序下载

找到本文档同目录下的“02 程序源码”文件夹，找到“05 手势识别”程序进行下载，具体下载步骤可参考[4.2 程序下载说明\(必看\)](#)。

### 4.7.4 运行效果

设备上电后会先完成初始化，机械臂自动回到初始等待位，屏幕上显示当前运行状态，随后进入待机状态。启动后，系统会持续检测摄像头画面中的手势信息，当识别结果稳定后，会触发对应的预设动作组，使机械臂执行相应动作；屏幕还会同步显示最近一次识别到的手势名称，便于观察当前识别结果。停止运行后，识别流程结束，系统回到等待状态，但界面上仍可看到最近一次识别结果，方便继续调试和演示。

实际使用时，先正常上电并等待初始化结束，确认机械臂已经回到初始位；将手放到摄像头可识别范围内；按下 KEY2 启动按键后，系统开始进入手势识别状态，此时做出不同手势即可触发对应动作；如果需要结束识别，再按一次启动按键即可停止；如果需要让机械臂重新回到初始姿态，可以按下复位按键，设备会立即执行回位动作，便于重新开始新一轮演示。



### 4.7.5 程序分析

system\_task\_handle.cpp

1. 文件开头先引入手势识别工程需要的核心模块，包括系统任务接口、全局命令、机械臂控制、USB 串口控制、参数存储、手势识别状态机、AT32 OTA 和文件系统等。这里最关键的是 `GestureTracker.h`，它负责和 K230 视觉侧通信并获取手势识别结果；`Robot_Arm.h` 负责机械臂和主板外设；`usb_ctrl.h` 负责上位机串口命令入口。

```
#include "system_task_handle.h"
#include "Global.h"
#include "Robot_Arm.h"
#include "usb_ctrl.h"
#include <Preferences.h>
#include "GestureTracker.h"
#include "AT32_OTA.h"
#include <LittleFS.h>
#include <math.h>
#include <string.h>

#define CMD_SERVO_STREAM 0x68
```

2. 这些静态对象和全局变量保存手势识别运行期间需要共享的系统状态。`loop_with_sys_task` 用于注册周期事件，`at32_protocol` 用来解析 AT32 侧机械臂反馈，`prefs` 用来保存机械臂参数，`is_downloading` 用来在动作组下载时暂停常规任务，`g_last_servo_positions` 和 `g_last_servo_feedback_ms` 则缓存最近一次舵机反馈。

```
static const char* TAG = "gesture_recognition";
static TimerHandle_t TIMER;
static esp_event_loop_handle_t loop_with_sys_task;
static CommProtocol_t at32_protocol;
static Preferences prefs;
```

```
static bool is_downloading = false;

static int16_t g_last_servo_positions[6] = {2048, 2048, 2048, 2048, 2048, 2048};
static uint32_t g_last_servo_feedback_ms = 0;

uint8_t global_channel = 2;
uint8_t global_acc = 245;
volatile bool servo_uart_busy = false;

static float arm_kin[9] = {
    110.45f, 225.00f, 36.97f, 145.00f, 0.0f, 130.23f, 0.0f, 50.0f, 70.5f
};
```

3. 这里定义了手势识别工程的运行状态标记。 `s_was_tracker_busy` 用来判断识别任务是否刚从运行状态回到空闲状态， `s_button_enable_time` 和 `s_buttons_armed` 用于开机后的按键延迟使能，避免上电过程中误触发手势识别启动或停止。

```
static bool s_was_tracker_busy = false;
static uint32_t s_button_enable_time = 0;
static bool s_buttons_armed = false;

ESP_EVENT_DEFINE_BASE(SYS_TIMING_EVENTS);
```

4. `apply_oled_gesture_screen()` 负责刷新手势识别工程的 OLED 界面。 `gestureTracker.isBusy()` 为真时显示正在识别，并通过 `gestureTracker.getLastGestureStr()` 显示最近识别到的手势；如果没有有效结果则显示 `No Gesture`。空闲时显示启动入口、 `Gesture Ready` 和 `Idle`。

```
static void apply_oled_gesture_screen(void)
{
    arm.board.oled.set_custom_text(0, "Gesture Recognition");

    if (gestureTracker.isBusy()) {
        const char* gesture = gestureTracker.getLastGestureStr();
        arm.board.oled.set_custom_text(1, "Recognizing");
        arm.board.oled.set_custom_text(2, (gesture && gesture[0]) ? gesture : "No Gesture");
        arm.board.oled.set_custom_text(3, "KEY2: Stop");
    } else {
        arm.board.oled.set_custom_text(1, "KEY2: Start");
        arm.board.oled.set_custom_text(2, "Gesture Ready");
        arm.board.oled.set_custom_text(3, "Idle");
    }

    arm.board.oled.show_custom();
}
```

5. `perform_arm_poweron_reset()` 是手势识别工程启动前的机械臂上电复位流程。它先在 OLED 显示回零提示，再调用 `arm.reset_all(2500)` 让机械臂回到初始姿态，并在等待过程中持续抽取 AT32 反馈，保证进入识别运行前机械臂状态已经可用。

```
static void perform_arm_poweron_reset(void)
```

```

{
    arm.board.oled.set_custom_text(0, "Gesture Recognition");
    arm.board.oled.set_custom_text(1, "Arm homing...");
    arm.board.oled.set_custom_text(2, "Please wait");
    arm.board.oled.set_custom_text(3, "");
    arm.board.oled.show_custom();

    arm.reset_all(2500);
    uint32_t t0 = millis();
    while (millis() - t0 < 2600) {
        pump_at32_feedback();
        delay(20);
    }
    for (int i = 0; i < 8; i++) {
        arm.update_status();
        delay(50);
        while (servo.uart->available()) {
            uint8_t c = servo.uart->read();
            at32_protocol.parsing(&c, 1);
        }
    }
    Serial.println("[GestureRec] Power-on arm reset done");
}

```

6. `start_gesture_recognition()` 是本地启动手势识别的函数。它只在 `GestureTracker` 空闲时执行，启动前先调用 `gestureTracker.stop()` 清理旧状态，再调用 `gestureTracker.start()` 进入识别流程，随后根据 `gestureTracker.isBusy()` 判断启动是否成功并给出蜂鸣器提示。

```

static void start_gesture_recognition(void)
{
    if (gestureTracker.isBusy()) {
        Serial.println("[GestureRec] Busy, ignore KEY1 start");
        return;
    }

    gestureTracker.stop();
    delay(100);
    gestureTracker.start();

    if (gestureTracker.isBusy()) {
        arm.board.buzzer.set(100, 100, 2, 2000);
        Serial.println("[GestureRec] KEY1 -> start");
    } else {
        arm.board.buzzer.set(300, 100, 1, 1000);
        Serial.println("[GestureRec] start failed");
    }

    apply_oled_gesture_screen();
}

```

7. `stop_gesture_recognition()` 负责停止正在运行的手势识别。函数先判断 `gestureTracker.isBusy()`，只有识别任务确实在运行时才停止状态机、触发蜂鸣器提示并刷新 OLED，避免空闲状态下重复执行停止逻辑。

```
static void stop_gesture_recognition(void)
{
    if (!gestureTracker.isBusy()) {
        return;
    }

    gestureTracker.stop();
    arm.board.buzzer.set(80, 80, 1, 1800);
    apply_oled_gesture_screen();
    Serial.println("[GestureRec] KEY1 -> stop");
}
```

8. `buttons_ready()` 是开机后的按键保护函数。系统刚启动时会先读取并清掉两个按键的点击状态，在 `s_button_enable_time` 到达之前都不允许按键触发；时间到达后把 `s_buttons_armed` 置为真，后续按键才真正生效。

```
static bool buttons_ready(void)
{
    if (s_buttons_armed) {
        return true;
    }

    (void)arm.board.button.is_clicked(0);
    (void)arm.board.button.is_clicked(1);

    if (millis() < s_button_enable_time) {
        return false;
    }

    s_buttons_armed = true;
    Serial.println("[GestureRec] Buttons armed");
    return false;
}
```

9. `save_config()` 和 `load_config()` 负责保存与手势识别工程相关的机械臂基础参数。这里保存通道、加速度和运动学参数；加载时如果 NVS 中存在 `arm_kin`，会先做一个简单校验，再覆盖默认运动学参数，避免异常参数影响机械臂基础状态。

```
void save_config(void)
{
    prefs.begin("robot_cfg", false);
    prefs.putUChar("channel", global_channel);
    prefs.putUChar("acc", global_acc);
    prefs.putBytes("arm_kin", arm_kin, sizeof(arm_kin));
    prefs.end();
}

void load_config(void)
{
    prefs.begin("robot_cfg", true);
    global_channel = prefs.getUChar("channel", 2);
}
```

```

global_acc = prefs.getUChar("acc", 245);

if (prefs.isKey("arm_kin")) {
    float saved_kin[9];
    prefs.getBytes("arm_kin", saved_kin, sizeof(saved_kin));
    if (fabsf(saved_kin[0] - arm_kin[0]) < 0.01f) {
        memcpy(arm_kin, saved_kin, sizeof(arm_kin));
    }
}
prefs.end();
}

```

10. 周期定时器回调负责投递手势识别运行时需要的基础维护事件。它不会直接操作硬件，而是按周期投递蜂鸣器、电池、机械臂状态和 OLED 刷新事件；如果正在下载动作组，则直接返回，避免下载流程被周期任务干扰。

```

static void sys_timer_post_callback(TimerHandle_t xTimer)
{
    if (is_downloading) return;

    static uint32_t count = 0;
    if (count % BUZZER_UPDATE_PERIOD == 0) {
        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS, TIMING_EVENT_BUZZER_UPDATE,
        NULL, 0, 0);
    }
    if (count % BAT_UPDATE_PERIOD == 0) {
        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS, TIMING_EVENT_BAT_UPDATE,
        NULL, 0, 0);
    }
    if (count % SERVO_STATUS_UPDATE_PERIOD == 0) {
        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS,
        TIMING_EVENT_SERVO_STATUS_UPDATE, NULL, 0, 0);
    }
    if (count % OLED_REFRESH_PERIOD == 0) {
        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS, TIMING_EVENT_OLED_REFRESH,
        NULL, 0, 0);
    }

    count += TIMER_PERIOD;
}

```

11. `sys_timer_sub_handler()` 接收周期事件并执行具体维护动作。蜂鸣器和电池是主板基础状态维护，`arm.update_status()` 用于持续刷新机械臂状态，`apply_oled_gesture_screen()` 则让 OLED 根据 `GestureTracker` 当前状态显示空闲、正在识别或最近识别到的手势。

```

static void sys_timer_sub_handler(void* handler_args, esp_event_base_t base, int32_t id,
void* event_data)
{
    if (is_downloading) return;

    switch (id) {
        case TIMING_EVENT_BUZZER_UPDATE:
            arm.board.buzzer.update();
    }
}

```

```

        break;
    case TIMING_EVENT_BAT_UPDATE:
        arm.board.bat.update();
        break;
    case TIMING_EVENT_SERVO_STATUS_UPDATE:
        arm.update_status();
        break;
    case TIMING_EVENT_OLED_REFRESH:
        apply_oled_gesture_screen();
        break;
    default:
        break;
}
}

```

12. `func_ctrl_callback()` 是上位机串口命令入口。和手势识别主流程直接相关的是动作组下载保护、机械臂运动学参数设置和 `CMD_GESTURE_TRACK`；动作组下载时会把 `is_downloading` 置为真，让主循环暂停手势识别状态机和按键处理。

```

static void func_ctrl_callback(PacketTypeDef* self)
{
    uint8_t len;
    uint16_t bat_level;

    if (self->elements.id == 0xFF && self->elements.cmd == CMD_ACTION_GROUP_DOWNLOAD) {
        is_downloading = true;
        arm.board.action_group_download(self->elements.args[0], self->elements.args, self->elements.length - 2);
        is_downloading = false;
        return;
    }

    if (self->elements.id != 0xFF) return;

    switch (self->elements.cmd) {

```

13. `CMD_SET_KINEMATICS_PARAM` 用于从上位机更新机械臂运动学参数。虽然手势识别本身主要是视觉识别和显示结果，但工程仍然依赖机械臂基础参数和状态同步，所以参数更新后会写入本地 `arm_kin`、转发给舵机控制侧，并调用 `save_config()` 保存到 NVS。

```

    case CMD_SET_KINEMATICS_PARAM:
        if (self->elements.length - 2 >= (int)sizeof(arm_kin)) {
            memcpy(arm_kin, self->elements.args, sizeof(arm_kin));
            servo.tx_frame_write(0xFF, CMD_SET_KINEMATICS_PARAM, (uint8_t*)arm_kin, sizeof(arm_kin));
            save_config();
            arm.board.buzzer.set(100, 50, 1, 3000);
        }
        break;

```

14. `CMD_GESTURE_TRACK` 是上位机控制手势识别的主要命令。`sw == 0` 时停止当前识别、关闭蜂鸣器并刷新 OLED；`sw != 0` 时只做准备提示和界面刷新，没有直接调用 `gestureTracker.start()`，说明这个工程里真正启动识别仍由本机按键触发。

```
case CMD_GESTURE_TRACK:
    if (self->elements.length >= 3) {
        uint8_t sw = self->elements.args[0];
        if (sw == 0) {
            gestureTracker.stop();
            arm.board.buzzer.off();
            apply_oled_gesture_screen();
            Serial.println("[GestureRec] CMD stop");
        } else {
            arm.board.buzzer.set(30, 30, 1, 2500);
            apply_oled_gesture_screen();
            Serial.println("[GestureRec] CMD ready, press KEY1 to start");
        }
    }
    break;
```

15. `at32_packet_callback()` 处理 AT32 或舵机侧返回的数据。对手势识别工程来说，最直接相关的是舵机位置反馈：收到 `CMD_SERVO_STREAM` 或命令 96 后，代码解析 6 个舵机位置并更新时间戳，为机械臂状态同步和外部查询提供数据。

```
void at32_packet_callback(PacketTypeDef* rx_packet)
{
    if (is_downloading) return;

    if (rx_packet->elements.id == 0xFF &&
        rx_packet->elements.cmd == CMD_SERVO_STREAM &&
        rx_packet->elements.length >= 14) {
        for (int i = 0; i < 6; ++i) {
            int idx = i * 2;
            g_last_servo_positions[i] = (int16_t)((rx_packet->elements.args[idx + 1] << 8) |
rx_packet->elements.args[idx]);
        }
        g_last_servo_feedback_ms = millis();
        return;
    }

    if (rx_packet->elements.id == 0xFF &&
        rx_packet->elements.cmd == 96 &&
        rx_packet->elements.length >= 14) {
        for (int i = 0; i < 6; ++i) {
            int idx = i * 2;
            g_last_servo_positions[i] = (int16_t)((rx_packet->elements.args[idx + 1] << 8) |
rx_packet->elements.args[idx]);
        }
        g_last_servo_feedback_ms = millis();
        return;
    }
}
```

16. 当 AT32 返回当前机械臂坐标时，代码会解析 X、Y、Z、pitch、roll、claw 和 6 个舵机位置，并写入 `arm.current_pose` 与舵机缓存。手势识别虽然重点在视觉结果，但这些反馈仍用于保持机械臂状态、上位机查询和系统运行同步。

```
if (rx_packet->elements.cmd == CMD_GET_CUR_COORDS && rx_packet->elements.length
    >= 26) {
    int16_t raw_x = (int16_t)((rx_packet->elements.args[1] << 8) | rx_packet-
    >elements.args[0]);
    int16_t raw_y = (int16_t)((rx_packet->elements.args[3] << 8) | rx_packet-
    >elements.args[2]);
    int16_t raw_z = (int16_t)((rx_packet->elements.args[5] << 8) | rx_packet-
    >elements.args[4]);
    int16_t raw_p = (int16_t)((rx_packet->elements.args[7] << 8) | rx_packet-
    >elements.args[6]);
    int16_t raw_r = (int16_t)((rx_packet->elements.args[9] << 8) | rx_packet-
    >elements.args[8]);
    int16_t raw_c = (int16_t)((rx_packet->elements.args[11] << 8) | rx_packet-
    >elements.args[10]);

    arm.current_pose.x = (float)raw_x;
    arm.current_pose.y = (float)raw_y;
    arm.current_pose.z = (float)raw_z;
    arm.current_pose.pitch = (float)raw_p / 10.0f;
    arm.current_pose.roll = (float)raw_r;
    arm.current_pose.claw = (float)raw_c;

    for (int i = 0; i < 6; ++i) {
        int idx = 12 + i * 2;
        g_last_servo_positions[i] = (int16_t)((rx_packet->elements.args[idx + 1]
        << 8) | rx_packet->elements.args[idx]);
    }
    g_last_servo_feedback_ms = millis();
}
```

17. `pump_at32_feedback()`、`sync_arm_feedback()` 和 `get_last_servo_positions()` 是机械臂反馈辅助函数。它们分别负责从 AT32 串口抽取原始数据、等待新的机械臂反馈、向其他模块提供最近一次舵机位置缓存，保证系统运行时不会只依赖过期状态。

```
void pump_at32_feedback(void)
{
    while (servo.uart->available()) {
        uint8_t c = servo.uart->read();
        at32_protocol.parsing(&c, 1);
    }
}

bool sync_arm_feedback(uint32_t timeout_ms)
{
    arm.update_status();
    uint32_t start = millis();
    uint32_t prev_feedback_ms = g_last_servo_feedback_ms;
```

```

    while (millis() - start < timeout_ms) {
        serial_port.rec_handler();
        pump_at32_feedback();
        if (g_last_servo_feedback_ms != 0 && g_last_servo_feedback_ms != prev_feedback_ms) {
            return true;
        }
        delay(2);
    }

    return g_last_servo_feedback_ms != 0;
}

bool get_last_servo_positions(int16_t out_pos[6])
{
    if (g_last_servo_feedback_ms == 0) {
        return false;
    }
    memcpy(out_pos, g_last_servo_positions, sizeof(g_last_servo_positions));
    return true;
}

```

18. `system_loop_handler()` 是手势识别运行阶段的主循环。非下载状态下，它依次处理上位机串口、AT32 反馈和 `gestureTracker.update()`；随后检测识别任务是否刚从运行状态回到空闲状态，如果已经停止，就刷新 OLED 回到待机界面。

```

void system_loop_handler(void)
{
    if (!is_downloading) {
        serial_port.rec_handler();
        pump_at32_feedback();

        gestureTracker.update();

        bool busy_now = gestureTracker.isBusy();
        if (s_was_tracker_busy && !busy_now) {
            Serial.println("[GestureRec] Recognition stopped, idle");
            apply_oled_gesture_screen();
        }
        s_was_tracker_busy = busy_now;

        arm.board.button.update();
        arm.board.buzzer.update();
    }
}

```

19. 主循环中的按键逻辑只使用按键 ID 1 作为手势识别的启停入口。按键未使能时直接返回；按键使能后，如果当前正在识别就调用 `stop_gesture_recognition()`，否则调用 `start_gesture_recognition()`；下载状态下只处理上位机串口，不推进手势识别状态机。

```

if (!buttons_ready()) {
    s_was_tracker_busy = gestureTracker.isBusy();
    return;
}

```

```

        if (arm.board.button.is_clicked(1)) {
            if (gestureTracker.isBusy()) {
                stop_gesture_recognition();
            } else {
                start_gesture_recognition();
            }
        }

        s_was_tracker_busy = gestureTracker.isBusy();
    } else {
        serial_port.rec_handler();
    }
}

```

20. `register_system_task()` 是手势识别工程的初始化入口。它先保存事件循环句柄、设置 CPU 频率、加载机械臂配置，然后挂载 LittleFS，为后续动作组和系统文件访问做准备。

```

void register_system_task(esp_event_loop_handle_t *event_loop)
{
    loop_with_sys_task = *event_loop;
    setCpuFrequencyMhz(240);
    load_config();

    if (!LittleFS.begin(true)) {
        Serial.println("[LittleFS] Mount failed, formatting...");
        LittleFS.format();
        LittleFS.begin();
    }
    Serial.println("[LittleFS] Mounted successfully");
}

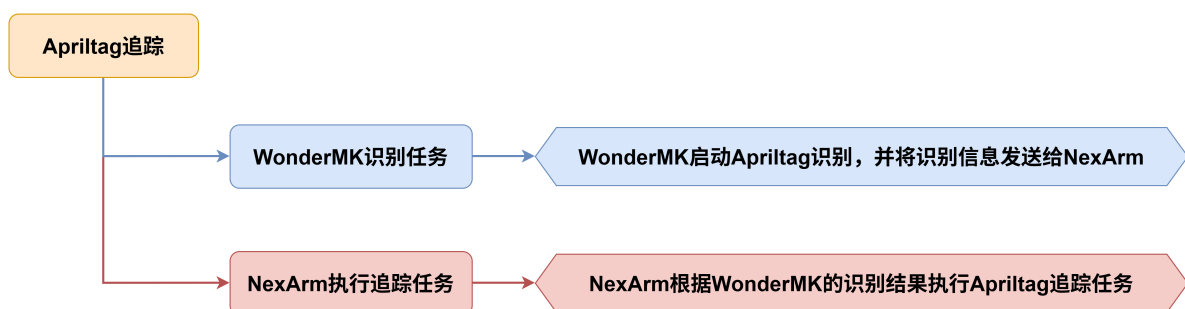
```

## 4.8 Apriltag追踪

### 4.8.1 项目介绍

本实验实现了基于 Apriltag 标签识别的机械臂自动追踪功能。系统能够根据视觉识别结果持续调整机械臂末端位置，使目标标签尽量保持在画面中心附近。

### 4.8.2 实现流程



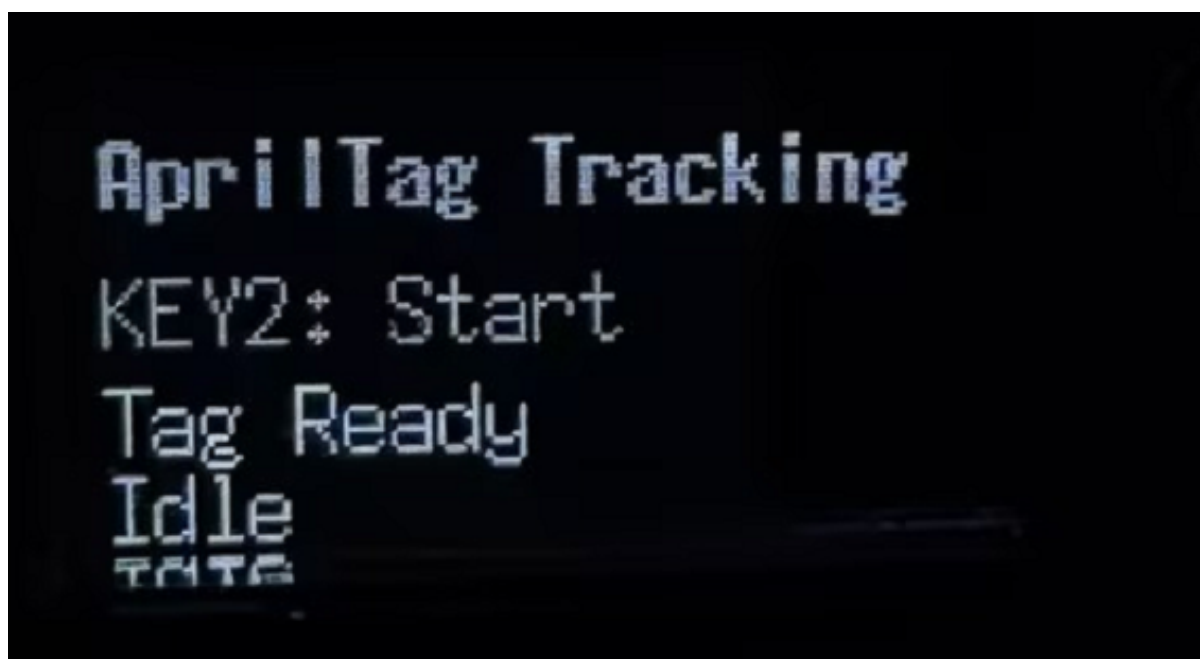
### 4.8.3 程序下载

找到本文档同目录下的“02 程序源码”文件夹，找到“06 Apriltag追踪”程序进行下载，具体下载步骤可参考[4.2 程序下载说明\(必看\)](#)。

### 4.8.4 运行效果

设备上电后会先完成初始化，机械臂自动回到初始观察位，屏幕上显示当前运行状态，随后进入等待状态。启动后，系统会持续识别画面中的标签目标，并根据目标在画面中的偏移不断修正机械臂位置，使末端跟随标签做左右和上下方向的动态调整；当停止运行后，跟随过程结束，机械臂回到初始姿态，界面状态也同步恢复到关闭状态。整体表现为设备能够对标签目标进行实时识别，并驱动机械臂稳定地进行动态追踪。

实际使用时，先正常上电并等待初始化结束，确认机械臂已经回到初始位；将带有标签的目标放到摄像头可识别范围内；按下 KEY2 启动按键后，系统开始进入追踪状态，机械臂会随着目标位置变化持续调整姿态；如果需要结束追踪，再按一次启动按键即可停止。



### 4.8.5 程序分析

`system_task_handle.cpp`

1. 文件开头先引入 Apriltag 追踪工程需要的核心模块，包括系统任务接口、全局命令、机械臂控制、USB 串口控制、参数存储、Apriltag 追踪状态机、AT32 OTA 和文件系统等。这里最关键的是 `AprilTagTracker.h`，它负责和 K230 视觉侧通信并根据 Apriltag 位置推动追踪；`Robot_Arm.h` 负责机械臂和主板外设；`usb_ctrl.h` 负责上位机串口命令入口。

```
#include "system_task_handle.h"
#include "Global.h"
#include "Robot_Arm.h"
#include "usb_ctrl.h"
#include <Preferences.h>
#include "AprilTagTracker.h"
#include "AT32_OTA.h"
#include <LittleFS.h>
#include <math.h>
#include <string.h>

#define CMD_SERVO_STREAM 0x68
```

2. 这些静态对象和全局变量保存 Apriltag 追踪运行期间需要共享的系统状态。`loop\_with\_sys\_task` 用于注册周期事件，`at32\_protocol` 用来解析 AT32 侧机械臂反馈，`prefs` 用来保存机械臂参数，`is\_downloading` 用来在动作组下载时暂停常规任务，`g\_last\_servo\_positions` 和 `g\_last\_servo\_feedback\_ms` 则缓存最近一次舵机反馈。

```
static const char* TAG = "apriltag_tracking";
static TimerHandle_t TIMER;
static esp_event_loop_handle_t loop_with_sys_task;
static CommProtocol_t at32_protocol;
static Preferences prefs;
static bool is_downloading = false;

static int16_t g_last_servo_positions[6] = {2048, 2048, 2048, 2048, 2048, 2048};
static uint32_t g_last_servo_feedback_ms = 0;

uint8_t global_channel = 2;
uint8_t global_acc = 245;
volatile bool servo_uart_busy = false;

static float arm_kin[9] = {
    110.45f, 225.00f, 36.97f, 145.00f, 0.0f, 130.23f, 0.0f, 50.0f, 70.5f
};
```

3. 这里定义了 Apriltag 追踪工程的运行状态标记。`s\_was\_tracker\_busy` 用来判断追踪任务是否刚从运行状态回到空闲状态，`s\_button\_enable\_time` 和 `s\_buttons\_armed` 用于开机后的按键延迟使能，避免上电过程中误触发 Apriltag 追踪启动或停止。

```
static bool s_was_tracker_busy = false;
static uint32_t s_button_enable_time = 0;
static bool s_buttons_armed = false;

ESP_EVENT_DEFINE_BASE(SYS_TIMING_EVENTS);
```

4. `apply\_oled\_apriltag\_screen()` 负责刷新 Apriltag 追踪工程的 OLED 界面。`aprilTagTracker.isBusy()` 为真时显示正在追踪标签和搜索状态，空闲时显示启动入口、`Tag Ready` 和 `Idle`，让用户能从屏幕判断当前是否可以启动追踪。

```
static void apply_oled_apriltag_screen(void)
{
    arm.board.oled.set_custom_text(0, "AprilTag Tracking");

    if (aprilTagTracker.isBusy()) {
        arm.board.oled.set_custom_text(1, "Tracking Tag");
        arm.board.oled.set_custom_text(2, "Status: Search");
        arm.board.oled.set_custom_text(3, "KEY2: Stop");
    } else {
        arm.board.oled.set_custom_text(1, "KEY2: Start");
        arm.board.oled.set_custom_text(2, "Tag Ready");
        arm.board.oled.set_custom_text(3, "Idle");
    }

    arm.board.oled.show_custom();
}
```

5. `perform_arm_poweron_reset()` 是 Apriltag 追踪启动前的机械臂上电复位流程。它先在 OLED 显示回零提示，再调用 `arm.reset_all(2500)` 让机械臂回到初始姿态，并在等待过程中持续抽取 AT32 反馈，保证进入追踪运行前机械臂状态已经可用。

```
static void perform_arm_poweron_reset(void)
{
    arm.board.oled.set_custom_text(0, "AprilTag Tracking");
    arm.board.oled.set_custom_text(1, "Arm homing...");
    arm.board.oled.set_custom_text(2, "Please wait");
    arm.board.oled.set_custom_text(3, "");
    arm.board.oled.show_custom();

    arm.reset_all(2500);
    uint32_t t0 = millis();
    while (millis() - t0 < 2600) {
        pump_at32_feedback();
        delay(20);
    }
    for (int i = 0; i < 8; i++) {
        arm.update_status();
        delay(50);
        while (servo.uart->available()) {
            uint8_t c = servo.uart->read();
            at32_protocol.parsing(&c, 1);
        }
    }
    Serial.println("[AprilTagTrack] Power-on arm reset done");
}
```

6. `start_apriltag_tracking()` 是本地启动 Apriltag 追踪的函数。它只在 `AprilTagTracker` 空闲时执行，启动前先调用 `aprilTagTracker.stop()` 清理旧状态，再调用 `aprilTagTracker.start()` 进入追踪流程，随后根据 `aprilTagTracker.isBusy()` 判断启动是否成功并给出蜂鸣器提示。

```
static void start_apriltag_tracking(void)
```

```

{
    if (aprilTagTracker.isBusy()) {
        Serial.println("[AprilTagTrack] Busy, ignore KEY1 start");
        return;
    }

    aprilTagTracker.stop();
    delay(100);
    aprilTagTracker.start();

    if (aprilTagTracker.isBusy()) {
        arm.board.buzzer.set(100, 100, 2, 2000);
        Serial.println("[AprilTagTrack] KEY1 -> start");
    } else {
        arm.board.buzzer.set(300, 100, 1, 1000);
        Serial.println("[AprilTagTrack] start failed");
    }

    apply_oled_apriltag_screen();
}

```

7. `stop_apriltag_tracking()` 负责停止正在运行的 Apriltag 追踪。函数先判断 `aprilTagTracker.isBusy()`，只有追踪任务确实在运行时才停止状态机、触发蜂鸣器提示并刷新 OLED，避免空闲状态下重复执行停止逻辑。

```

static void stop_apriltag_tracking(void)
{
    if (!aprilTagTracker.isBusy()) {
        return;
    }

    aprilTagTracker.stop();
    arm.board.buzzer.set(80, 80, 1, 1800);
    apply_oled_apriltag_screen();
    Serial.println("[AprilTagTrack] KEY1 -> stop");
}

```

8. `buttons_ready()` 是开机后的按键保护函数。系统刚启动时会先读取并清掉两个按键的点击状态，在 `s_button_enable_time` 到达之前都不允许按键触发；时间到达后把 `s_buttons_armed` 置为真，后续按键才真正生效。

```

static bool buttons_ready(void)
{
    if (s_buttons_armed) {
        return true;
    }

    (void)arm.board.button.is_clicked(0);
    (void)arm.board.button.is_clicked(1);

    if (millis() < s_button_enable_time) {

```

```

        return false;
    }

    s_buttons_armed = true;
    Serial.println("[AprilTagTrack] Buttons armed");
    return false;
}

```

9. `save_config()` 和 `load_config()` 负责保存与 Apriltag 追踪工程相关的机械臂基础参数。这里保存通道、加速度和运动学参数；加载时如果 NVS 中存在 `arm_kin`，会先做一个简单校验，再覆盖默认运动学参数，避免异常参数影响标签追踪动作。

```

void save_config(void)
{
    prefs.begin("robot_cfg", false);
    prefs.putUChar("channel", global_channel);
    prefs.putUChar("acc", global_acc);
    prefs.putBytes("arm_kin", arm_kin, sizeof(arm_kin));
    prefs.end();
}

void load_config(void)
{
    prefs.begin("robot_cfg", true);
    global_channel = prefs.getUChar("channel", 2);
    global_acc = prefs.getUChar("acc", 245);

    if (prefs.isKey("arm_kin")) {
        float saved_kin[9];
        prefs.getBytes("arm_kin", saved_kin, sizeof(saved_kin));
        if (fabsf(saved_kin[0] - arm_kin[0]) < 0.01f) {
            memcpy(arm_kin, saved_kin, sizeof(arm_kin));
        }
    }
    prefs.end();
}

```

10. 周期定时器回调负责投递 Apriltag 追踪运行时需要的基础维护事件。它不会直接操作硬件，而是按周期投递蜂鸣器、电池、机械臂状态和 OLED 刷新事件；如果正在下载动作组，则直接返回，避免下载流程被周期任务干扰。

```

static void sys_timer_post_callback(TimerHandle_t xTimer)
{
    if (is_downloading) return;

    static uint32_t count = 0;
    if (count % BUZZER_UPDATE_PERIOD == 0) {
        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS, TIMING_EVENT_BUZZER_UPDATE,
        NULL, 0, 0);
    }
    if (count % BAT_UPDATE_PERIOD == 0) {
        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS, TIMING_EVENT_BAT_UPDATE,
        NULL, 0, 0);
    }
}

```

```

    }
    if (count % SERVO_STATUS_UPDATE_PERIOD == 0) {
        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS,
TIMING_EVENT_SERVO_STATUS_UPDATE, NULL, 0, 0);
    }
    if (count % OLED_REFRESH_PERIOD == 0) {
        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS, TIMING_EVENT_OLED_REFRESH,
NULL, 0, 0);
    }

    count += TIMER_PERIOD;
}

```

11. `sys_timer_sub_handler()` 接收周期事件并执行具体维护动作。蜂鸣器和电池是主板基础状态维护，`arm.update_status()` 用于持续刷新机械臂状态，`apply_oled_apriltag_screen()` 则让 OLED 根据 `AprilTagTracker` 当前状态显示空闲或正在追踪标签。

```

static void sys_timer_sub_handler(void* handler_args, esp_event_base_t base, int32_t id,
void* event_data)
{
    if (is_downloading) return;

    switch (id) {
        case TIMING_EVENT_BUZZER_UPDATE:
            arm.board.buzzer.update();
            break;
        case TIMING_EVENT_BAT_UPDATE:
            arm.board.bat.update();
            break;
        case TIMING_EVENT_SERVO_STATUS_UPDATE:
            arm.update_status();
            break;
        case TIMING_EVENT_OLED_REFRESH:
            apply_oled_apriltag_screen();
            break;
        default:
            break;
    }
}

```

12. `func_ctrl_callback()` 是上位机串口命令入口。和 Apriltag 追踪主流程直接相关的是动作组下载保护、机械臂运动学参数设置和 `CMD_APRILTAG_TRACK`；动作组下载时会把 `is_downloading` 置为真，让主循环暂停 Apriltag 追踪状态机和按键处理。

```

static void func_ctrl_callback(PacketTypeDef* self)
{
    uint8_t len;
    uint16_t bat_level;

    if (self->elements.id == 0xFF && self->elements.cmd == CMD_ACTION_GROUP_DOWNLOAD) {
        is_downloading = true;
    }
}

```

```

        arm.board.action_group_download(self->elements.args[0], self->elements.args, self->elements.length - 2);
        is_downloading = false;
        return;
    }

    if (self->elements.id != 0xFF) return;

    switch (self->elements.cmd) {

```

13. `CMD_SET_KINEMATICS_PARAM` 用于从上位机更新机械臂运动学参数。Apriltag 追踪需要根据标签在画面中的位置持续修正机械臂动作，所以参数更新后会写入本地 `arm_kin`、转发给舵机控制侧，并调用 `save_config()` 保存到 NVS。

```

    case CMD_SET_KINEMATICS_PARAM:
        if (self->elements.length - 2 >= (int)sizeof(arm_kin)) {
            memcpy(arm_kin, self->elements.args, sizeof(arm_kin));
            servo.tx_frame_write(0xFF, CMD_SET_KINEMATICS_PARAM, (uint8_t*)arm_kin,
sizeof(arm_kin));
            save_config();
            arm.board.buzzer.set(100, 50, 1, 3000);
        }
        break;

```

14. `CMD_APRILTAG_TRACK` 是上位机控制 Apriltag 追踪的主要命令。`sw == 0` 时停止当前追踪、关闭蜂鸣器并刷新 OLED；`sw != 0` 时只做准备提示和界面刷新，没有直接调用 `aprilTagTracker.start()`，说明这个工程里真正启动追踪仍由本机按键触发。

```

    case CMD_APRILTAG_TRACK:
        if (self->elements.length >= 3) {
            uint8_t sw = self->elements.args[0];
            if (sw == 0) {
                aprilTagTracker.stop();
                arm.board.buzzer.off();
                apply_oled_apriltag_screen();
                Serial.println("[AprilTagTrack] CMD stop");
            } else {
                arm.board.buzzer.set(30, 30, 1, 2500);
                apply_oled_apriltag_screen();
                Serial.println("[AprilTagTrack] CMD ready, press KEY1 to start");
            }
        }
        break;

```

15. `at32_packet_callback()` 处理 AT32 或舵机侧返回的数据。对 Apriltag 追踪工程来说，最直接相关的是舵机位置反馈：收到 `CMD_SERVO_STREAM` 或命令 96 后，代码解析 6 个舵机位置并更新时间戳，为机械臂状态同步和外部查询提供数据。

```

void at32_packet_callback(PacketTypeDef* rx_packet)
{

```

```

    if (is_downloading) return;

    if (rx_packet->elements.id == 0xFF &&
        rx_packet->elements.cmd == CMD_SERVO_STREAM &&
        rx_packet->elements.length >= 14) {
        for (int i = 0; i < 6; ++i) {
            int idx = i * 2;
            g_last_servo_positions[i] = (int16_t)((rx_packet->elements.args[idx + 1] << 8) |
rx_packet->elements.args[idx]);
        }
        g_last_servo_feedback_ms = millis();
        return;
    }

    if (rx_packet->elements.id == 0xFF &&
        rx_packet->elements.cmd == 96 &&
        rx_packet->elements.length >= 14) {
        for (int i = 0; i < 6; ++i) {
            int idx = i * 2;
            g_last_servo_positions[i] = (int16_t)((rx_packet->elements.args[idx + 1] << 8) |
rx_packet->elements.args[idx]);
        }
        g_last_servo_feedback_ms = millis();
        return;
    }
}

```

16. 当 AT32 返回当前机械臂坐标时，代码会解析 X、Y、Z、pitch、roll、claw 和 6 个舵机位置，并写入 `arm.current_pose` 与舵机缓存。Apriltag 追踪要根据标签中心持续修正机械臂位置，这些反馈数据可以帮助系统维持机械臂状态同步。

```

    if (rx_packet->elements.cmd == CMD_GET_CUR_COORDS && rx_packet->elements.length
>= 26) {
        int16_t raw_x = (int16_t)((rx_packet->elements.args[1] << 8) | rx_packet-
>elements.args[0]);
        int16_t raw_y = (int16_t)((rx_packet->elements.args[3] << 8) | rx_packet-
>elements.args[2]);
        int16_t raw_z = (int16_t)((rx_packet->elements.args[5] << 8) | rx_packet-
>elements.args[4]);
        int16_t raw_p = (int16_t)((rx_packet->elements.args[7] << 8) | rx_packet-
>elements.args[6]);
        int16_t raw_r = (int16_t)((rx_packet->elements.args[9] << 8) | rx_packet-
>elements.args[8]);
        int16_t raw_c = (int16_t)((rx_packet->elements.args[11] << 8) | rx_packet-
>elements.args[10]);

        arm.current_pose.x = (float)raw_x;
        arm.current_pose.y = (float)raw_y;
        arm.current_pose.z = (float)raw_z;
        arm.current_pose.pitch = (float)raw_p / 10.0f;
        arm.current_pose.roll = (float)raw_r;
        arm.current_pose.claw = (float)raw_c;
    }
}

```

```

        for (int i = 0; i < 6; ++i) {
            int idx = 12 + i * 2;
            g_last_servo_positions[i] = (int16_t)((rx_packet->elements.args[idx + 1]
<< 8) | rx_packet->elements.args[idx]);
        }
        g_last_servo_feedback_ms = millis();
    }
}

```

17. `pump_at32_feedback()`、`sync_arm_feedback()` 和 `get_last_servo_positions()` 是机械臂反馈辅助函数。它们分别负责从 AT32 串口抽取原始数据、等待新的机械臂反馈、向其他模块提供最近一次舵机位置缓存，保证追踪运行时不会只依赖过期状态。

```

void pump_at32_feedback(void)
{
    while (servo.uart->available()) {
        uint8_t c = servo.uart->read();
        at32_protocol.parsing(&c, 1);
    }
}

bool sync_arm_feedback(uint32_t timeout_ms)
{
    arm.update_status();
    uint32_t start = millis();
    uint32_t prev_feedback_ms = g_last_servo_feedback_ms;

    while (millis() - start < timeout_ms) {
        serial_port.rec_handler();
        pump_at32_feedback();
        if (g_last_servo_feedback_ms != 0 && g_last_servo_feedback_ms != prev_feedback_ms) {
            return true;
        }
        delay(2);
    }

    return g_last_servo_feedback_ms != 0;
}

bool get_last_servo_positions(int16_t out_pos[6])
{
    if (g_last_servo_feedback_ms == 0) {
        return false;
    }
    memcpy(out_pos, g_last_servo_positions, sizeof(g_last_servo_positions));
    return true;
}

```

18. `system_loop_handler()` 是 Apriltag 追踪运行阶段的主循环。非下载状态下，它依次处理上位机串口、AT32 反馈和 `apriltagTracker.update()`；随后检测追踪任务是否刚从运行状态回到空闲状态，如果已经停止，就刷新 OLED 回到待机界面。

```

void system_loop_handler(void)
{
    if (!is_downloading) {
        serial_port.rec_handler();
        pump_at32_feedback();

        aprilTagTracker.update();

        bool busy_now = aprilTagTracker.isBusy();
        if (s_was_tracker_busy && !busy_now) {
            Serial.println("[AprilTagTrack] Tracking stopped, idle");
            apply_oled_apriltag_screen();
        }
        s_was_tracker_busy = busy_now;

        arm.board.button.update();
        arm.board.buzzer.update();
    }
}

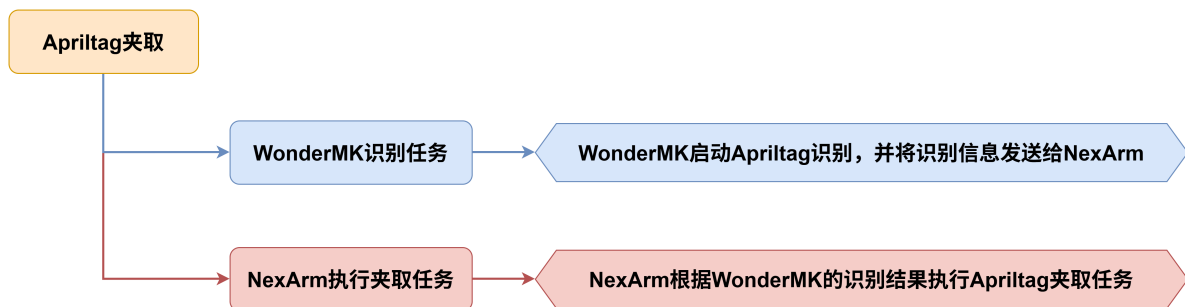
```

## 4.9 Apriltag夹取

### 4.9.1 项目介绍

该实验实现了基于 AprilTag 标签识别的机械臂自动夹取与放置功能。系统能够根据视觉识别结果完成目标定位、抓取偏移补偿、夹取搬运和回位复位。

### 4.9.2 实现流程



### 4.9.3 程序下载

找到本文档同目录下的“02 程序源码”文件夹，找到“07 Apriltag夹取”程序进行下载，具体下载步骤可参考[4.2 程序下载说明\(必看\)](#)。

### 4.9.4 运行效果

设备上电后会先完成初始化，机械臂自动回到初始等待位，随后进入待机状态。启动后，系统会持续识别画面中的标签目标，并驱动机械臂不断微调位置，使目标逐步进入适合抓取的区域；当目标位置稳定后，机械臂会自动下降，张开夹爪完成对位，随后闭合夹爪夹住目标，再抬升并移动到预放置区域，最后松开目标并回到初始姿态，等待下一次启动。整个过程表现为设备能够完成从识别、定位到抓取、搬运、释放的一整套自动化动作。

实际使用时，先正常上电并等待初始化结束，确认机械臂已经回到初始位；将带有标签的目标放到摄像头可识别范围内；按下 **KEY2** 按键后，系统开始自动识别并执行夹取流程。

# AprilTag Gripping

KEY2: Start

Grip Ready

Idle

1816

## 4.9.5 程序分析

### system\_task\_handle.cpp

1. 文件开头先引入 Apriltag 夹取工程需要的核心模块，包括系统任务接口、全局命令、机械臂控制、USB 串口控制、参数存储、Apriltag 夹取状态机、AT32 OTA 和文件系统等。这里最关键的是 `AprilTagGrabber.h`，它负责和 K230 视觉侧通信，并根据识别到的 Apriltag 位置执行搜索、对准、下探、夹取、抬升和放置流程；`Robot_Arm.h` 负责机械臂和主板外设；`usb_ctrl.h` 负责上位机串口命令入口。

```
#include "system_task_handle.h"
#include "Global.h"
#include "Robot_Arm.h"
#include "usb_ctrl.h"
#include <Preferences.h>
#include "AprilTagGrabber.h"
#include "AT32_OTA.h"
#include <LittleFS.h>
#include <math.h>
#include <string.h>

#define CMD_SERVO_STREAM 0x68
```

2. 这些静态对象和全局变量保存 Apriltag 夹取运行期间需要共享的系统状态。`loop_with_sys_task` 用于注册周期事件，`at32_protocol` 用来解析 AT32 侧机械臂反馈，`prefs` 用来保存机械臂参数和夹取偏移，`is_downloading` 用来在动作组下载时暂停常规任务，`g_last_servo_positions` 和 `g_last_servo_feedback_ms` 则缓存最近一次舵机反馈。

```
static const char* TAG = "apriltag_gripping";
static TimerHandle_t TIMER;
static esp_event_loop_handle_t loop_with_sys_task;
static CommProtocol_t at32_protocol;
```

```
static Preferences prefs;
static bool is_downloading = false;

static int16_t g_last_servo_positions[6] = {2048, 2048, 2048, 2048, 2048, 2048};
static uint32_t g_last_servo_feedback_ms = 0;

uint8_t global_channel = 2;
uint8_t global_acc = 245;
volatile bool servo_uart_busy = false;

static float arm_kin[9] = {
    110.45f, 225.00f, 36.97f, 145.00f, 0.0f, 130.23f, 0.0f, 50.0f, 70.5f
};
```

3. 这里定义了 Apriltag 夹取工程的运行状态标记。s\_was\_tracker\_busy 用来判断夹取任务是否刚从运行状态回到空闲状态，s\_button\_enable\_time 和 s\_buttons\_armed 用于开机后的按键延迟使能，避免上电过程中误触发夹取任务启动或停止。

```
static bool s_was_tracker_busy = false;
static uint32_t s_button_enable_time = 0;
static bool s_buttons_armed = false;

ESP_EVENT_DEFINE_BASE(SYS_TIMING_EVENTS);
```

4. apply\_oled\_apriltag\_screen() 负责刷新 Apriltag 夹取工程的 OLED 界面。aprilTagGrabber.isBusy() 为真时显示正在夹取标签和搜索状态，空闲时显示启动入口、Grip Ready 和 Idle，让用户能从屏幕判断当前是否可以启动夹取流程。

```
static void apply_oled_apriltag_screen(void)
{
    arm.board.oled.set_custom_text(0, "AprilTag Gripping");

    if (aprilTagGrabber.isBusy()) {
        arm.board.oled.set_custom_text(1, "Gripping Tag");
        arm.board.oled.set_custom_text(2, "Status: Search");
        arm.board.oled.set_custom_text(3, "KEY1: Stop");
    } else {
        arm.board.oled.set_custom_text(1, "KEY1: Start");
        arm.board.oled.set_custom_text(2, "Grip Ready");
        arm.board.oled.set_custom_text(3, "Idle");
    }

    arm.board.oled.show_custom();
}
```

5. perform\_arm\_poweron\_reset() 是 Apriltag 夹取启动前的机械臂上电复位流程。它先在 OLED 显示回零提示，再调用 arm.reset\_all(2500) 让机械臂回到初始姿态，并在等待过程中持续抽取 AT32 反馈，保证进入夹取运行前机械臂状态已经可用。

```
static void perform_arm_poweron_reset(void)
```

```

{
    arm.board.oled.set_custom_text(0, "AprilTag Gripping");
    arm.board.oled.set_custom_text(1, "Arm homing...");
    arm.board.oled.set_custom_text(2, "Please wait");
    arm.board.oled.set_custom_text(3, "");
    arm.board.oled.show_custom();

    arm.reset_all(2500);
    uint32_t t0 = millis();
    while (millis() - t0 < 2600) {
        pump_at32_feedback();
        delay(20);
    }
    for (int i = 0; i < 8; i++) {
        arm.update_status();
        delay(50);
        while (servo.uart->available()) {
            uint8_t c = servo.uart->read();
            at32_protocol.parsing(&c, 1);
        }
    }
    Serial.println("[AprilTagGrip] Power-on arm reset done");
}

```

6. `start_apriltag_gripping()` 是本地启动 Apriltag 夹取的函数。它只在 `AprilTagGrabber` 空闲时执行，启动前先调用 `aprilTagGrabber.stop()` 清理旧状态，再调用 `aprilTagGrabber.start()` 进入夹取状态机，随后根据 `aprilTagGrabber.isBusy()` 判断启动是否成功并给出蜂鸣器提示。

```

static void start_apriltag_gripping(void)
{
    if (aprilTagGrabber.isBusy()) {
        Serial.println("[AprilTagGrip] Busy, ignore KEY1 start");
        return;
    }

    aprilTagGrabber.stop();
    delay(100);
    aprilTagGrabber.start();

    if (aprilTagGrabber.isBusy()) {
        arm.board.buzzer.set(100, 100, 2, 2000);
        Serial.println("[AprilTagGrip] KEY1 -> start");
    } else {
        arm.board.buzzer.set(300, 100, 1, 1000);
        Serial.println("[AprilTagGrip] start failed");
    }

    apply_oled_apriltag_screen();
}

```

7. `stop_apriltag_gripping()` 负责停止正在运行的 Apriltag 夹取任务。函数先判断 `aprilTagGrabber.isBusy()`，只有夹取任务确实在运行时才停止状态机、触发蜂鸣器提示并刷新 OLED，避免空闲状态下重复执行停止逻辑。

```
static void stop_apriltag_gripping(void)
{
    if (!aprilTagGrabber.isBusy()) {
        return;
    }

    aprilTagGrabber.stop();
    arm.board.buzzer.set(80, 80, 1, 1800);
    apply_oled_apriltag_screen();
    Serial.println("[AprilTagGrip] KEY1 -> stop");
}
```

8. `buttons_ready()` 是开机后的按键保护函数。系统刚启动时会先读取并清掉两个按键的点击状态，在 `s_button_enable_time` 到达之前都不允许按键触发；时间到达后把 `s_buttons_armed` 置为真，后续按键才真正生效。

```
static bool buttons_ready(void)
{
    if (s_buttons_armed) {
        return true;
    }

    (void)arm.board.button.is_clicked(0);
    (void)arm.board.button.is_clicked(1);

    if (millis() < s_button_enable_time) {
        return false;
    }

    s_buttons_armed = true;
    Serial.println("[AprilTagGrip] Buttons armed");
    return false;
}
```

9. `save_config()` 保存基础运行参数，包括无线通道、全局加速度和机械臂运动学参数。Apriltag 夹取依赖机械臂运动精度，因此运动学参数会随上位机标定结果一起持久化。

```
void save_config(void)
{
    prefs.begin("robot_cfg", false);
    prefs.putUChar("channel", global_channel);
    prefs.putUChar("acc", global_acc);
    prefs.putBytes("arm_kin", arm_kin, sizeof(arm_kin));
    prefs.end();
}
```

10. `load_config()` 加载机械臂基础参数，并额外恢复 Apriltag 夹取偏移。`ag_x`、`ag_y`、`ag_z` 会传给 `apriltagGrabber.setOffsets()`，这些偏移会影响标签识别后实际下探和夹取的位置。

```
void load_config(void)
{
    prefs.begin("robot_cfg", true);
    global_channel = prefs.getUChar("channel", 2);
    global_acc = prefs.getUChar("acc", 245);

    if (prefs.isKey("arm_kin")) {
        float saved_kin[9];
        prefs.getBytes("arm_kin", saved_kin, sizeof(saved_kin));
        if (fabsf(saved_kin[0] - arm_kin[0]) < 0.01f) {
            memcpy(arm_kin, saved_kin, sizeof(arm_kin));
        }
    }

    if (prefs.isKey("ag_x") && prefs.isKey("ag_y") && prefs.isKey("ag_z")) {
        float ag_x = prefs.getFloat("ag_x", 15.0f);
        float ag_y = prefs.getFloat("ag_y", 50.0f);
        float ag_z = prefs.getFloat("ag_z", 60.0f);
        apriltagGrabber.setOffsets(ag_x, ag_y, ag_z);
    }
    prefs.end();
}
```

11. 周期定时器回调负责投递 Apriltag 夹取运行时需要的基础维护事件。它不会直接操作硬件，而是按周期投递蜂鸣器、电池、机械臂状态和 OLED 刷新事件；如果正在下载动作组，则直接返回，避免下载流程被周期任务干扰。

```
static void sys_timer_post_callback(TimerHandle_t xTimer)
{
    if (is_downloading) return;

    static uint32_t count = 0;
    if (count % BUZZER_UPDATE_PERIOD == 0) {
        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS, TIMING_EVENT_BUZZER_UPDATE,
        NULL, 0, 0);
    }
    if (count % BAT_UPDATE_PERIOD == 0) {
        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS, TIMING_EVENT_BAT_UPDATE,
        NULL, 0, 0);
    }
    if (count % SERVO_STATUS_UPDATE_PERIOD == 0) {
        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS,
        TIMING_EVENT_SERVO_STATUS_UPDATE, NULL, 0, 0);
    }
    if (count % OLED_REFRESH_PERIOD == 0) {
        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS, TIMING_EVENT_OLED_REFRESH,
        NULL, 0, 0);
    }

    count += TIMER_PERIOD;
}
```

```
}
```

12. `sys_timer_sub_handler()` 接收周期事件并执行具体维护动作。蜂鸣器和电池是主板基础状态维护，`arm.update_status()` 用于持续刷新机械臂状态，`apply_oled_apriltag_screen()` 则让 OLED 根据 `AprilTagGrabber` 当前状态显示空闲或正在夹取标签。

```
static void sys_timer_sub_handler(void* handler_args, esp_event_base_t base, int32_t id,
void* event_data)
{
    if (is_downloading) return;

    switch (id) {
        case TIMING_EVENT_BUZZER_UPDATE:
            arm.board.buzzer.update();
            break;
        case TIMING_EVENT_BAT_UPDATE:
            arm.board.bat.update();
            break;
        case TIMING_EVENT_SERVO_STATUS_UPDATE:
            arm.update_status();
            break;
        case TIMING_EVENT_OLED_REFRESH:
            apply_oled_apriltag_screen();
            break;
        default:
            break;
    }
}
```

13. `func_ctrl_callback()` 是上位机串口命令入口。和 Apriltag 夹取主流程直接相关的是动作组下载保护、机械臂运动学参数设置、`CMD_APRILTAG_GRAB` 启停控制以及 `CMD_APRILTAG_SET_OFFSET` 夹取偏移标定；动作组下载时会把 `is_downloading` 置为真，让主循环暂停夹取状态机和按键处理。

```
static void func_ctrl_callback(PacketTypeDef* self)
{
    uint8_t len;
    uint16_t bat_level;

    if (self->elements.id == 0xFF && self->elements.cmd == CMD_ACTION_GROUP_DOWNLOAD) {
        is_downloading = true;
        arm.board.action_group_download(self->elements.args[0], self->elements.args, self-
>elements.length - 2);
        is_downloading = false;
        return;
    }

    if (self->elements.id != 0xFF) return;

    switch (self->elements.cmd) {
```

14. `CMD_SET_KINEMATICS_PARAM` 用于从上位机更新机械臂运动学参数。Apriltag 夹取需要机械臂准确到达搜索、下探、夹取、抬升和放置位置，所以参数更新后会写入本地 `arm_kin`、转发给舵机控制侧，并调用 `save_config()` 保存到 NVS。

```
case CMD_SET_KINEMATICS_PARAM:
    if (self->elements.length - 2 >= (int)sizeof(arm_kin)) {
        memcpy(arm_kin, self->elements.args, sizeof(arm_kin));
        servo.tx_frame_write(0xFF, CMD_SET_KINEMATICS_PARAM, (uint8_t*)arm_kin,
sizeof(arm_kin));
        save_config();
        arm.board.buzzer.set(100, 50, 1, 3000);
    }
    break;
```

15. `CMD_APRILTAG_GRAB` 是上位机控制 Apriltag 夹取的主要命令。`sw == 0` 时停止当前夹取、关闭蜂鸣器并刷新 OLED；`sw != 0` 时只做准备提示和界面刷新，没有直接调用 `aprilTagGrabber.start()`，说明这个工程里真正启动夹取仍由本机按键触发。

```
case CMD_APRILTAG_GRAB:
    if (self->elements.length >= 3) {
        uint8_t sw = self->elements.args[0];
        if (sw == 0) {
            aprilTagGrabber.stop();
            arm.board.buzzer.off();
            apply_oled_apriltag_screen();
            Serial.println("[AprilTagGrip] CMD stop");
        } else {
            arm.board.buzzer.set(30, 30, 1, 2500);
            apply_oled_apriltag_screen();
            Serial.println("[AprilTagGrip] CMD ready, press KEY1 to start");
        }
    }
    break;
```

16. `CMD_APRILTAG_SET_OFFSET` 是 Apriltag 夹取标定入口。上位机传入 X、Y 和抓取 Z 位置后，代码调用 `aprilTagGrabber.setOffsets()` 更新夹取补偿，并把 `ag_x`、`ag_y`、`ag_z` 写入 `Preferences`，保证下次开机仍然使用标定结果。

```
case CMD_APRILTAG_SET_OFFSET:
    if (self->elements.length >= 14) {
        float x_off, y_off, z_grab;
        memcpy(&x_off, &self->elements.args[0], 4);
        memcpy(&y_off, &self->elements.args[4], 4);
        memcpy(&z_grab, &self->elements.args[8], 4);
        aprilTagGrabber.setOffsets(x_off, y_off, z_grab);

        prefs.begin("robot_cfg", false);
        prefs.putFloat("ag_x", x_off);
        prefs.putFloat("ag_y", y_off);
        prefs.putFloat("ag_z", z_grab);
        prefs.end();
    }
```

```

        arm.board.buzzer.set(100, 50, 2, 3000);
        Serial.printf("[AprilTagGrip] offsets x=%.1f y=%.1f z=%.1f\n", x_off, y_off,
z_grab);
    }
    break;

```

17. `at32_packet_callback()` 处理 AT32 或舵机侧返回的数据。对 Apriltag 夹取工程来说，最直接相关的是舵机位置反馈：收到 `CMD_SERVO_STREAM` 或命令 96 后，代码解析 6 个舵机位置并更新时间戳，为夹取过程中的机械臂状态同步提供数据。

```

void at32_packet_callback(PacketTypeDef* rx_packet)
{
    if (is_downloading) return;

    if (rx_packet->elements.id == 0xFF &&
        rx_packet->elements.cmd == CMD_SERVO_STREAM &&
        rx_packet->elements.length >= 14) {
        for (int i = 0; i < 6; ++i) {
            int idx = i * 2;
            g_last_servo_positions[i] = (int16_t)((rx_packet->elements.args[idx + 1] << 8) |
rx_packet->elements.args[idx]);
        }
        g_last_servo_feedback_ms = millis();
        return;
    }

    if (rx_packet->elements.id == 0xFF &&
        rx_packet->elements.cmd == 96 &&
        rx_packet->elements.length >= 14) {
        for (int i = 0; i < 6; ++i) {
            int idx = i * 2;
            g_last_servo_positions[i] = (int16_t)((rx_packet->elements.args[idx + 1] << 8) |
rx_packet->elements.args[idx]);
        }
        g_last_servo_feedback_ms = millis();
        return;
    }
}

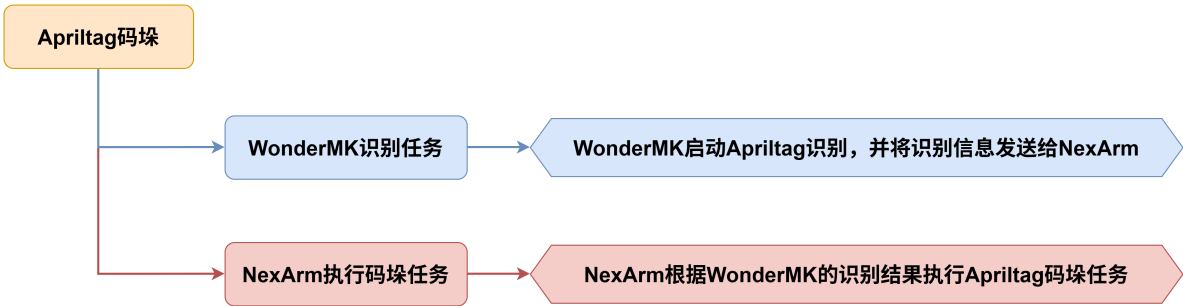
```

## 4.10 Apriltag码垛

### 4.10.1 项目介绍

本实验实现了基于 AprilTag 标签识别的机械臂自动码垛功能。系统能够完成目标定位、自动夹取、按层高度依次放置以及回位复位，形成连续的堆叠演示流程。

### 4.10.2 实现流程



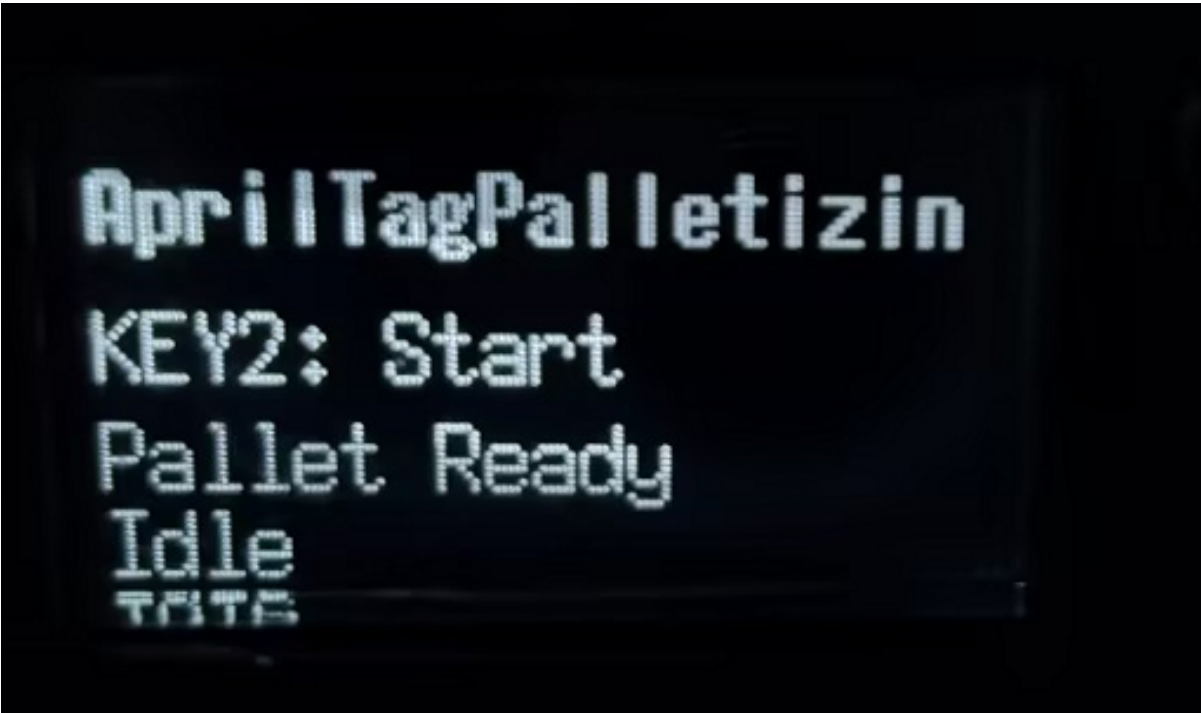
### 4.10.3 程序下载

找到本文档同目录下的“02 程序源码”文件夹，找到“08 Apriltag码垛”程序进行下载，具体下载步骤可参考[4.2 程序下载说明\(必看\)](#)。

### 4.10.4 运行效果

设备上电后会先完成初始化，机械臂自动回到初始等待位，屏幕上会显示当前运行状态和当前码垛层数，随后进入待机状态。启动后，系统会持续识别画面中的标签目标，并驱动机械臂不断微调位置，使目标逐步进入适合抓取的区域；目标位置稳定后，机械臂会自动下降、闭合夹爪完成抓取，再抬升并移动到固定码垛区域进行放置。放置高度会随层数递增，第一层完成后，第二层会在上一层基础上继续抬高放置，第三层再进一步上移，从而形成明显的逐层堆叠效果；每次放置完成后机械臂会回到搜索姿态，等待继续识别下一次目标。

实际使用时，先正常上电并等待初始化结束，确认机械臂已经回到初始位；将带有标签的目标放到摄像头可识别范围内；按下 **KEY2** 启动按键后，系统开始自动识别并执行夹取码垛流程；如果需要中途停止，可以再按一次启动按键结束当前演示。



## 4.10.5 程序分析

### system\_task\_handle.cpp

1. 文件开头先引入 Apriltag 码垛工程需要的核心模块，包括系统任务接口、全局命令、机械臂控制、USB 串口控制、参数存储、Apriltag 码垛状态机、AT32 OTA 和文件系统等。这里最关键的是 `AprilTagPalletizer.h`，它负责和 K230 视觉侧通信，并根据识别到的 Apriltag 位置执行搜索、对准、夹取、抬升、放置和码垛槽位推进流程；`Robot_Arm.h` 负责机械臂和主板外设；`usb_ctrl.h` 负责上位机串口命令入口。

```
#include "system_task_handle.h"
#include "Global.h"
#include "Robot_Arm.h"
#include "usb_ctrl.h"
#include <Preferences.h>
#include "AprilTagPalletizer.h"
#include "AT32_OTA.h"
#include <LittleFS.h>
#include <math.h>
#include <string.h>
```

```
#define CMD_SERVO_STREAM 0x68
```

2. 这些静态对象和全局变量保存 Apriltag 码垛运行期间需要共享的系统状态。`loop_with_sys_task` 用于注册周期事件，`at32_protocol` 用来解析 AT32 侧机械臂反馈，`prefs` 用来保存机械臂参数和夹取偏移，`is_downloading` 用来在动作组下载时暂停常规任务，`g_last_servo_positions` 和 `g_last_servo_feedback_ms` 则缓存最近一次舵机反馈。

```
static const char* TAG = "apriltag_palletizing";
static TimerHandle_t TIMER;
static esp_event_loop_handle_t loop_with_sys_task;
static CommProtocol_t at32_protocol;
static Preferences prefs;
static bool is_downloading = false;

static int16_t g_last_servo_positions[6] = {2048, 2048, 2048, 2048, 2048, 2048};
static uint32_t g_last_servo_feedback_ms = 0;

uint8_t global_channel = 2;
uint8_t global_acc = 245;
volatile bool servo_uart_busy = false;

static float arm_kin[9] = {
    110.45f, 225.00f, 36.97f, 145.00f, 0.0f, 130.23f, 0.0f, 50.0f, 70.5f
};
```

3. 这里定义了 Apriltag 码垛工程的运行状态标记。`s_was_tracker_busy` 用来判断码垛任务是否刚从运行状态回到空闲状态，`s_button_enable_time` 和 `s_buttons_armed` 用于开机后的按键延迟使能，避免上电过程中误触发码垛任务启动或停止。

```
static bool s_was_tracker_busy = false;
static uint32_t s_button_enable_time = 0;
static bool s_buttons_armed = false;

ESP_EVENT_DEFINE_BASE(SYS_TIMING_EVENTS);
```

4. `apply_oled_apriltag_screen()` 负责刷新 Apriltag 码垛工程的 OLED 界面。

`aprilTagPalletizer.isBusy()` 为真时显示正在码垛，并通过 `getPlacedCount()` 和 `getSlotCount()` 显示当前槽位进度；空闲时显示启动入口、Pallet Ready 和 Idle。

```
static void apply_oled_apriltag_screen(void)
{
    arm.board.oled.set_custom_text(0, "AprilTagPalletizin");

    if (aprilTagPalletizer.isBusy()) {
        uint8_t slot = aprilTagPalletizer.getPlacedCount() + 1;
        uint8_t total = aprilTagPalletizer.getSlotCount();
        if (slot > total) {
            slot = total;
        }
        arm.board.oled.set_custom_text(1, "Palletizing");
        arm.board.oled.set_custom_text(2, String("Slot: ") + String(slot) + "/" +
String(total));
        arm.board.oled.set_custom_text(3, "KEY2: Stop");
    } else {
        arm.board.oled.set_custom_text(1, "KEY2: Start");
        arm.board.oled.set_custom_text(2, "Pallet Ready");
        arm.board.oled.set_custom_text(3, "Idle");
    }

    arm.board.oled.show_custom();
}
```

5. `perform_arm_poweron_reset()` 是 Apriltag 码垛启动前的机械臂上电复位流程。它先在 OLED 显示回零提示，再调用 `arm.reset_all(2500)` 让机械臂回到初始姿态，并在等待过程中持续抽取 AT32 反馈，保证进入码垛运行前机械臂状态已经可用。

```
static void perform_arm_poweron_reset(void)
{
    arm.board.oled.set_custom_text(0, "AprilTagPalletizin");
    arm.board.oled.set_custom_text(1, "Arm homing...");
    arm.board.oled.set_custom_text(2, "Please wait");
    arm.board.oled.set_custom_text(3, "");
    arm.board.oled.show_custom();

    arm.reset_all(2500);
    uint32_t t0 = millis();
    while (millis() - t0 < 2600) {
        pump_at32_feedback();
        delay(20);
    }
}
```

```

    for (int i = 0; i < 8; i++) {
        arm.update_status();
        delay(50);
        while (servo.uart->available()) {
            uint8_t c = servo.uart->read();
            at32_protocol.parsing(&c, 1);
        }
    }
    Serial.println("[AprilTagPallet] Power-on arm reset done");
}

```

6. `reset_arm_after_stop()` 是码垛任务被手动停止后的机械臂复位流程。它先显示停止和复位提示，再调用 `arm.reset_all(2000)`，等待期间持续处理 AT32 反馈和蜂鸣器刷新，最后补充更新机械臂状态，确保中途停止后机械臂回到安全姿态。

```

static void reset_arm_after_stop(void)
{
    arm.board.oled.set_custom_text(0, "AprilTagPalletizin");
    arm.board.oled.set_custom_text(1, "Stopping...");
    arm.board.oled.set_custom_text(2, "Arm reset");
    arm.board.oled.set_custom_text(3, "");
    arm.board.oled.show_custom();

    arm.reset_all(2000);
    uint32_t t0 = millis();
    while (millis() - t0 < 2100) {
        pump_at32_feedback();
        arm.board.buzzer.update();
        delay(20);
    }

    for (int i = 0; i < 6; i++) {
        arm.update_status();
        delay(40);
        while (servo.uart->available()) {
            uint8_t c = servo.uart->read();
            at32_protocol.parsing(&c, 1);
        }
    }

    Serial.println("[AprilTagPallet] Arm reset after KEY1 stop");
}

```

7. `start_apriltag_palletizing()` 是本地启动 Apriltag 码垛的函数。它只在 `AprilTagPalletizer` 空闲时执行，启动前先调用 `aprilTagPalletizer.stop()` 清理旧状态，再调用 `aprilTagPalletizer.start()` 进入码垛状态机，随后根据 `aprilTagPalletizer.isBusy()` 判断启动是否成功并给出蜂鸣器提示。

```

static void start_apriltag_palletizing(void)
{
    if (aprilTagPalletizer.isBusy()) {
        Serial.println("[AprilTagPallet] Busy, ignore KEY1 start");
    }
}

```

```

        return;
    }

    aprilTagPalletizer.stop();
    delay(100);
    aprilTagPalletizer.start();

    if (aprilTagPalletizer.isBusy()) {
        arm.board.buzzer.set(100, 100, 2, 2000);
        Serial.println("[AprilTagPallet] KEY1 -> start");
    } else {
        arm.board.buzzer.set(300, 100, 1, 1000);
        Serial.println("[AprilTagPallet] start failed");
    }

    apply_oled_apriltag_screen();
}

```

8. `stop_apriltag_palletizing()` 负责停止正在运行的 Apriltag 码垛任务。函数先判断 `aprilTagPalletizer.isBusy()`，只有任务确实在运行时才停止状态机；如果 `reset_arm` 为真，还会调用 `reset_arm_after_stop()` 让机械臂复位，这比普通夹取工程多了一层停止后的安全处理。

```

static void stop_apriltag_palletizing(bool reset_arm = false)
{
    if (!aprilTagPalletizer.isBusy()) {
        return;
    }

    aprilTagPalletizer.stop();
    arm.board.buzzer.set(80, 80, 1, 1800);
    if (reset_arm) {
        reset_arm_after_stop();
    }
    apply_oled_apriltag_screen();
    Serial.println(reset_arm ? "[AprilTagPallet] KEY1 -> stop reset" : "[AprilTagPallet]
stop");
}

```

9. `buttons_ready()` 是开机后的按键保护函数。系统刚启动时会先读取并清掉两个按键的点击状态，在 `s_button_enable_time` 到达之前都不允许按键触发；时间到达后把 `s_buttons_armed` 置为真，后续按键才真正生效。

```

static bool buttons_ready(void)
{
    if (s_buttons_armed) {
        return true;
    }

    (void)arm.board.button.is_clicked(0);
    (void)arm.board.button.is_clicked(1);
}

```

```

    if (millis() < s_button_enable_time) {
        return false;
    }

    s_buttons_armed = true;
    Serial.println("[AprilTagPallet] Buttons armed");
    return false;
}

```

10. `save_config()` 保存基础运行参数，包括无线通道、全局加速度和机械臂运动学参数。Apriltag 码垛依赖机械臂运动精度，因此运动学参数会随上位机标定结果一起持久化。

```

void save_config(void)
{
    prefs.begin("robot_cfg", false);
    prefs.putUChar("channel", global_channel);
    prefs.putUChar("acc", global_acc);
    prefs.putBytes("arm_kin", arm_kin, sizeof(arm_kin));
    prefs.end();
}

```

11. `load_config()` 加载机械臂基础参数，并额外恢复 Apriltag 码垛的抓取偏移。`ag_x`、`ag_y`、`ag_z` 会传给 `aprilTagPalletizer.setOffsets()`，这些偏移会影响标签识别后实际下探、夹取和码垛放置前的目标补偿。

```

void load_config(void)
{
    prefs.begin("robot_cfg", true);
    global_channel = prefs.getUChar("channel", 2);
    global_acc = prefs.getUChar("acc", 245);

    if (prefs.isKey("arm_kin")) {
        float saved_kin[9];
        prefs.getBytes("arm_kin", saved_kin, sizeof(saved_kin));
        if (fabsf(saved_kin[0] - arm_kin[0]) < 0.01f) {
            memcpy(arm_kin, saved_kin, sizeof(arm_kin));
        }
    }

    if (prefs.isKey("ag_x") && prefs.isKey("ag_y") && prefs.isKey("ag_z")) {
        float ag_x = prefs.getFloat("ag_x", 15.0f);
        float ag_y = prefs.getFloat("ag_y", 50.0f);
        float ag_z = prefs.getFloat("ag_z", 60.0f);
        aprilTagPalletizer.setOffsets(ag_x, ag_y, ag_z);
    }
    prefs.end();
}

```

12. 周期定时器回调负责投递 Apriltag 码垛运行时需要的基础维护事件。它不会直接操作硬件，而是按周期投递蜂鸣器、电池、机械臂状态和 OLED 刷新事件；如果正在下载动作组，则直接返回，避免下载流程被周期任务干扰。

```

static void sys_timer_post_callback(TimerHandle_t xTimer)

```

```

{
    if (is_downloading) return;

    static uint32_t count = 0;
    if (count % BUZZER_UPDATE_PERIOD == 0) {
        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS, TIMING_EVENT_BUZZER_UPDATE,
        NULL, 0, 0);
    }
    if (count % BAT_UPDATE_PERIOD == 0) {
        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS, TIMING_EVENT_BAT_UPDATE,
        NULL, 0, 0);
    }
    if (count % SERVO_STATUS_UPDATE_PERIOD == 0) {
        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS,
        TIMING_EVENT_SERVO_STATUS_UPDATE, NULL, 0, 0);
    }
    if (count % OLED_REFRESH_PERIOD == 0) {
        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS, TIMING_EVENT_OLED_REFRESH,
        NULL, 0, 0);
    }

    count += TIMER_PERIOD;
}

```

13. `sys_timer_sub_handler()` 接收周期事件并执行具体维护动作。蜂鸣器和电池是主板基础状态维护，`arm.update_status()` 用于持续刷新机械臂状态，`apply_oled_apriltag_screen()` 则让 OLED 根据 `AprilTagPalletizer` 当前状态显示空闲或正在码垛的槽位进度。

```

static void sys_timer_sub_handler(void* handler_args, esp_event_base_t base, int32_t id,
void* event_data)
{
    if (is_downloading) return;

    switch (id) {
        case TIMING_EVENT_BUZZER_UPDATE:
            arm.board.buzzer.update();
            break;
        case TIMING_EVENT_BAT_UPDATE:
            arm.board.bat.update();
            break;
        case TIMING_EVENT_SERVO_STATUS_UPDATE:
            arm.update_status();
            break;
        case TIMING_EVENT_OLED_REFRESH:
            apply_oled_apriltag_screen();
            break;
        default:
            break;
    }
}

```

14. `func_ctrl_callback()` 是上位机串口命令入口。和 Apriltag 码垛主流程直接相关的是动作组下载保护、机械臂运动学参数设置、`CMD_APRILTAG_GRAB` 启停控制以及 `CMD_APRILTAG_SET_OFFSET` 抓取偏移标定；动作组下载时会把 `is_downloading` 置为真，让主循环暂停码垛状态机和按键处理。

```
static void func_ctrl_callback(PacketTypeDef* self)
{
    uint8_t len;
    uint16_t bat_level;

    if (self->elements.id == 0xFF && self->elements.cmd == CMD_ACTION_GROUP_DOWNLOAD) {
        is_downloading = true;
        arm.board.action_group_download(self->elements.args[0], self->elements.args, self->elements.length - 2);
        is_downloading = false;
        return;
    }

    if (self->elements.id != 0xFF) return;

    switch (self->elements.cmd) {
```

15. `CMD_SET_KINEMATICS_PARAM` 用于从上位机更新机械臂运动学参数。Apriltag 码垛需要机械臂准确到达搜索、夹取、抬升和多个码垛槽位位置，所以参数更新后会写入本地 `arm_kin`、转发给舵机控制侧，并调用 `save_config()` 保存到 NVS。

```
case CMD_SET_KINEMATICS_PARAM:
    if (self->elements.length - 2 >= (int)sizeof(arm_kin)) {
        memcpy(arm_kin, self->elements.args, sizeof(arm_kin));
        servo.tx_frame_write(0xFF, CMD_SET_KINEMATICS_PARAM, (uint8_t*)arm_kin, sizeof(arm_kin));
        save_config();
        arm.board.buzzer.set(100, 50, 1, 3000);
    }
    break;
```

16. `CMD_APRILTAG_GRAB` 在码垛工程中也作为上位机启停入口使用。`sw == 0` 时停止当前码垛、关闭蜂鸣器并刷新 OLED；`sw != 0` 时只做准备提示和界面刷新，没有直接调用 `aprilTagPalletizer.start()`，说明这个工程里真正启动码垛仍由本机按键触发。

```
case CMD_APRILTAG_GRAB:
    if (self->elements.length >= 3) {
        uint8_t sw = self->elements.args[0];
        if (sw == 0) {
            aprilTagPalletizer.stop();
            arm.board.buzzer.off();
            apply_oled_apriltag_screen();
            Serial.println("[AprilTagPallet] CMD stop");
        } else {
            arm.board.buzzer.set(30, 30, 1, 2500);
            apply_oled_apriltag_screen();
            Serial.println("[AprilTagPallet] CMD ready, press KEY1 to start");
        }
    }
```

```

    }
}
break;

```

17. `CMD_APRILTAG_SET_OFFSET` 是 Apriltag 码垛的抓取偏移标定入口。上位机传入 X、Y 和抓取 Z 位置后，代码调用 `aprilTagPalletizer.setOffsets()` 更新补偿，并把 `ag_x`、`ag_y`、`ag_z` 写入 `Preferences`，保证下次开机仍然使用标定结果。

```

case CMD_APRILTAG_SET_OFFSET:
    if (self->elements.length >= 14) {
        float x_off, y_off, z_grab;
        memcpy(&x_off, &self->elements.args[0], 4);
        memcpy(&y_off, &self->elements.args[4], 4);
        memcpy(&z_grab, &self->elements.args[8], 4);
        aprilTagPalletizer.setOffsets(x_off, y_off, z_grab);

        prefs.begin("robot_cfg", false);
        prefs.putFloat("ag_x", x_off);
        prefs.putFloat("ag_y", y_off);
        prefs.putFloat("ag_z", z_grab);
        prefs.end();

        arm.board.buzzer.set(100, 50, 2, 3000);
        Serial.printf("[AprilTagPallet] offsets x=%.1f y=%.1f z=%.1f\n", x_off,
y_off, z_grab);
    }
    break;

```

18. `at32_packet_callback()` 处理 AT32 或舵机侧返回的数据。对 Apriltag 码垛工程来说，最直接相关的是舵机位置反馈：收到 `CMD_SERVO_STREAM` 或命令 96 后，代码解析 6 个舵机位置并更新时间戳，为码垛过程中的机械臂状态同步提供数据。

```

void at32_packet_callback(PacketTypeDef* rx_packet)
{
    if (is_downloading) return;

    if (rx_packet->elements.id == 0xFF &&
        rx_packet->elements.cmd == CMD_SERVO_STREAM &&
        rx_packet->elements.length >= 14) {
        for (int i = 0; i < 6; ++i) {
            int idx = i * 2;
            g_last_servo_positions[i] = (int16_t)((rx_packet->elements.args[idx + 1] << 8) |
rx_packet->elements.args[idx]);
        }
        g_last_servo_feedback_ms = millis();
        return;
    }

    if (rx_packet->elements.id == 0xFF &&
        rx_packet->elements.cmd == 96 &&
        rx_packet->elements.length >= 14) {

```

```

        for (int i = 0; i < 6; ++i) {
            int idx = i * 2;
            g_last_servo_positions[i] = (int16_t)((rx_packet->elements.args[idx + 1] << 8) |
rx_packet->elements.args[idx]);
        }
        g_last_servo_feedback_ms = millis();
        return;
    }

```

19. 当 AT32 返回当前机械臂坐标时，代码会解析 X、Y、Z、pitch、roll、claw 和 6 个舵机位置，并写入 `arm.current_pose` 与舵机缓存。Apriltag 码垛需要在搜索、夹取、抬升和放置槽位过程中掌握机械臂状态，这些反馈数据可以帮助系统维持状态同步。

```

        if (rx_packet->elements.cmd == CMD_GET_CUR_COORDS && rx_packet->elements.length
>= 26) {
            int16_t raw_x = (int16_t)((rx_packet->elements.args[1] << 8) | rx_packet-
>elements.args[0]);
            int16_t raw_y = (int16_t)((rx_packet->elements.args[3] << 8) | rx_packet-
>elements.args[2]);
            int16_t raw_z = (int16_t)((rx_packet->elements.args[5] << 8) | rx_packet-
>elements.args[4]);
            int16_t raw_p = (int16_t)((rx_packet->elements.args[7] << 8) | rx_packet-
>elements.args[6]);
            int16_t raw_r = (int16_t)((rx_packet->elements.args[9] << 8) | rx_packet-
>elements.args[8]);
            int16_t raw_c = (int16_t)((rx_packet->elements.args[11] << 8) | rx_packet-
>elements.args[10]);

            arm.current_pose.x = (float)raw_x;
            arm.current_pose.y = (float)raw_y;
            arm.current_pose.z = (float)raw_z;
            arm.current_pose.pitch = (float)raw_p / 10.0f;
            arm.current_pose.roll = (float)raw_r;
            arm.current_pose.claw = (float)raw_c;

            for (int i = 0; i < 6; ++i) {
                int idx = 12 + i * 2;
                g_last_servo_positions[i] = (int16_t)((rx_packet->elements.args[idx + 1]
<< 8) | rx_packet->elements.args[idx]);
            }
            g_last_servo_feedback_ms = millis();
        }

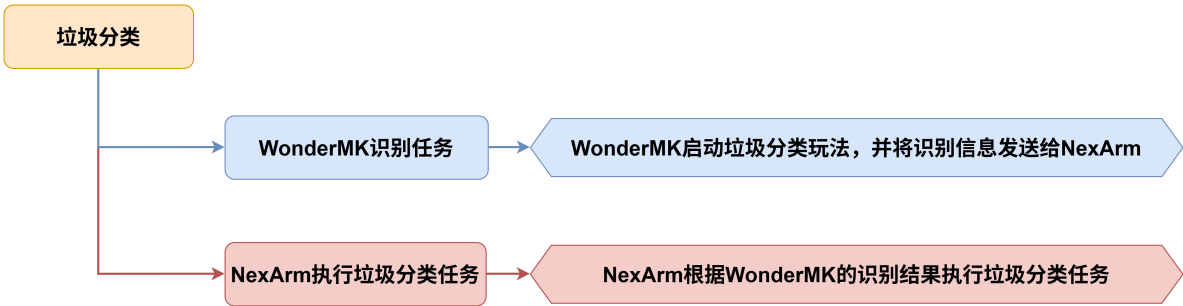
```

## 4.11 垃圾分类

### 4.11.1 项目介绍

本实验实现了基于视觉识别的垃圾分类夹取功能。系统能够识别不同垃圾类别，自动完成目标跟踪、抓取、搬运到对应投放区域并回位，形成一套完整的自动分类流程。

### 4.11.2 实现流程



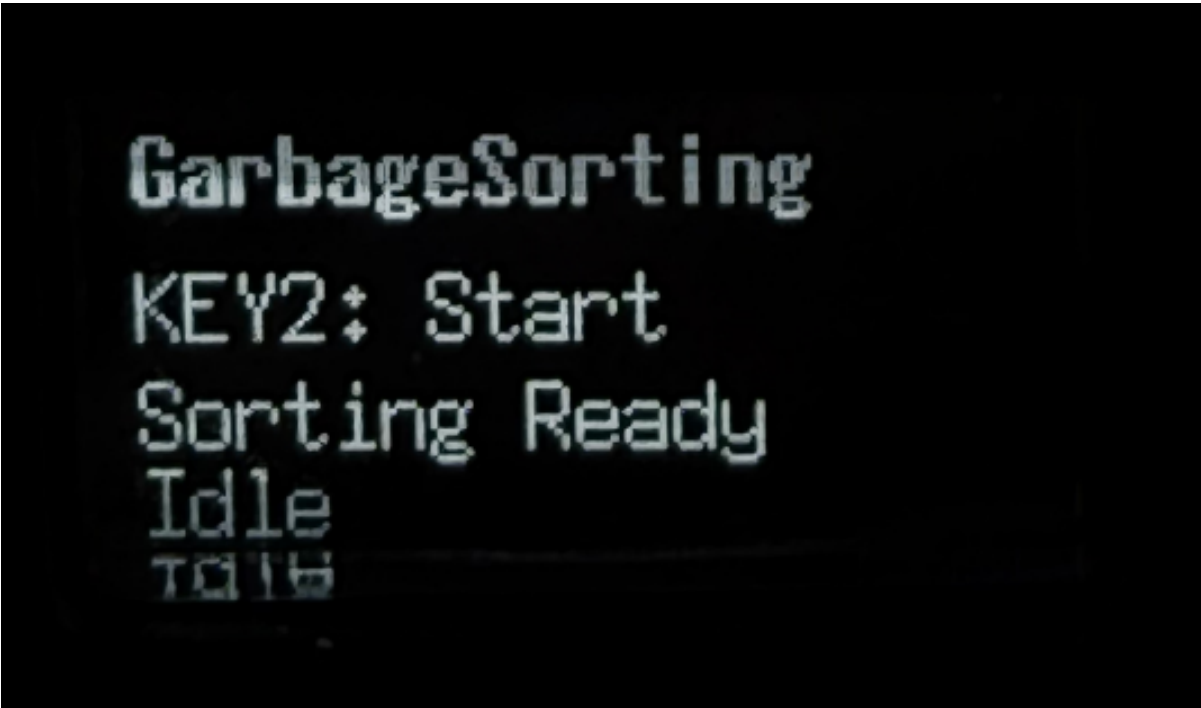
### 4.11.3 程序下载

找到本文档同目录下的“02 程序源码”文件夹，找到“09 垃圾分类”程序进行下载，具体下载步骤可参考[4.2 程序下载说明\(必看\)](#)。

### 4.11.4 运行效果

设备上电后会先完成初始化，机械臂自动回到初始等待位，屏幕上显示当前状态和固定识别阈值，随后进入待机状态。启动后，系统会持续检测画面中的垃圾目标，当识别结果达到设定阈值并且目标位置逐渐稳定在画面中心附近时，机械臂会自动下降并闭合夹爪完成抓取，然后抬升目标，移动到对应的分类投放区域，松开目标后再回到初始姿态。整个过程表现为设备能够根据识别到的垃圾类别，自动完成“识别、对位、夹取、分类放置、复位”的连续动作。

实际使用时，先正常上电并等待初始化结束，确认机械臂已经回到初始位、屏幕显示正常；将待分类目标放到摄像头可识别范围内；按下 KEY2 启动按键后，设备开始自动识别并执行分类夹取流程；一次分类完成后，机械臂会自动回到等待位置；如果需要继续演示下一次分类，只需重新放置目标并再次按下启动按键即可。



## 4.11.5 程序分析

### system\_task\_handle.cpp

1. 文件开头先引入垃圾分类工程需要的核心模块，包括系统任务接口、全局命令、机械臂控制、USB 串口控制、参数存储、垃圾分类夹取状态机、AT32 OTA 和文件系统等。这里最关键的是 `GarbageGrabber.h`，它负责和 K230 视觉侧通信，并根据识别到的垃圾目标执行搜索、对准、下探、夹取、抬升和分类放置流程；`Robot_Arm.h` 负责机械臂和主板外设；`usb_ctrl.h` 负责上位机串口命令入口。

```
#include "system_task_handle.h"
#include "Global.h"
#include "Robot_Arm.h"
#include "usb_ctrl.h"
#include <Preferences.h>
#include "GarbageGrabber.h"
#include "AT32_OTA.h"
#include <LittleFS.h>
#include <math.h>
#include <string.h>

#define CMD_SERVO_STREAM 0x68
```

2. 这些静态对象和全局变量保存垃圾分类运行期间需要共享的系统状态。`loop_with_sys_task` 用于注册周期事件，`at32_protocol` 用来解析 AT32 侧机械臂反馈，`prefs` 用来保存机械臂参数和夹取偏移，`is_downloading` 用来在动作组下载时暂停常规任务，`g_last_servo_positions` 和 `g_last_servo_feedback_ms` 则缓存最近一次舵机反馈。

```
static const char* TAG = "garbage_sorting";
static TimerHandle_t TIMER;
static esp_event_loop_handle_t loop_with_sys_task;
static CommProtocol_t at32_protocol;
static Preferences prefs;
static bool is_downloading = false;

static int16_t g_last_servo_positions[6] = {2048, 2048, 2048, 2048, 2048, 2048};
static uint32_t g_last_servo_feedback_ms = 0;

uint8_t global_channel = 2;
uint8_t global_acc = 245;
volatile bool servo_uart_busy = false;

static float arm_kin[9] = {
    110.45f, 225.00f, 36.97f, 145.00f, 0.0f, 130.23f, 0.0f, 50.0f, 70.5f
};
```

3. 这里定义了垃圾分类工程的运行状态标记。`s_was_tracker_busy` 用来判断垃圾分类任务是否刚从运行状态回到空闲状态，`s_button_enable_time` 和 `s_buttons_armed` 用于开机后的按键延迟使能，避免上电过程中误触发分类任务启动或停止。

```
static bool s_was_tracker_busy = false;
static uint32_t s_button_enable_time = 0;
static bool s_buttons_armed = false;

ESP_EVENT_DEFINE_BASE(SYS_TIMING_EVENTS);
```

4. `apply_oled_garbage_screen()` 负责刷新垃圾分类工程的 OLED 界面。`garbageGrabber.isBusy()` 为真时显示正在分类和搜索状态，空闲时显示启动入口、`Sorting Ready` 和 `Idle`，让用户能从屏幕判断当前是否可以启动垃圾分类流程。

```
static void apply_oled_garbage_screen(void)
{
    arm.board.oled.set_custom_text(0, "GarbageSorting");

    if (garbageGrabber.isBusy()) {
        arm.board.oled.set_custom_text(1, "Sorting");
        arm.board.oled.set_custom_text(2, "Status: Search");
        arm.board.oled.set_custom_text(3, "KEY2: Stop");
    } else {
        arm.board.oled.set_custom_text(1, "KEY2: Start");
        arm.board.oled.set_custom_text(2, "Sorting Ready");
        arm.board.oled.set_custom_text(3, "Idle");
    }

    arm.board.oled.show_custom();
}
```

5. `perform_arm_poweron_reset()` 是垃圾分类启动前的机械臂上电复位流程。它先在 OLED 显示回零提示，再调用 `arm.reset_all(2500)` 让机械臂回到初始姿态，并在等待过程中持续抽取 AT32 反馈，保证进入分类运行前机械臂状态已经可用。

```
static void perform_arm_poweron_reset(void)
{
    arm.board.oled.set_custom_text(0, "GarbageSorting");
    arm.board.oled.set_custom_text(1, "Arm homing...");
    arm.board.oled.set_custom_text(2, "Please wait");
    arm.board.oled.set_custom_text(3, "");
    arm.board.oled.show_custom();

    arm.reset_all(2500);
    uint32_t t0 = millis();
    while (millis() - t0 < 2600) {
        pump_at32_feedback();
        delay(20);
    }
    for (int i = 0; i < 8; i++) {
        arm.update_status();
        delay(50);
        while (servo.uart->available()) {
            uint8_t c = servo.uart->read();
            at32_protocol.parsing(&c, 1);
        }
    }
}
```

```

    }
}
Serial.println("[GarbageSorting] Power-on arm reset done");
}

```

6. `reset_arm_after_stop()` 是垃圾分类任务被手动停止后的机械臂复位流程。它先显示停止和复位提示，再调用 `arm.reset_all(2000)`，等待期间持续处理 AT32 反馈和蜂鸣器刷新，最后补充更新机械臂状态，确保中途停止后机械臂回到安全姿态。

```

static void reset_arm_after_stop(void)
{
    arm.board.oled.set_custom_text(0, "GarbageSorting");
    arm.board.oled.set_custom_text(1, "Stopping...");
    arm.board.oled.set_custom_text(2, "Arm reset");
    arm.board.oled.set_custom_text(3, "");
    arm.board.oled.show_custom();

    arm.reset_all(2000);
    uint32_t t0 = millis();
    while (millis() - t0 < 2100) {
        pump_at32_feedback();
        arm.board.buzzer.update();
        delay(20);
    }

    for (int i = 0; i < 6; i++) {
        arm.update_status();
        delay(40);
        while (servo.uart->available()) {
            uint8_t c = servo.uart->read();
            at32_protocol.parsing(&c, 1);
        }
    }

    Serial.println("[GarbageSorting] Arm reset after KEY1 stop");
}

```

7. `start_garbage_sorting()` 是本地启动垃圾分类的函数。它只在 `GarbageGrabber` 空闲时执行，启动前先调用 `garbageGrabber.stop()` 清理旧状态，再调用 `garbageGrabber.start()` 进入分类夹取状态机，随后根据 `garbageGrabber.isBusy()` 判断启动是否成功并给出蜂鸣器提示。

```

static void start_garbage_sorting(void)
{
    if (garbageGrabber.isBusy()) {
        Serial.println("[GarbageSorting] Busy, ignore KEY1 start");
        return;
    }

    garbageGrabber.stop();
    delay(100);
    garbageGrabber.start();
}

```

```

    if (garbageGrabber.isBusy()) {
        arm.board.buzzer.set(100, 100, 2, 2000);
        Serial.println("[GarbageSorting] KEY1 -> start");
    } else {
        arm.board.buzzer.set(300, 100, 1, 1000);
        Serial.println("[GarbageSorting] start failed");
    }

    apply_oled_garbage_screen();
}

```

8. `stop_garbage_sorting()` 负责停止正在运行的垃圾分类任务。函数先判断 `garbageGrabber.isBusy()`，只有任务确实在运行时才停止状态机；如果 `reset_arm` 为真，还会调用 `reset_arm_after_stop()` 让机械臂复位，保证中途停止后机械臂不会停留在抓取或放置过程中的危险姿态。

```

static void stop_garbage_sorting(bool reset_arm = false)
{
    if (!garbageGrabber.isBusy()) {
        return;
    }

    garbageGrabber.stop();
    arm.board.buzzer.set(80, 80, 1, 1800);
    if (reset_arm) {
        reset_arm_after_stop();
    }
    apply_oled_garbage_screen();
    Serial.println(reset_arm ? "[GarbageSorting] KEY1 -> stop reset" : "[GarbageSorting] stop");
}

```

9. `buttons_ready()` 是开机后的按键保护函数。系统刚启动时会先读取并清掉两个按键的点击状态，在 `s_button_enable_time` 到达之前都不允许按键触发；时间到达后把 `s_buttons_armed` 置为真，后续按键才真正生效。

```

static bool buttons_ready(void)
{
    if (s_buttons_armed) {
        return true;
    }

    (void)arm.board.button.is_clicked(0);
    (void)arm.board.button.is_clicked(1);

    if (millis() < s_button_enable_time) {
        return false;
    }

    s_buttons_armed = true;
    Serial.println("[GarbageSorting] Buttons armed");
}

```

```

    return false;
}

```

10. `save_config()` 保存基础运行参数，包括无线通道、全局加速度和机械臂运动学参数。垃圾分类需要机械臂准确完成搜索、夹取、抬升和分类放置，因此运动学参数会随上位机标定结果一起持久化。

```

void save_config(void)
{
    prefs.begin("robot_cfg", false);
    prefs.putUChar("channel", global_channel);
    prefs.putUChar("acc", global_acc);
    prefs.putBytes("arm_kin", arm_kin, sizeof(arm_kin));
    prefs.end();
}

```

11. `load_config()` 加载机械臂基础参数，并额外恢复垃圾分类的抓取偏移。这里读取 `ag_x`、`ag_y`，但把 `ag_z` 固定为 `60.0f` 后传给 `garbageGrabber.setOffsets()`，说明该工程抓取高度使用固定值，而 X/Y 偏移可通过标定持久化。

```

void load_config(void)
{
    prefs.begin("robot_cfg", true);
    global_channel = prefs.getUChar("channel", 2);
    global_acc = prefs.getUChar("acc", 245);

    if (prefs.isKey("arm_kin")) {
        float saved_kin[9];
        prefs.getBytes("arm_kin", saved_kin, sizeof(saved_kin));
        if (fabsf(saved_kin[0] - arm_kin[0]) < 0.01f) {
            memcpy(arm_kin, saved_kin, sizeof(arm_kin));
        }
    }

    if (prefs.isKey("ag_x") && prefs.isKey("ag_y") && prefs.isKey("ag_z")) {
        float ag_x = prefs.getFloat("ag_x", 15.0f);
        float ag_y = prefs.getFloat("ag_y", 50.0f);
        float ag_z = 60.0f;
        garbageGrabber.setOffsets(ag_x, ag_y, ag_z);
    }
    prefs.end();
}

```

12. 周期定时器回调负责投递垃圾分类运行时需要的基础维护事件。它不会直接操作硬件，而是按周期投递蜂鸣器、电池、机械臂状态和 OLED 刷新事件；如果正在下载动作组，则直接返回，避免下载流程被周期任务干扰。

```

static void sys_timer_post_callback(TimerHandle_t xTimer)
{
    if (is_downloading) return;

    static uint32_t count = 0;
    if (count % BUZZER_UPDATE_PERIOD == 0) {

```

```

        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS, TIMING_EVENT_BUZZER_UPDATE,
NULL, 0, 0);
    }
    if (count % BAT_UPDATE_PERIOD == 0) {
        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS, TIMING_EVENT_BAT_UPDATE,
NULL, 0, 0);
    }
    if (count % SERVO_STATUS_UPDATE_PERIOD == 0) {
        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS,
TIMING_EVENT_SERVO_STATUS_UPDATE, NULL, 0, 0);
    }
    if (count % OLED_REFRESH_PERIOD == 0) {
        esp_event_post_to(loop_with_sys_task, SYS_TIMING_EVENTS, TIMING_EVENT_OLED_REFRESH,
NULL, 0, 0);
    }

    count += TIMER_PERIOD;
}

```

13. `sys_timer_sub_handler()` 接收周期事件并执行具体维护动作。蜂鸣器和电池是主板基础状态维护，`arm.update_status()` 用于持续刷新机械臂状态，`apply_oled_garbage_screen()` 则让 OLED 根据 `GarbageGrabber` 当前状态显示空闲或正在分类搜索。

```

static void sys_timer_sub_handler(void* handler_args, esp_event_base_t base, int32_t id,
void* event_data)
{
    if (is_downloading) return;

    switch (id) {
        case TIMING_EVENT_BUZZER_UPDATE:
            arm.board.buzzer.update();
            break;
        case TIMING_EVENT_BAT_UPDATE:
            arm.board.bat.update();
            break;
        case TIMING_EVENT_SERVO_STATUS_UPDATE:
            arm.update_status();
            break;
        case TIMING_EVENT_OLED_REFRESH:
            apply_oled_garbage_screen();
            break;
        default:
            break;
    }
}

```

14. `func_ctrl_callback()` 是上位机串口命令入口。和垃圾分类主流程直接相关的是动作组下载保护、机械臂运动学参数设置、`CMD_GARBAGE_GRAB` 启停和阈值控制，以及 `CMD_APRILTAG_SET_OFFSET` 抓取偏移标定；动作组下载时会把 `is_downloading` 置为真，让主循环暂停垃圾分类状态机和按键处理。

```

static void func_ctrl_callback(PacketTypeDef* self)

```

```

{
    uint8_t len;
    uint16_t bat_level;

    if (self->elements.id == 0xFF && self->elements.cmd == CMD_ACTION_GROUP_DOWNLOAD) {
        is_downloading = true;
        arm.board.action_group_download(self->elements.args[0], self->elements.args, self->elements.length - 2);
        is_downloading = false;
        return;
    }

    if (self->elements.id != 0xFF) return;

    switch (self->elements.cmd) {

```

15. `CMD_SET_KINEMATICS_PARAM` 用于从上位机更新机械臂运动学参数。垃圾分类需要机械臂准确到达搜索、夹取、抬升和不同分类放置位置，所以参数更新后会写入本地 `arm_kin`、转发给舵机控制侧，并调用 `save_config()` 保存到 NVS。

```

case CMD_SET_KINEMATICS_PARAM:
    if (self->elements.length - 2 >= (int)sizeof(arm_kin)) {
        memcpy(arm_kin, self->elements.args, sizeof(arm_kin));
        servo.tx_frame_write(0xFF, CMD_SET_KINEMATICS_PARAM, (uint8_t*)arm_kin,
sizeof(arm_kin));
        save_config();
        arm.board.buzzer.set(100, 50, 1, 3000);
    }
    break;

```

16. `CMD_GARBAGE_GRAB` 是上位机控制垃圾分类的主要命令。命令参数中如果带有 1 到 100 的阈值，会先调用 `garbageGrabber.setThreshold()` 更新识别置信度阈值；`sw == 0` 时停止分类，`sw != 0` 时会直接重启并启动 `garbageGrabber.start()`，这和前几个工程中“只提示按键启动”的逻辑不同。

```

case CMD_GARBAGE_GRAB:
    if (self->elements.length >= 3) {
        uint8_t sw = self->elements.args[0];
        if (self->elements.length >= 4) {
            uint8_t thresh = self->elements.args[1];
            if (thresh > 0 && thresh <= 100) {
                garbageGrabber.setThreshold(thresh);
            }
        }

        if (sw == 0) {
            garbageGrabber.stop();
            arm.board.buzzer.off();
            apply_oled_garbage_screen();
            Serial.println("[GarbageSorting] CMD stop");
        } else {
            garbageGrabber.start();
        }
    }

```

```

        delay(100);
        garbageGrabber.start();
        arm.board.buzzer.set(100, 100, 2, 2000);
        apply_oled_garbage_screen();
        Serial.println("[GarbageSorting] CMD start");
    }
}
break;

```

17. `CMD_APRILTAG_SET_OFFSET` 在垃圾分类工程中被复用为抓取偏移标定入口。上位机传入 X、Y 和 Z 后，代码会把 Z 固定为 60.0f，再调用 `garbageGrabber.setOffsets()` 更新补偿，并把 `ag_x`、`ag_y`、`ag_z` 写入 `Preferences`，保证下次开机仍然使用标定结果。

```

case CMD_APRILTAG_SET_OFFSET:
    if (self->elements.length >= 14) {
        float x_off, y_off, z_grab;
        memcpy(&x_off, &self->elements.args[0], 4);
        memcpy(&y_off, &self->elements.args[4], 4);
        memcpy(&z_grab, &self->elements.args[8], 4);
        z_grab = 60.0f;
        garbageGrabber.setOffsets(x_off, y_off, z_grab);

        prefs.begin("robot_cfg", false);
        prefs.putFloat("ag_x", x_off);
        prefs.putFloat("ag_y", y_off);
        prefs.putFloat("ag_z", z_grab);
        prefs.end();

        arm.board.buzzer.set(100, 50, 2, 3000);
        Serial.printf("[GarbageSorting] offsets x=%.1f y=%.1f z=%.1f\n", x_off,
y_off, z_grab);
    }
    break;

```

18. `at32_packet_callback()` 处理 AT32 或舵机侧返回的数据。对垃圾分类工程来说，最直接相关的是舵机位置反馈：收到 `CMD_SERVO_STREAM` 或命令 96 后，代码解析 6 个舵机位置并更新时间戳，为分类夹取过程中的机械臂状态同步提供数据。

```

void at32_packet_callback(PacketTypeDef* rx_packet)
{
    if (is_downloading) return;

    if (rx_packet->elements.id == 0xFF &&
        rx_packet->elements.cmd == CMD_SERVO_STREAM &&
        rx_packet->elements.length >= 14) {
        for (int i = 0; i < 6; ++i) {
            int idx = i * 2;
            g_last_servo_positions[i] = (int16_t)((rx_packet->elements.args[idx + 1] << 8) |
rx_packet->elements.args[idx]);
        }
        g_last_servo_feedback_ms = millis();
    }
}

```

```

        return;
    }

    if (rx_packet->elements.id == 0xFF &&
        rx_packet->elements.cmd == 96 &&
        rx_packet->elements.length >= 14) {
        for (int i = 0; i < 6; ++i) {
            int idx = i * 2;
            g_last_servo_positions[i] = (int16_t)((rx_packet->elements.args[idx + 1] << 8) |
rx_packet->elements.args[idx]);
        }
        g_last_servo_feedback_ms = millis();
        return;
    }

```

19. 当 AT32 返回当前机械臂坐标时，代码会解析 X、Y、Z、pitch、roll、claw 和 6 个舵机位置，并写入 `arm.current_pose` 与舵机缓存。垃圾分类需要在搜索、下探、夹取、抬升和分类放置过程中掌握机械臂状态，这些反馈数据可以帮助系统维持状态同步。

```

        if (rx_packet->elements.cmd == CMD_GET_CUR_COORDS && rx_packet->elements.length
>= 26) {
            int16_t raw_x = (int16_t)((rx_packet->elements.args[1] << 8) | rx_packet-
>elements.args[0]);
            int16_t raw_y = (int16_t)((rx_packet->elements.args[3] << 8) | rx_packet-
>elements.args[2]);
            int16_t raw_z = (int16_t)((rx_packet->elements.args[5] << 8) | rx_packet-
>elements.args[4]);
            int16_t raw_p = (int16_t)((rx_packet->elements.args[7] << 8) | rx_packet-
>elements.args[6]);
            int16_t raw_r = (int16_t)((rx_packet->elements.args[9] << 8) | rx_packet-
>elements.args[8]);
            int16_t raw_c = (int16_t)((rx_packet->elements.args[11] << 8) | rx_packet-
>elements.args[10]);

            arm.current_pose.x = (float)raw_x;
            arm.current_pose.y = (float)raw_y;
            arm.current_pose.z = (float)raw_z;
            arm.current_pose.pitch = (float)raw_p / 10.0f;
            arm.current_pose.roll = (float)raw_r;
            arm.current_pose.claw = (float)raw_c;

            for (int i = 0; i < 6; ++i) {
                int idx = 12 + i * 2;
                g_last_servo_positions[i] = (int16_t)((rx_packet->elements.args[idx + 1]
<< 8) | rx_packet->elements.args[idx]);
            }
            g_last_servo_feedback_ms = millis();
        }

```

20. `pump_at32_feedback()`、`sync_arm_feedback()` 和 `get_last_servo_positions()` 是机械臂反馈辅助函数。它们分别负责从 AT32 串口抽取原始数据、等待新的机械臂反馈、向其他模块提供最近一次舵机位置缓存，保证垃圾分类运行时不会只依赖过期状态。

```

void pump_at32_feedback(void)
{
    while (servo.uart->available()) {
        uint8_t c = servo.uart->read();
        at32_protocol.parsing(&c, 1);
    }
}

bool sync_arm_feedback(uint32_t timeout_ms)
{
    arm.update_status();
    uint32_t start = millis();
    uint32_t prev_feedback_ms = g_last_servo_feedback_ms;

    while (millis() - start < timeout_ms) {
        serial_port.rec_handler();
        pump_at32_feedback();
        if (g_last_servo_feedback_ms != 0 && g_last_servo_feedback_ms != prev_feedback_ms) {
            return true;
        }
        delay(2);
    }

    return g_last_servo_feedback_ms != 0;
}

bool get_last_servo_positions(int16_t out_pos[6])
{
    if (g_last_servo_feedback_ms == 0) {
        return false;
    }
    memcpy(out_pos, g_last_servo_positions, sizeof(g_last_servo_positions));
    return true;
}

```

21. `system_loop_handler()` 是垃圾分类运行阶段的主循环。非下载状态下，它依次处理上位机串口、AT32 反馈和 `garbageGrabber.update()`；随后检测分类任务是否刚从运行状态回到空闲状态，如果已经停止，就刷新 OLED 回到待机界面。

```

void system_loop_handler(void)
{
    if (!is_downloading) {
        serial_port.rec_handler();
        pump_at32_feedback();

        garbageGrabber.update();

        bool busy_now = garbageGrabber.isBusy();
        if (s_was_tracker_busy && !busy_now) {
            Serial.println("[GarbageSorting] GarbageSorting stopped, idle");
            apply_oled_garbage_screen();
        }
    }
}

```

```
s_was_tracker_busy = busy_now;
```

```
arm.board.button.update();
```

```
arm.board.buzzer.update();
```